

Binius64 Blueprint

August 15, 2026

Abstract

This document specifies Binius64, a hash-based zero-knowledge succinct non-interactive argument of knowledge (ZK-SNARK) tailored to computations expressed over 64-bit machine words. Binius64 arithmetizes a computation as a circuit built from four native constraint forms—vanishing, bitwise AND, 64-bit unsigned integer multiplication, and multiplication in the GHASH field $\mathbb{F}_{2^{128}}$ —acting on XOR-accumulations of bit-shifted words. This word-level constraint language matches the instruction set of modern processors and avoids the roughly 64-fold blowup that bit-level circuit encodings incur. The protocol works entirely in characteristic 2 and draws its verifier challenges from the GHASH field $\mathbb{F}_{2^{128}}$, so that the prover’s field arithmetic is carried by the carryless-multiplication instructions (CLMUL, PMULL) already present on x86-64 and ARM64 processors for the AES-GCM and GMAC modes. Binius64 has a transparent setup, rests only on the collision resistance of a hash function modeled as a random oracle, and is plausibly post-quantum secure. Its proofs are succinct in size. We obtain zero knowledge by composing the core protocol with a Spartan-based outer argument, and treat the construction in full. This document is intended as the canonical protocol specification: it develops the construction in detail while deferring the proofs of standard cryptographic primitives to the peer-reviewed literature.

Contents

1	Introduction	1
1.1	Motivation	1
1.2	Why Not Binary Spartan?	2
1.3	Outline of the Construction	2
2	Preliminaries	3
2.1	Binary Fields	4
2.2	Sumcheck	5
2.2.1	Univariate Skip	6
2.3	Linear Interactive Oracle Proofs	7
3	The Constraint System	8
3.1	Prover Data Layout	8
3.2	Shifted Value Indices	9
3.3	Constraint Arrays	10
3.4	Zero Constraints	10
3.5	BitAnd Constraints	10
3.6	IntMul Constraints	11
3.7	BinMul Constraints	11
4	The Binius64 LIOP	11
4.1	Protocol Overview	12
4.2	Shift Indicator Polynomials	13
4.3	The BitAnd Reduction	14
4.3.1	The Rijndael Field and Deterministic Zerocheck	14
4.3.2	The Univariate-Skip Protocol	15
4.3.3	Prover Algorithm	16
4.4	The Zero Reduction	16

4.5	The IntMul Reduction	17
4.5.1	Multiplying in the Exponent	17
4.5.2	Exponentiation Subprotocols	18
4.5.3	Fixed-Base Exponentiation	19
4.5.4	Variable-Base Exponentiation	20
4.5.5	Combined Protocol	20
4.5.6	Prover Algorithm	22
4.6	The BinMul Reduction	23
4.6.1	Packing the Operands	23
4.6.2	The Zerocheck	23
4.6.3	Prover Algorithm	24
4.7	The Shift Reduction	24
4.7.1	Unifying the Output Claims	24
4.7.2	Mathematizing Constraint Arrays	25
4.7.3	Prover Algorithm	26
4.8	Ring-Switching to the Packed Witness	28
4.9	Public Input Check	30
5	Basic SNARK Compiler	30
5.1	Compiling LIOPs to SNARKs	31
5.2	BaseFold	31
5.3	Lifted BaseFold	32
5.4	Batched BaseFold	33
5.5	The Binius64 SNARK	34
6	The IronSpartan LIOP	34
6.1	RICS and the Spartan PIOP	34
6.2	IronSpartan Overview	35
6.3	Zero-Knowledge Sumcheck Machinery	36
6.3.1	Libra and Its Failure in Characteristic 2	37
6.3.2	Zero-Knowledge Multilinear-Extension Check	38
6.4	Dummy-Constraint Masking	39
6.5	The IronSpartan LIOP	39
6.5.1	Constraint-Check Reduction	39
6.5.2	Reduction to Inner-Product Claims on Matrix Extensions	39
7	Zero-Knowledge Compiler	40
7.1	Proof Composition for ZK-SNARKs	40
7.2	Batched ZK BaseFold	41
7.3	Compiling Binius64 to a ZK-SNARK	42
A	Expressing Common Operations	43
B	Arithmetization of Shift Indicators	43
B.1	Logical Shifts and Rotation	43
B.2	Arithmetic Right Shift	44
B.3	Parallel 32-bit Shifts	45
C	Batched Product Checks of Unequal Depths	45
C.1	Setting	45
C.2	One-Padding	46
C.3	The Protocol	46
C.4	Prover Algorithm	48
D	Zero-Knowledge of the Sumcheck Machinery	49

1 Introduction

1.1 Motivation

A *succinct non-interactive argument of knowledge*, or SNARK, lets a verifier check the result of a computation far more cheaply than re-executing it, while remaining convinced—up to a negligible soundness error—that the prover holds a witness for which the computation was carried out correctly. SNARKs are the central primitive of *verifiable computing* [Set20]: a small, trusted device can audit the output of an untrusted, possibly faulty, and far more powerful one. When the argument additionally reveals nothing about the witness beyond the truth of the statement, it is *zero-knowledge*, the property that underpins privacy-preserving applications such as verifiable credentials and confidential transactions.

Many of the computations one most wants to prove—hash functions such as SHA-256, SHA-3, and Blake3, block ciphers, and the bit manipulation at the core of cryptographic primitives—are naturally expressed as operations on machine words: 64-bit XOR, AND, shifts, rotations, and integer multiplication. General-purpose SNARKs built over large prime fields can prove such operations only after an expensive bit-by-bit decomposition, and even most binary-field systems pay a cost per individual bit. Binius64 instead takes the 64-bit word as its unit of arithmetization. The constraint system (§3) offers exactly four native constraint forms—*Zero*, the vanishing of an XOR-accumulation of shifted words; *BitAnd*, a bitwise AND between two such accumulations; *IntMul*, an unsigned 64-bit integer multiplication; and *BinMul*, a multiplication in the GHASH field $\mathbb{F}_{2^{128}}$ —and derives the remaining common operations (NOT, OR, integer addition and subtraction; §A) from them. The key device is the *shifted value index*, which fuses a reference to a prover word with a pair of shift operations applied in sequence, so that shifts and rotations are absorbed into the operands of the AND and multiplication constraints rather than charged as separate constraints. Operating on words rather than bits reduces the number of constraints by up to a factor of 64 relative to a bit-level encoding, while retaining the efficiency of binary-field arithmetic.

That efficiency is rooted in the choice of field. Binius64 works entirely in characteristic 2. Binary-field addition is bitwise XOR; the prime field $\mathbb{F}_2$ is the natural home of a single bit; and—decisively for performance—multiplication in the GHASH field $\mathbb{F}_{2^{128}}$ is accelerated in hardware on precisely the processors we target. The carryless-multiplication instructions CLMUL on x86-64 and PMULL on ARM64, shipped to support the AES-GCM authenticated-encryption mode and the GMAC message-authentication code, supply exactly the field multiplication the prover needs. We accordingly take $K := \mathbb{F}_{2^{128}}$, with security parameter $\lambda = 128$, as the field from which the verifier draws its random challenges (§2.1). The protocol is designed throughout for high throughput on commodity x86-64 and ARM64 CPUs, including the SIMD units of modern mobile processors, which makes it well suited to local proving.

The construction is hash-based. Its only cryptographic assumption is the collision resistance of a hash function, modeled in the security analysis as a random oracle; there is no trusted setup and no structured reference string, and the scheme is plausibly secure against quantum adversaries. These are conservative assumptions by the standards of the SNARK literature, and they are inherited directly from the polynomial commitment layer rather than bolted on.

At present, Binius64 is succinct in *proof size*: a proof is polylogarithmic in the size of the computation being proved. Verification is not yet succinct—the verifier reads the circuit description in the course of checking a proof—and reducing verification cost is ongoing work. We are careful to state this distinction precisely where it matters (§5.5); the generic compiler of §5 is succinct in both proof size and verifier time whenever the source protocol’s queries are succinct, a condition the Binius64 verifier does not currently meet because it evaluates the constraint matrices directly.

1.2 Why Not Binary Spartan?

A natural way to build a multilinear SNARK over binary fields would be to adapt Spartan [Set20] to $\mathbb{F}_2$. The witness becomes a single multilinear polynomial over $\mathbb{F}_2$, which the binary-field polynomial commitment of Diamond and Posen [DP24] commits efficiently, and the rank-one constraints are checked by a sumcheck. The approach is appealing, but it founders on the *wiring check*. A rank-one constraint system over n witness bits has matrices A, B, C with $O(n)$ nonzero entries, and the prover’s partial evaluation of these matrices entails $O(n)$ multiplications between elements of the large challenge

field K and bits of $\mathbb{F}_2$. With $\lambda = \log_2 |K|$ the security parameter, this is $O(\lambda n)$ bit operations, and proving the sparse-matrix openings incurs a second factor of λ . The large field infects what ought to be a bit-level computation.

Binius64 removes this bottleneck by working at word granularity. Packing 64 bits into a word shrinks the constraint-matrix entry count by up to $64\times$; modeling the word width as scaling with λ , the prover’s wiring cost falls from $O(\lambda n)$ back to $O(n)$ bit operations. The native BitAnd, IntMul, and BinMul constraints, together with the shifted value indices, are what let the system express realistic computations at this granularity without reintroducing per-bit overhead.

We do nonetheless return to Spartan, in a different guise. The zero-knowledge construction of §7 uses *IronSpartan* (§6), a commit-and-prove adaptation of Spartan, as an *outer* argument—applied not to the word-level witness but to the small circuit that describes the Binius64 verifier. There the assignment is already polylogarithmic in the size of the original computation, so the $O(\lambda n)$ bottleneck above never arises, and Spartan’s simplicity and direct support for zero knowledge become exactly the right tools.

1.3 Outline of the Construction

The document develops Binius64 in layers, separating the information-theoretic protocol from its cryptographic compilation.

§2 fixes notation and recalls the building blocks: multilinear polynomials and the equality indicator, the binary fields used (the prime field $\mathbb{F}_2$, the Rijndael field $\mathbb{F}_{2^8}$, and the challenge field $K = \mathbb{F}_{2^{128}}$), the sumcheck protocol and its variants (the MLE-check, the zerocheck, batching, and the univariate skip), and the abstraction of a *linear interactive oracle proof* (LIOP) that organizes the rest of the document.

§3 defines the constraint system as a self-contained combinatorial object: prover data laid out as 64-bit words, the eight shift operations and the shifted value indices that name shifted words, and the BitAnd, IntMul, and BinMul constraints built from XOR-accumulations of these. Appendix A shows how the everyday operations of a 64-bit instruction set reduce to this language.

§4 specifies the *Binius64 LIOP*, the heart of the protocol. The prover commits a single oracle—the packed witness—and a sequence of sumcheck-based reductions (the GKR-based IntMul reduction, the BinMul reduction, the BitAnd reduction, the shift reduction, ring-switching to the packed witness, and the public-input check) collapse satisfaction of the entire constraint system into a single linear query on that oracle.

§5 gives a generic compiler turning any LIOP into a SNARK, through the chain $\text{LIOP} \xrightarrow{\text{BaseFold}} \text{LIOP} \xrightarrow{\text{BCS}} \text{IP} \xrightarrow{\text{Fiat-Shamir}} \text{SNARK}$. Each LIOP linear query is discharged by a BaseFold [ZCF24] proximity test on a constrained Reed–Solomon code over K , the oracles are replaced by Merkle commitments via the BCS transform [BCS16], and the interaction is removed by Fiat–Shamir. Applied to the Binius64 LIOP, this yields the (non-zero-knowledge) Binius64 SNARK of §5.5.

§6 specifies the *IronSpartan LIOP*, a second, zero-knowledge LIOP for rank-one constraint systems, recast as a commit-and-prove argument. §7 then assembles the zero-knowledge SNARK by *proof composition*: the Binius64 LIOP is run as an inner protocol with its messages encrypted under one-time pads, and IronSpartan certifies in zero knowledge that the inner verifier would have accepted. The accompanying *Batched ZK BaseFold* compiler masks the polynomial-commitment layer, so that the composed argument is zero-knowledge end to end.

This is a protocol specification rather than a self-contained cryptographic treatise. Our aim is to pin down the construction precisely enough to serve as the reference for an implementation, and to fill in the design choices and engineering details that sit between the academic results we build on and a working system. Where a security property follows from established, peer-reviewed work, we cite it and state how it is used rather than reprove it; the novel reductions are developed in full.

2 Preliminaries

This section fixes notation and recalls the polynomial-algebra and protocol building blocks used throughout the document.

Boolean hypercube For $n \geq 0$, we write $\mathcal{B}_n := \{0, 1\}^n$ for the n -dimensional Boolean hypercube. We will frequently identify $\mathcal{B}_n$ with the set $\{0, \dots, 2^n - 1\}$ of nonnegative integers via the little-endian bit decomposition $i \mapsto (i_0, \dots, i_{n-1})$, where $i = \sum_{j=0}^{n-1} i_j \cdot 2^j$.

Constant vectors For a length $n \geq 0$, we write 0^n and 1^n for the all-zeros and all-ones vectors, identified with the corresponding vertices of $\mathcal{B}_n$. We use the same string-power notation for any constant block of n identical coordinates within a longer argument list.

Functions For sets S and V , the set of functions $S \rightarrow V$ is denoted V^S . We use the same symbol f both for a function $f \in V^{\mathcal{B}_n}$ and for its corresponding length- 2^n list of values; context will distinguish.

Inner product For any field K and vectors $u, v \in K^n$, we write $\langle u, v \rangle := \sum_{i=0}^{n-1} u_i \cdot v_i$ for the standard inner product.

Tensor product expansion For vectors $u \in K^a$ and $v \in K^b$, the *tensor product* $u \otimes v \in K^{a \cdot b}$ is the vector whose entry indexed by $(i, j) \in \{0, \dots, a-1\} \times \{0, \dots, b-1\}$ is $u_i \cdot v_j$, under the lexicographic enumeration of pairs. For $n \geq 0$ and $x = (x_0, \dots, x_{n-1}) \in K^n$, the *tensor product expansion* is the map $\text{TensorExpand}_n: K^n \rightarrow K^{2^n}$ defined by

$$\text{TensorExpand}_n(x) := \bigotimes_{i=0}^{n-1} (1 - x_i, x_i),$$

where each factor $(1 - x_i, x_i)$ is in K^2 . The entry of $\text{TensorExpand}_n(x)$ indexed by $w \in \mathcal{B}_n$ equals $\prod_{i=0}^{n-1} ((1 - x_i)^{1-w_i} \cdot x_i^{w_i})$, which is precisely $\tilde{\text{eq}}_n(x, w)$ for the equality indicator defined below. Thus $\text{TensorExpand}_n(x) = (\tilde{\text{eq}}_n(x, w))_{w \in \mathcal{B}_n}$ is the table of values of the partial evaluation $\tilde{\text{eq}}_n(x, \cdot)$ on the hypercube. The expansion can be computed in $O(2^n)$ field operations [Tha22, Lem. 3.8].

Multilinear polynomials For a ring R and indeterminates $X_0, \dots, X_{n-1}$, we write $R[X_0, \dots, X_{n-1}]^{\leq 1}$ for the subspace of polynomials of individual degree at most one in each indeterminate; these are the *multilinear* polynomials. The evaluation map $t \mapsto (t(u))_{u \in \mathcal{B}_n}$ is a K -linear bijection between $K[X_0, \dots, X_{n-1}]^{\leq 1}$ and $K^{\mathcal{B}_n}$; for a function $f \in K^{\mathcal{B}_n}$, we write $\tilde{f} \in K[X_0, \dots, X_{n-1}]^{\leq 1}$ for the unique multilinear whose restriction to $\mathcal{B}_n$ equals f , the *multilinear extension* of f . When f takes values in the subfield $\mathbb{F}_2 \subset K$, $\tilde{f}$ has $\mathbb{F}_2$ -coefficients. See [Tha22, Fact 3.5] for an alternative direct proof of uniqueness.

Equality indicator The *equality indicator* is the bivariate multilinear $\tilde{\text{eq}}(X, Y) := (1 - X)(1 - Y) + XY$, which in characteristic 2 simplifies to $1 + X + Y$. For $n \geq 1$ its n -fold version is the $2n$ -variate multilinear

$$\tilde{\text{eq}}_n(X, Y) := \prod_{i=0}^{n-1} \tilde{\text{eq}}(X_i, Y_i), \quad X, Y \in K^n,$$

taking value 1 on the diagonal of $\mathcal{B}_n \times \mathcal{B}_n$ and 0 elsewhere; the multilinear extension of any $f: \mathcal{B}_n \rightarrow K$ decomposes as $\tilde{f}(X) = \sum_{u \in \mathcal{B}_n} f(u) \cdot \tilde{\text{eq}}_n(u, X)$. The characteristic-2 form uses only $n-1$ multiplications, so $\tilde{\text{eq}}_n(r, r')$ can be evaluated in $O(n)$ field operations at any pair of points $r, r' \in K^n$. We abbreviate $\tilde{\text{eq}}_1$ as $\tilde{\text{eq}}$, and also write the bare symbol $\tilde{\text{eq}}$ when referring to the family generically; an *applied* indicator of arity $n \geq 2$ always carries its subscript.

Vector arguments We routinely flatten vector arguments into scalar ones: when a function g expects $a + b$ scalar inputs and $u \in K^a$, $v \in K^b$, we write $g(u, v)$ for $g(u_0, \dots, u_{a-1}, v_0, \dots, v_{b-1})$, listing the coordinates of each vector in order, and we group consecutive scalar arguments back into vectors when convenient. This abuse is most frequent for the equality indicator $\tilde{\text{eq}}_n$, whose two length- n vector arguments are often written out coordinate-wise (as in $\tilde{\text{eq}}_3(\rho_0, \rho_1, \rho_2, u_0, u_1, u_2)$).

GKR The *GKR protocol* [GKR15] is an interactive reduction that, given a layered arithmetic circuit and an evaluation claim on the multilinear extension of its output layer at a point in K^n , recursively reduces, layer by layer via sumcheck on the wiring polynomials, to evaluation claims on the multilinear extension of the input layer.

2.1 Binary Fields

We work exclusively in characteristic 2. Three binary fields recur throughout: the prime field $\mathbb{F}_2$, in which the witness lives; the *Rijndael field* $\mathbb{F}_{2^8}$, used by the prover in the BitAnd reduction (§4.3); and a cryptographically large extension field $\mathbb{F}_{2^\lambda}$, for a power-of-two security parameter λ , from which the verifier draws random challenges. We abbreviate $K := \mathbb{F}_{2^\lambda}$ in protocol specifications and elsewhere when convenient, and write K^* for its multiplicative group of units; in practice we take $\lambda = 128$. We require $8 \mid \lambda$, so that $\mathbb{F}_{2^8}$ embeds into K (see *Subfield embedding* below). The construction imposes two further divisibility conditions relating λ to the padded witness and public-segment sizes; these are stated, and arranged by padding, in §4.1.

Freshman’s dream. In characteristic 2 the binomial cross-term vanishes, so squaring is additive: $(\alpha + \beta)^2 = \alpha^2 + \beta^2$ for all $\alpha, \beta \in \mathbb{F}_{2^n}$. Iterating, $(\alpha + \beta)^{2^j} = \alpha^{2^j} + \beta^{2^j}$ for every $j \geq 0$. This is the elementary form of the Frobenius linearity revisited below, and underlies the Frobenius-twisting manipulations of §4.5.

Construction. For any $n \geq 1$, fixing an irreducible polynomial $f(X) \in \mathbb{F}_2[X]$ of degree n , the quotient ring $\mathbb{F}_2[X]/(f(X))$ is a field of size 2^n , which we denote $\mathbb{F}_{2^n}$. Each $\alpha \in \mathbb{F}_{2^n}$ is represented as a polynomial of degree less than n , so the monomials $1, X, \dots, X^{n-1}$ form an $\mathbb{F}_2$ -basis of $\mathbb{F}_{2^n}$; concretely, elements of $\mathbb{F}_{2^n}$ are n -bit strings, and those of $K = \mathbb{F}_{2^\lambda}$ are λ -bit strings. Two binary fields from cryptographic practice serve as running examples: the *Rijndael field* $\mathbb{F}_{2^8} \cong \mathbb{F}_2[X]/(X^8 + X^4 + X^3 + X + 1)$, the base field of the AES block cipher, and the *GHASH field* $\mathbb{F}_{2^{128}} \cong \mathbb{F}_2[X]/(X^{128} + X^7 + X^2 + X + 1)$, used in the AES-GCM authenticated-encryption mode and the GMAC message-authentication code.

Subfield embedding. When $m \mid n$, the field $\mathbb{F}_{2^m}$ occurs inside $\mathbb{F}_{2^n}$ as the unique subfield of size 2^m . We exploit this for the Rijndael field $\mathbb{F}_{2^8}$, which embeds into K under the assumption $8 \mid \lambda$ via a fixed embedding $\iota: \mathbb{F}_{2^8} \hookrightarrow K$. We refer to [DP24] for further background on representing and computing in binary fields.

Algebraic properties. The multiplicative group $\mathbb{F}_{2^n}^*$ is cyclic, so we may fix a generator $g \in \mathbb{F}_{2^n}^*$ whose powers g^a , for $a \in \{0, \dots, 2^n - 2\}$, exhaust $\mathbb{F}_{2^n}^*$. The *Frobenius endomorphism* $\varphi: \mathbb{F}_{2^n} \rightarrow \mathbb{F}_{2^n}$ sending $\alpha \mapsto \alpha^2$ is $\mathbb{F}_2$ -linear, and in finite fields it is invertible. The n -fold iterate φ^n acts as the identity on $\mathbb{F}_{2^n}$; we use this in §4.5 to manipulate evaluation claims involving Frobenius twists on K .

2.2 Sumcheck

The *sumcheck protocol* [LFKN92; Tha22] is an interactive reduction that transforms a sum claim $s \stackrel{?}{=} \sum_{v \in \mathcal{B}_n} F(v)$ over a polynomial $F \in K[X_0, \dots, X_{n-1}]$ into a single evaluation claim $s' \stackrel{?}{=} F(r)$ at a verifier-chosen point $r \in K^n$, with soundness error $n \cdot d/|K|$, where d is the maximum individual degree of F . In our applications F is a *composite of multilinear*,

$$F(X) = C(\tilde{f}_0(X), \dots, \tilde{f}_{m-1}(X)),$$

for multilinear $\tilde{f}_0, \dots, \tilde{f}_{m-1} \in K[X_0, \dots, X_{n-1}]^{\leq 1}$ and a *composition polynomial* C of total degree d (so F has individual degree at most d). For such F the prover runs in $O(2^n \cdot m \cdot d)$ field operations and the verifier in $O(n \cdot d)$ time [Gru24, §3]. The reduction is given in Protocol 2.1; several variants recur throughout the document. We bind variables from the highest index to the lowest—in round i binding X_{n-i-1} and sampling the challenge r_{n-i-1} —a convention we use throughout by default, as it lends itself to a more SIMD-efficient prover algorithm.

PROTOCOL 2.1 (Sumcheck). Claim: $\sum_{x \in \mathcal{B}_n} F(x) = s$, for $F \in K[X_0, \dots, X_{n-1}]$ of individual degree $\leq d$.

Rounds: Set $s^{(0)} := s$. For $i = 0, \dots, n-1$, write $k := n-i-1$ (so round i binds X_k):

1. The prover sends the *round polynomial*

$$R^{(i)}(X) := \sum_{v \in \mathcal{B}_k} F(v, X, r_{k+1}, \dots, r_{n-1}) \in K[X], \quad \deg R^{(i)} \leq d,$$

where v ranges over the low variables $X_0, \dots, X_{k-1}$ and $r_{k+1}, \dots, r_{n-1}$ are the previously sampled challenges; d of its $d+1$ coefficients are transmitted, the verifier recovering the last from the check below.

2. The verifier checks $R^{(i)}(0) + R^{(i)}(1) = s^{(i)}$, samples $r_k \leftarrow K$, and sets $s^{(i+1)} := R^{(i)}(r_k)$.

Output: the evaluation claim $F(r) = s^{(n)}$ at the point $r := (r_0, \dots, r_{n-1}) \in K^n$.

MLE-check. A *multilinear extension check*, or *MLE-check*, is a sumcheck specialized to summands of the form $\tilde{\mathbf{eq}}_n(\tau, \cdot) \cdot g(\cdot)$ for a challenge $\tau \in K^n$; since $\sum_{x \in \mathcal{B}_n} \tilde{\mathbf{eq}}_n(\tau, x) \cdot g(x) = \tilde{g}(\tau)$, an MLE-check reduces an evaluation claim on $\tilde{g}$ at τ to an evaluation claim on g (Protocol 2.2). The tensor structure of $\tilde{\mathbf{eq}}$ enables a more efficient prover algorithm [Gru24, §3].

PROTOCOL 2.2 (Multilinear-extension check). **Claim:** $\sum_{x \in \mathcal{B}_n} \tilde{\mathbf{eq}}_n(\tau, x) \cdot g(x) = s$, for a point $\tau \in K^n$ and $g \in K[X_0, \dots, X_{n-1}]$ of individual degree $\leq d$.

Rounds: Set $s^{(0)} := s$. For $i = 0, \dots, n-1$, write $k := n - i - 1$ (so round i binds X_k):

1. The prover sends the *round polynomial*

$$R^{(i)}(X) := \sum_{v \in \mathcal{B}_k} \tilde{\mathbf{eq}}_k((\tau_0, \dots, \tau_{k-1}), v) \cdot g(v, X, r_{k+1}, \dots, r_{n-1}) \in K[X], \quad \deg R^{(i)} \leq d,$$

where v ranges over the low variables $X_0, \dots, X_{k-1}$ and the equality-indicator weight on those unbound variables has been pulled out; d of its $d+1$ coefficients are transmitted.

2. The verifier checks $s^{(i)} = (1 - \tau_k) R^{(i)}(0) + \tau_k R^{(i)}(1)$, samples $r_k \leftarrow K$, and sets $s^{(i+1)} := R^{(i)}(r_k)$.

Output: the evaluation claim $g(r) = s^{(n)}$ at the point $r := (r_0, \dots, r_{n-1}) \in K^n$.

Zerocheck. A *zerocheck* reduces a claim that a polynomial $t \in K[X_0, \dots, X_{n-1}]$ vanishes on $\mathcal{B}_n$ to an MLE-check: the verifier samples $\tau \leftarrow K^n$ and the parties run an MLE-check on $\tilde{\mathbf{eq}}_n(\tau, \cdot) \cdot t(\cdot)$ with claimed sum 0. Soundness error is $n/|K|$ above the underlying MLE-check.

Batching. Several sumcheck claims $s_i \stackrel{?}{=} \sum_{v \in \mathcal{B}_n} F_i(v)$ over polynomials $F_0, \dots, F_{k-1} \in K[X_0, \dots, X_{n-1}]$ on a *common* number of variables can be discharged by a single sumcheck [CFS17]: the verifier samples a batching challenge $\alpha \leftarrow K$, and the parties run sumcheck on

$$\sum_{i=0}^{k-1} \alpha^i s_i \stackrel{?}{=} \sum_{v \in \mathcal{B}_n} \sum_{i=0}^{k-1} \alpha^i F_i(v).$$

Soundness of the reduction follows from Schwartz–Zippel applied to the batching polynomial in α .

Variable padding. The batched sumcheck above assumes the F_i share a common number of variables; when they do not, we pad each up to the common count. To raise an n -variate claim $\sum_{x \in \mathcal{B}_n} F(x) = s$ to $n+\nu$ variables, the parties instead sumcheck the padded summand $F(X_0, \dots, X_{n-1}) \cdot \tilde{\mathbf{eq}}_\nu(0^\nu, (X_n, \dots, X_{n+\nu-1}))$ over $\mathcal{B}_{n+\nu}$, with the same target s . The extra factor $\tilde{\mathbf{eq}}_\nu(0^\nu, \cdot)$ is supported on the single point $X_n = \dots = X_{n+\nu-1} = 0$, where it equals 1, so summing over the ν padding variables contributes a factor of 1 and the padded claim holds iff the original one does.

Since we bind variables from the highest index down, the ν padding variables $X_n, \dots, X_{n+\nu-1}$ are bound first—in rounds $i = 0, \dots, \nu-1$ —and the original variables only afterwards. The padding rounds are essentially free, and the remaining rounds reproduce the unpadded sumcheck up to a constant:

- In a padding round $i < \nu$ (binding $X_{n+\nu-1-i}$), every original variable is still unbound, so summing over them returns the original target s , and the round polynomial is

$$R^{(i)}(X) = s \cdot \tilde{\mathbf{eq}}(0, X) \cdot \tilde{\mathbf{eq}}_i(0^i, (r_{n+\nu-i}, \dots, r_{n+\nu-1})),$$

the bivariate $\tilde{\mathbf{eq}}(0, X) = 1 - X$ for the current variable times the constant accumulated from the already-bound padding challenges. The prover touches F not at all.

- In a round $i \geq \nu$, all padding variables are bound and contribute the constant $c := \tilde{\mathbf{eq}}_\nu(0^\nu, (r_n, \dots, r_{n+\nu-1}))$, so the round polynomial is the unpadding one scaled by c ,

$$R^{(i)}(X) = c \cdot R_F^{(i-\nu)}(X),$$

where $R_F^{(i-\nu)}$ is the round polynomial of the original n -variate sumcheck on F in its round $i - \nu$.

A polynomial entering the batch below the common variable count therefore costs only a string of trivial padding rounds before its ordinary sumcheck begins. The multiplicative analog of this padding—batching binary-tree grand-product arguments over trees of unequal depths—is developed in Appendix C.

2.2.1 Univariate Skip

The *univariate skip* [Gru24, §5] cuts the round count, and lets the prover work over a subfield, when the composite’s inputs are defined over a subfield of the challenge field.

The long axis. Fix a k -dimensional $\mathbb{F}_2$ -linear subspace $D \subset K$, so $|D| = 2^k$, and choose an $\mathbb{F}_2$ -basis $(\zeta_0, \dots, \zeta_{k-1})$ identifying $\mathcal{B}_k \cong D$ via $i \mapsto \hat{i} := i_0\zeta_0 + \dots + i_{k-1}\zeta_{k-1}$. We call this distinguished, size- $|D|$ dimension the *long axis*, and decorate quantities living on it with hats.

Oblong multilinear. An *oblong-multilinear* polynomial is multilinear in all but one variable, in which it instead has univariate degree below $|D| = 2^k$ over the domain D . The *oblong-multilinearization* of a multilinear t on $\mathcal{B}_k \times \mathcal{B}_\ell$ is

$$\hat{t}(\hat{I}, X) := \sum_{i \in \mathcal{B}_k} \delta_D(\hat{I}, \hat{i}) \cdot t(i, X),$$

where δ_D is the bivariate equality-indicator on D , of individual degree below 2^k .

The reduction. The univariate skip runs sumcheck on a composite $C(\hat{f}_0, \dots, \hat{f}_{m-1})$ of oblong multilinear. The first round, ranging over the size- $|D|$ domain in place of $\{0, 1\}$, is discharged by a single univariate polynomial send rather than k binary rounds; the partial specialization

$$\hat{t}(r_{\hat{i}}, X) = \sum_{i \in \mathcal{B}_k} \delta_D(r_{\hat{i}}, \hat{i}) \cdot t(i, X)$$

at the resulting challenge $r_{\hat{i}} \in K$ recovers an ordinary multilinear for the remaining rounds. Since the first-round univariate has degree below $d \cdot |D|$, where d is the composition degree, the prover can carry out its arithmetic entirely within any subfield $\mathbb{F} \subseteq K$ with $|\mathbb{F}| \geq d \cdot |D|$, embedding into K only once $r_{\hat{i}}$ is drawn; this is the source of the prover speedup when the inputs $\hat{f}_j$ take values in a small subfield.

2.3 Linear Interactive Oracle Proofs

A *linear interactive oracle proof* (LIOP) is an interactive protocol between a prover P and a verifier V for a relation $R \subset \{0, 1\}^* \times \{0, 1\}^*$. Both parties take as input a public instance x ; the prover additionally holds a witness w with $(x, w) \in R$. The protocol proceeds in rounds.

- In each round, the prover may send one or more *oracle messages* together with a possibly empty non-oracle message. The i -th oracle message is a function $\pi_i: \mathcal{B}_{n_i} \rightarrow K$ for some $n_i \geq 0$ fixed by the protocol; equivalently, π_i is a vector in $K^{2^{n_i}}$. The verifier does not access π_i directly during the protocol. The protocol additionally fixes, for each oracle, a *randomizable support* $I_i \subseteq \mathcal{B}_{n_i}$, a (possibly empty) subset of coordinates whose role is explained below; unless stated otherwise $I_i = \emptyset$.

- The verifier responds with a challenge sampled uniformly from K (or, when convenient, from some fixed challenge space).
- After all rounds, the verifier issues a set of *linear queries*. The j -th linear query targets some oracle π_{i_j} and is specified by an n_{i_j} -variate multilinear polynomial $t_j \in K[X_0, \dots, X_{n_{i_j}-1}]^{\leq 1}$ together with a target value $s_j \in K$. The query is *satisfied* if

$$\sum_{w \in \mathcal{B}_{n_{i_j}}} t_j(w) \cdot \pi_{i_j}(w) = s_j;$$

i.e., the inner product of the oracle with the table of hypercube evaluations of t_j equals the target. Every linear query must *vanish on the randomizable support* of its target oracle: $t_j(w) = 0$ for all $w \in I_{i_j}$. The verifier accepts iff all linear queries are satisfied. The list of queries is a deterministic function of the transcript.

The LIOP is *complete* if for every $(x, w) \in R$ the honest protocol execution makes V accept with probability 1, and *sound with error* ε if for every x such that $(x, w) \notin R$ for all w and every (possibly cheating) prover P^* , the probability that V accepts is at most ε . Knowledge soundness is defined analogously via an efficient extractor.

Randomizable support. The randomizable support I_i marks coordinates of π_i that the honest prover fills with independent uniform randomness, drawing $\pi_i(w) \leftarrow K$ for each $w \in I_i$; the remaining coordinates of π_i are determined by the protocol as usual. The well-formedness requirement that every query targeting π_i vanish on I_i makes this randomness *free*: for any such query,

$$\sum_{w \in \mathcal{B}_{n_{i_j}}} t_j(w) \cdot \pi_{i_j}(w) = \sum_{w \in \mathcal{B}_{n_{i_j}} \setminus I_{i_j}} t_j(w) \cdot \pi_{i_j}(w),$$

since the omitted terms all have $t_j(w) = 0$. The query targets therefore depend only on the values of π_i off its randomizable support, so the random fill affects neither completeness nor soundness, and an LIOP with $I_i \neq \emptyset$ inherits these properties from its $I_i = \emptyset$ restriction. The randomness carries no information about the witness through the LIOP's own queries; it instead perturbs the *encoded* oracle of the SNARK compiler (§5), where it serves to mask the codeword openings that would otherwise leak. This is the mechanism underlying the zero-knowledge compiler (§7); the basic compiler of §5 and the Binius64 LIOP of §4—until the latter is used as the inner LIOP of the zero-knowledge composition of §7—take $I_i = \emptyset$ throughout and recover the standard notion. Choosing I_i large enough, and with coordinates suitably placed, to make the codeword openings simulatable is a quantitative condition deferred to §7.

Succinct queries. A linear query specified by (t_j, s_j) is *succinct* if the polynomial t_j can be evaluated at an arbitrary point of $K^{n_{i_j}}$ in $\text{poly}(n_{i_j})$ operations over K . All linear queries appearing in our LIOPs are succinct. The succinct queries used here are closely related to, but strictly more restrictive than, the *succinct linear forms* of Chiesa, Fenzi, and Weissenberg [CFW26, §5], which permit arbitrary K -linear functionals admitting polylogarithmic-time evaluation; in our setting the functional is always the inner product against the hypercube evaluation table of a succinctly evaluable multilinear polynomial.

The basic example is the *multilinear-evaluation query* at a point $r \in K^{n_i}$: taking $t(X) := \tilde{\mathbf{e}}_{\mathbf{q}_{n_i}}(r, X)$, the constraint $\sum_{w \in \mathcal{B}_{n_i}} \tilde{\mathbf{e}}_{\mathbf{q}_{n_i}}(r, w) \cdot \pi_i(w) = s$ is precisely the assertion that the multilinear extension $\tilde{\pi}_i$ takes value s at r , and the succinctness of $\tilde{\mathbf{e}}_{\mathbf{q}}$ makes the query succinct. LIOPs whose linear queries are all of this form are sometimes called polynomial IOPs (PIOPs); we use the broader term LIOP to admit other succinct multilinear t_j where convenient.

The compilation of an LIOP into a SNARK using a polynomial commitment scheme is recalled in §5.

3 The Constraint System

The Binius64 constraint system operates on 64-bit words and supports four native constraint forms: *Zero constraints*, which express $\mathbb{F}_2$ -linear relations among shifted prover words; *BitAnd constraints*,

which express bitwise-AND relations between XOR-accumulations of shifted prover words; *IntMul constraints*, which express unsigned-integer multiplication on 64-bit operands; and *BinMul constraints*, which express multiplication in the GHASH field $\mathbb{F}_{2^{128}}$ on operands each carried by a pair of 64-bit words. This section defines the constraint language as a combinatorial object, independent of any protocol that verifies it. Appendix A sketches how this language expresses common derived operations (bitwise NOT, OR; integer addition and subtraction).

3.1 Prover Data Layout

At runtime, the prover supplies a list of 64-bit words. The list is divided into three contiguous *stretches* of compile-time-fixed lengths n_{const} , n_{inout} , and n_{witness} , denoted respectively the *constant*, *input-output*, and *witness* parts. We write w for the concatenation of all three, so that w has total length

$$n_{\text{words}} := n_{\text{const}} + n_{\text{inout}} + n_{\text{witness}},$$

and we write $w[y] \in \mathcal{B}_{64}$ for the y -th word, indexed by $y \in \{0, \dots, n_{\text{words}} - 1\}$.

The constant and input-output stretches are public: the verifier holds them. The constant stretch is fixed at compile time and is shared across all proofs; we record its values via a map $C: \{0, \dots, n_{\text{const}} - 1\} \rightarrow \mathcal{B}_{64}$. The input-output stretch is the public statement for the current proof, supplied as a map $u: \{0, \dots, n_{\text{inout}} - 1\} \rightarrow \mathcal{B}_{64}$. The remaining witness stretch is private to the prover and impacts the verifier's runtime only polylogarithmically. Throughout, we use the term *prover data* (rather than *witness*) for w to emphasize that only part of w is private.

The prover data w *satisfies* the constraint system with respect to the statement u if all of the following hold:

- the constant stretch matches C : $w[y] = C(y)$ for each $y \in \{0, \dots, n_{\text{const}} - 1\}$;
- the input-output stretch matches u : $w[n_{\text{const}} + y] = u(y)$ for each $y \in \{0, \dots, n_{\text{inout}} - 1\}$;
- every Zero constraint is satisfied (§3.4);
- every BitAnd constraint is satisfied (§3.5);
- every IntMul constraint is satisfied (§3.6);
- every BinMul constraint is satisfied (§3.7).

3.2 Shifted Value Indices

The constraints in this system act not on raw prover words but on *shifted* versions of them. We support eight shift operations:

$$\text{ShiftOps} := \{\text{sll}, \text{srl}, \text{sra}, \text{ror}, \text{sll32}, \text{srl32}, \text{sra32}, \text{ror32}\}.$$

The first four are full-width 64-bit operations. For $v \in \mathcal{B}_{64}$ and $s \in \{0, \dots, 63\}$:

- $\text{sll}(v, s)$ shifts v towards the more-significant end by s bits, zero-filling the vacated low bits;
- $\text{srl}(v, s)$ shifts v towards the less-significant end by s bits, zero-filling the vacated high bits;
- $\text{sra}(v, s)$ shifts v towards the less-significant end by s bits, replicating the most-significant bit into the vacated high positions;
- $\text{ror}(v, s)$ rotates v right by s bits, so that the s least-significant bits of v wrap into the most-significant positions of the result.

The remaining four operations are the 32-bit *parallel* variants sll32 , srl32 , sra32 , ror32 . Each treats v as a pair of independent 32-bit halves (the low and high halves of v) and applies the corresponding shift to both halves in parallel. The 32-bit operations use only the five least-significant bits of the 6-bit shift amount s ; the most-significant bit of s is ignored.

Since $|\text{ShiftOps}| = 8$, we index the shift operations by the hypercube $\mathcal{B}_3$: we write $\text{op}(o)$ for the o -th operation in the order listed above, under the little-endian identification of $\mathcal{B}_3$ with $\{0, \dots, 7\}$

(§2). The enumeration is chosen so that the most-significant bit of o selects the width—clear for the four full-width operations, set for the four 32-bit parallel variants—while the two least-significant bits select the base operation `sll`, `srl`, `sra`, `ror` in either width.

A *shifted value index* names a prover word together with a pair of shift operations applied to it in sequence. Formally it is a tuple

$$(y, \text{op}_1, s_1, \text{op}_2, s_2) \in \{0, \dots, n_{\text{words}} - 1\} \times (\text{ShiftOps} \times \{0, \dots, 63\})^2,$$

naming a prover-data index y , an *inner* shift (op_1, s_1) , and an *outer* shift (op_2, s_2) . We write `val` for the value it represents:

$$\text{val}(y, \text{op}_1, s_1, \text{op}_2, s_2) := \text{op}_2(\text{op}_1(w[y], s_1), s_2) \in \mathcal{B}_{64}.$$

Since $s = 0$ acts as the identity for every $\text{op} \in \text{ShiftOps}$ — $\text{sll}(v, 0) = \text{srl}(v, 0) = \dots = v$ —a zero amount in either slot suppresses that shift, so single shifts and raw prover words are the special cases $s_2 = 0$ and $s_1 = s_2 = 0$ rather than a separate form. For example, $(55, \text{sll}, 10, \text{sll}, 0)$ represents `w[55] << 10`; $(55, \text{sll}, 40, \text{sra}, 40)$ represents `(int64_t) (w[55] << 40) >> 40`, the sign-extension of the low 24 bits of `w[55]`; and $(55, \text{sll}, 0, \text{sll}, 0)$ represents `w[55]` itself.

Composing two shifts inside a single index is strictly more expressive than composing them across two indices: the latter would need an intermediate prover word, hence a witness entry and a Zero constraint to define it, whereas the former is free at the level of the constraint system and, as §4.7 shows, nearly free in the protocol. Common patterns that a single shift cannot express—sign-extension of a sub-word field, extraction of a bit field to a fixed position, masking off a byte by shifting it off one end and back—become single index entries.

Canonical form. A zero shift amount makes the accompanying operation immaterial, so a value can be named by several tuples. We fix a canonical form, which makes the representation unique: when $s_2 = 0$ we take $\text{op}_2 := \text{sll}$, when $s_1 = 0$ we take $\text{op}_1 := \text{sll}$, and a lone shift is placed in the inner slot, so that $s_1 = 0$ implies $s_2 = 0$. A canonical index with $s_2 \neq 0$ therefore has $s_1 \neq 0$ as well. We call such an index *doubly shifted*, one with $s_2 = 0 \neq s_1$ *singly shifted*, and one with $s_1 = s_2 = 0$ *unshifted*. Nothing in the reduction of §4.7 depends on this normalization for correctness.

3.3 Constraint Arrays

We use a notational device to package XOR-accumulations of shifted value indices. Fix some total count n_{cons} of constraints, and assume that for each $x \in \{0, \dots, n_{\text{cons}} - 1\}$ we are given a list L_x of shifted value indices. The *constraint array* associated with $(L_x)_{x=0}^{n_{\text{cons}}-1}$ is the length- n_{cons} array $z \in \mathcal{B}_{64}^{n_{\text{cons}}}$ defined by

$$z[x] := \bigoplus_{(y, \text{op}_1, s_1, \text{op}_2, s_2) \in L_x} \text{op}_2(\text{op}_1(w[y], s_1), s_2).$$

We will define Zero, BitAnd, IntMul, and BinMul constraints in terms of arrays of this form.

3.4 Zero Constraints

The simplest constraint form asserts that a single constraint array vanishes. The constraint system specifies a total number n_{zero} of Zero constraints. The x -th Zero constraint is a single list of shifted value indices

$$L_x^d, \quad x \in \{0, \dots, n_{\text{zero}} - 1\},$$

inducing one constraint array $d \in \mathcal{B}_{64}^{n_{\text{zero}}}$ as in §3.3. The prover data w *satisfies* the Zero constraints if for each $x \in \{0, \dots, n_{\text{zero}} - 1\}$,

$$d[x] = \bigoplus_{(y, \text{op}_1, s_1, \text{op}_2, s_2) \in L_x^d} \text{op}_2(\text{op}_1(w[y], s_1), s_2) = 0.$$

A Zero constraint is thus an arbitrary $\mathbb{F}_2$ -linear relation among shifted prover words. In particular, taking L_x^d to be a two-element list constrains two shifted values to be equal, and taking it to be a three-element list $\{u, v, z\}$ forces $z = u \oplus v$, so bitwise XOR is native. Such relations are already expressible

with BitAnd constraints alone—mask one side with the all-ones constant (Appendix A)—but a Zero constraint carries one constraint array rather than three, and §4.4 discharges it with no prover message and no sumcheck round at all.

3.5 BitAnd Constraints

The constraint system specifies a total number n_{and} of BitAnd constraints. The x -th BitAnd constraint is a triple of lists of shifted value indices

$$(L_x^a, L_x^b, L_x^c), \quad x \in \{0, \dots, n_{\text{and}} - 1\},$$

which together induce three constraint arrays $a, b, c \in \mathcal{B}_{64}^{n_{\text{and}}}$ as in §3.3: for each $x \in \{0, \dots, n_{\text{and}} - 1\}$,

$$a[x] := \bigoplus_{u \in L_x^a} \text{val}(u), \quad b[x] := \bigoplus_{u \in L_x^b} \text{val}(u), \quad c[x] := \bigoplus_{u \in L_x^c} \text{val}(u).$$

The prover data w *satisfies* the BitAnd constraints if for each $x \in \{0, \dots, n_{\text{and}} - 1\}$,

$$a[x] \wedge b[x] = c[x],$$

where $\wedge$ denotes bitwise AND on 64-bit words.

3.6 IntMul Constraints

The constraint system specifies a total number n_{mul} of IntMul constraints. The x -th IntMul constraint is a tuple of four lists of shifted value indices

$$(L_x^a, L_x^b, L_x^{c_{\text{lo}}}, L_x^{c_{\text{hi}}}), \quad x \in \{0, \dots, n_{\text{mul}} - 1\},$$

inducing four length- n_{mul} constraint arrays $a, b, c_{\text{lo}}, c_{\text{hi}} \in \mathcal{B}_{64}^{n_{\text{mul}}}$ as in §3.3. Each word $a[x] \in \mathcal{B}_{64}$ has bits $a[i, x] \in \mathbb{F}_2$ for $i = 0, \dots, 63$; for any such bit vector we write

$$\llbracket a \rrbracket[x] := \sum_{i=0}^{63} 2^i \cdot a[i, x] \in \{0, \dots, 2^{64} - 1\}$$

for the nonnegative integer encoded by the word at x (and likewise for $b, c_{\text{lo}}, c_{\text{hi}}$). The prover data w *satisfies* the IntMul constraints if for each $x \in \{0, \dots, n_{\text{mul}} - 1\}$,

$$\llbracket a \rrbracket[x] \cdot \llbracket b \rrbracket[x] = 2^{64} \cdot \llbracket c_{\text{hi}} \rrbracket[x] + \llbracket c_{\text{lo}} \rrbracket[x],$$

the multiplication taken over the integers. Equivalently, $\llbracket c_{\text{hi}} \rrbracket[x]$ and $\llbracket c_{\text{lo}} \rrbracket[x]$ are the high and low 64-bit halves of the 128-bit unsigned product $\llbracket a \rrbracket[x] \cdot \llbracket b \rrbracket[x]$.

3.7 BinMul Constraints

The constraint system specifies a total number n_{binmul} of BinMul constraints. A BinMul constraint multiplies two elements of the GHASH field $\mathbb{F}_{2^{128}} \cong \mathbb{F}_2[X]/(X^{128} + X^7 + X^2 + X + 1)$ (§2.1). Since $\mathbb{F}_{2^{128}}$ is a 128-dimensional $\mathbb{F}_2$ -space while a prover word holds only 64 bits, each field element is carried by a pair of words—a *low* word supplying the coefficients of $1, X, \dots, X^{63}$ and a *high* word supplying the coefficients of $X^{64}, \dots, X^{127}$. Concretely, for a pair of words $(z_{\text{lo}}, z_{\text{hi}}) \in \mathcal{B}_{64} \times \mathcal{B}_{64}$ with bits $z_{\text{lo}}[i], z_{\text{hi}}[i] \in \mathbb{F}_2$, we write

$$\langle\langle z_{\text{lo}}, z_{\text{hi}} \rangle\rangle := \sum_{i=0}^{63} z_{\text{lo}}[i] \cdot X^i + \sum_{i=0}^{63} z_{\text{hi}}[i] \cdot X^{64+i} \in \mathbb{F}_{2^{128}}$$

for the field element it encodes, the monomials $1, X, \dots, X^{127}$ being the $\mathbb{F}_2$ -basis of $\mathbb{F}_{2^{128}}$ fixed in §2.1.

The x -th BinMul constraint is a tuple of six lists of shifted value indices

$$(L_x^{a_{\text{lo}}}, L_x^{a_{\text{hi}}}, L_x^{b_{\text{lo}}}, L_x^{b_{\text{hi}}}, L_x^{c_{\text{lo}}}, L_x^{c_{\text{hi}}}), \quad x \in \{0, \dots, n_{\text{binmul}} - 1\},$$

inducing six length- n_{binmul} constraint arrays $a_{\text{lo}}, a_{\text{hi}}, b_{\text{lo}}, b_{\text{hi}}, c_{\text{lo}}, c_{\text{hi}} \in \mathcal{B}_{64}^{n_{\text{binmul}}}$ as in §3.3. Each operand pair is read as a GHASH field element via $\langle\langle \cdot \rangle\rangle$: the pair $(a_{\text{lo}}[x], a_{\text{hi}}[x])$ encodes $\langle\langle a_{\text{lo}}[x], a_{\text{hi}}[x] \rangle\rangle \in \mathbb{F}_{2^{128}}$, and likewise for b and c . The prover data w satisfies the BinMul constraints if for each $x \in \{0, \dots, n_{\text{binmul}} - 1\}$,

$$\langle\langle a_{\text{lo}}[x], a_{\text{hi}}[x] \rangle\rangle \cdot \langle\langle b_{\text{lo}}[x], b_{\text{hi}}[x] \rangle\rangle = \langle\langle c_{\text{lo}}[x], c_{\text{hi}}[x] \rangle\rangle,$$

the multiplication taken in the GHASH field $\mathbb{F}_{2^{128}}$.

4 The Binius64 LIOP

This section specifies the Binius64 LIOP: an interactive oracle protocol that, given a constraint system (§3) and a public statement u , verifies that the prover holds a witness w satisfying the constraint system with respect to u . The compilation of this LIOP into a SNARK is discussed in §5.

Throughout, we assume the constraint-system parameters are padded so that $n_{\text{words}} = 2^{\ell_{\text{words}}}$, $n_{\text{zero}} = 2^{\ell_{\text{zero}}}$, $n_{\text{and}} = 2^{\ell_{\text{and}}}$, $n_{\text{mul}} = 2^{\ell_{\text{mul}}}$, $n_{\text{binmul}} = 2^{\ell_{\text{binmul}}}$, and $n_{\text{public}} := n_{\text{const}} + n_{\text{inout}} = 2^{\ell_{\text{public}}}$ are all powers of two. The deterministic zerocheck of §4.3.1 consumes three constraint-index variables, so we additionally require $\ell_{\text{and}} \geq 3$. We additionally pad both the witness and the public segment so that $\lambda \mid 64 \cdot n_{\text{words}}$ and $\lambda \mid 64 \cdot n_{\text{public}}$; equivalently, $\ell_{\text{words}} \geq \log_2(\lambda) - 6$ and $\ell_{\text{public}} \geq \log_2(\lambda) - 6$. With $\lambda = 128$, this means $\ell_{\text{words}}, \ell_{\text{public}} \geq 1$. The first condition is needed for the packed-witness construction of §4.8 and the second for the public input check of §4.9. When the LIOP is compiled to a zero-knowledge SNARK (§7), the packed-witness oracle additionally carries a randomizable support (§2.3); the compiler of §7.2 imposes a lower bound on its size, and the padding above must be enlarged to host it. Padding entries are taken to be zero (or, for constraint-array lists, the empty list). Where convenient we identify $\mathcal{B}_{\ell_{\text{words}}} \cong \{0, \dots, n_{\text{words}} - 1\}$ and similarly for the other domains.

4.1 Protocol Overview

The protocol commits to a single K -valued oracle obtained by packing the witness w word-by-word, runs a sequence of sumcheck-based reductions, and finally issues a single linear query on the packed oracle. Concretely, w is viewed as a function $w: \mathcal{B}_6 \times \mathcal{B}_{\ell_{\text{words}}} \rightarrow \mathbb{F}_2$, where $(j, y) \in \mathcal{B}_6 \times \mathcal{B}_{\ell_{\text{words}}}$ indexes bit j of word y ; its multilinear extension is the $6 + \ell_{\text{words}}$ -variate multilinear

$$\tilde{w}(J_0, \dots, J_5, Y_0, \dots, Y_{\ell_{\text{words}}-1}) \in K[J, Y]^{\leq 1}.$$

The prover does not commit to w directly: it instead commits to the packed function $\text{pack}_\lambda(w): \mathcal{B}_{\ell_{\text{pack}}} \rightarrow K$, defined in §4.8, which encodes λ consecutive bits of w as a single K -element. The packed witness is the actual LIOP oracle, in keeping with the K -valued-oracle convention of §2.3.

The LIOP proceeds in nine phases:

1. **Packed witness commitment.** The prover sends the packed witness oracle $\pi := \text{pack}_\lambda(w): \mathcal{B}_{\ell_{\text{pack}}} \rightarrow K$, where $\ell_{\text{pack}} := \ell_{\text{words}} - (\log_2 \lambda - 6)$ and pack_λ is the packing map defined in §4.8. (Implementations commit to π via the polynomial commitment scheme of §5, but the LIOP itself treats π as an oracle of length $2^{\ell_{\text{pack}}}$.)
2. **IntMul reduction (§4.5).** A GKR-based reduction transforms the satisfaction of the n_{mul} IntMul constraints into per-bit evaluation claims on the four multilinear extensions $\tilde{a}, \tilde{b}, \tilde{c}_{\text{lo}}, \tilde{c}_{\text{hi}}$ of the IntMul constraint arrays, leaving the bit-index free.
3. **BinMul reduction (§4.6).** A zerocheck-based reduction transforms the satisfaction of the n_{binmul} BinMul constraints into per-bit evaluation claims on the multilinear extensions of the six BinMul constraint arrays $a_{\text{lo}}, a_{\text{hi}}, b_{\text{lo}}, b_{\text{hi}}, c_{\text{lo}}, c_{\text{hi}}$, leaving the bit-index free.
4. **BitAnd reduction (§4.3).** A sumcheck-based reduction transforms the satisfaction of the n_{and} BitAnd constraints into evaluation claims on the three oblong-multilinearizations $\hat{a}, \hat{b}, \hat{c}$ of the BitAnd constraint arrays. Its univariate skip samples the shared bit-index oblong point r_i against which the two multiplication reductions' outputs are folded (§4.7.1).

5. **Zero reduction (§4.4).** The satisfaction of the n_{zero} Zero constraints is transformed into a single evaluation claim on the oblong-multilinearization $\widehat{d}$ of the Zero constraint array, at the point the BitAnd reduction's sumcheck has just output. The phase carries no prover message and no sumcheck round.
6. **Shift reduction (§4.7).** The evaluation claims produced by the Zero, BitAnd, IntMul, and BinMul reductions are batched, and a single sumcheck reduces them to one evaluation claim on $\widetilde{w}$ at a point $(r_j, r_y) \in K^6 \times K^{\ell_{\text{words}}}$.
7. **Ring-switching (§4.8).** The evaluation claim $\widetilde{w}(r_j, r_y) = t$ on the $\mathbb{F}_2$ -valued multilinear $\widetilde{w}$ is converted into a linear query on the packed K -valued oracle π , via the protocol of Diamond and Posen [DP24].
8. **Public input check (§4.9).** A separate linear query on π certifies that the packing of the public segment $C \parallel u$ agrees with the prefix of π that contains it. This query is then random-combined with the ring-switching query into a single linear query.
9. **Witness query.** The verifier issues the combined linear query on π .

The two multiplication reductions (phases 2 and 3) must precede the BitAnd reduction (phase 4): the BitAnd univariate skip samples the shared bit-index oblong point $r_{\hat{i}}$, and soundness requires that the multiplication reductions' per-bit outputs already be fixed when $r_{\hat{i}}$ is drawn (§4.7.1). Phases 2 and 3 are mutually independent and may run in either order. Phase 5 evaluates at the point produced by the BitAnd sumcheck and so must follow phase 4; since it carries no message, this is a matter of bookkeeping rather than of interaction.

4.2 Shift Indicator Polynomials

Each of the eight shift operations in **ShiftOps** (§3.2) acts on $\mathcal{B}_{64}$ at the bit level. To verify claims about shifted prover words via sumcheck, we lift each shift operation to a multilinear polynomial identity. The key tool is the *shift indicator polynomial*, a multilinear in 18 variables whose values on $\mathcal{B}_6^3$ describe which bit of the original word ends up in which bit of the shifted word. Following Diamond and Posen [DG25, §4.3], the existence of such an identity reduces shifted-witness evaluation claims to single witness evaluation claims, provided the shift indicator admits an efficient evaluation algorithm.

Recall (§4.1) that the prover data w is viewed as a function $w: \mathcal{B}_6 \times \mathcal{B}_{\ell_{\text{words}}} \rightarrow \mathbb{F}_2$ with multilinear extension $\widetilde{w}(J, Y)$. For each $\text{op} \in \text{ShiftOps}$, the *shifted witness function* is

$$\text{shift}_{\text{op}}(w): \mathcal{B}_{\ell_{\text{words}}} \times \mathcal{B}_6 \rightarrow \mathcal{B}_{64}, \quad \text{shift}_{\text{op}}(w)(y, s) := \text{op}(w[y], s).$$

Composing with bit extraction gives a function on $\mathcal{B}_6 \times \mathcal{B}_{\ell_{\text{words}}} \times \mathcal{B}_6$ sending (i, y, s) to the i -th bit of $\text{op}(w[y], s)$; we write $\widetilde{\text{shift}}_{\text{op}}(\widetilde{w})(I, Y, S)$ for its multilinear extension.

For each op we define the *shift indicator* function $\text{shift-ind}_{\text{op}}: \mathcal{B}_6 \times \mathcal{B}_6 \times \mathcal{B}_6 \rightarrow \mathbb{F}_2$ by

$$\text{shift-ind}_{\text{op}}(i, j, s) := \begin{cases} 1 & \text{if the } i\text{-th bit of } \text{op}(v, s) \text{ equals the } j\text{-th bit of } v \text{ for every } v \in \mathcal{B}_{64}, \\ 0 & \text{otherwise.} \end{cases}$$

For example, $\text{shift-ind}_{\text{sll}}(30, 20, 10) = 1$ because $\text{sll}(v, 10)_{30} = v_{20}$ for all v , whereas $\text{shift-ind}_{\text{sll}}(9, j, 10) = 0$ for all j because $\text{sll}(v, 10)_9 = 0$ for all v . We write $\widetilde{\text{shift-ind}}_{\text{op}}(I, J, S)$ for the corresponding multilinear extension, an 18-variate multilinear with $\mathbb{F}_2$ -coefficients.

The defining identity of the shift indicator polynomial is, for each op ,

$$\widetilde{\text{shift}}_{\text{op}}(\widetilde{w})(I, Y, S) = \sum_{j \in \mathcal{B}_6} \widetilde{\text{shift-ind}}_{\text{op}}(I, j, S) \cdot \widetilde{w}(j, Y). \quad (1)$$

To verify (1), it suffices to check it on the cube: fix $i, s \in \mathcal{B}_6$ and $y \in \mathcal{B}_{\ell_{\text{words}}}$. Either some unique $j^* \in \mathcal{B}_6$ has $\text{shift-ind}_{\text{op}}(i, j^*, s) = 1$, in which case the right-hand side reduces to $\widetilde{w}(j^*, y) = w[y]_{j^*} = \text{op}(w[y], s)_i$; or no such j^* exists, in which case $\text{op}(w[y], s)_i = 0$ by definition of op on zero-fill bits, and the right-hand side is zero.

Identity (1) enables the standard reduction: given an evaluation claim on $\widetilde{\text{shift}}_{\text{op}}(\tilde{w})$ at some point $(r_i, r_y, r_s) \in K^{6+\ell_{\text{words}}+6}$, a 6-round sumcheck binds the index j , leaving evaluation claims on $\widetilde{\text{shift-ind}}_{\text{op}}(r_i, r_j, r_s)$ (which the verifier evaluates directly) and on $\tilde{w}(r_j, r_y)$ (a single query to the witness oracle).

Composing two shifts. The shift indicators compose, which is what makes the doubly shifted value indices of §3.2 tractable. The key structural fact, already used above, is that for each (i, s) there is *most one* $j \in \mathcal{B}_6$ with $\text{shift-ind}_{\text{op}}(i, j, s) = 1$: output bit i of $\text{op}(v, s)$ is either a single fixed input bit of v or the constant zero. This holds for all eight operations, including **sra**, where the vacated high positions all take the single input bit $j = 63$, and including the 32-bit variants, where the property holds within each half. Consequently

$$\text{op}_2(\text{op}_1(v, s_1), s_2)_i = \sum_{k \in \mathcal{B}_6} \sum_{j \in \mathcal{B}_6} \text{shift-ind}_{\text{op}_2}(i, k, s_2) \cdot \text{shift-ind}_{\text{op}_1}(k, j, s_1) \cdot v_j \quad (2)$$

for every $v \in \mathcal{B}_{64}$: the inner factor selects the input bit j that supplies bit k of the intermediate word $\text{op}_1(v, s_1)$, and the outer factor selects the bit k of that intermediate word which output bit i reproduces. The index k ranges over the bit positions of the intermediate word, which is never materialized as a witness value; it exists only as a summation index, and the shift reduction (§4.7) carries it as one more sumcheck variable.

Interpolation over the operations. The shift reduction treats the shift operation as one more hypercube coordinate, so we assemble the eight indicators into a single polynomial. Using the indexing $o \mapsto \text{op}(o)$ of ShiftOps by $\mathcal{B}_3$ (§3.2), define the *interpolated shift indicator* $\text{shift-ind} : \mathcal{B}_6 \times \mathcal{B}_6 \times \mathcal{B}_6 \times \mathcal{B}_3 \rightarrow \mathbb{F}_2$ by

$$\text{shift-ind}(i, j, s, o) := \text{shift-ind}_{\text{op}(o)}(i, j, s),$$

and write $\widetilde{\text{shift-ind}}(I, J, S, O)$ for its multilinear extension, a 21-variate multilinear with $\mathbb{F}_2$ -coefficients. Expanding the multilinear extension along its last three variables (§2) gives

$$\widetilde{\text{shift-ind}}(I, J, S, O) = \sum_{o \in \mathcal{B}_3} \tilde{\text{eq}}_3(O, o) \cdot \widetilde{\text{shift-ind}}_{\text{op}(o)}(I, J, S), \quad (3)$$

since both sides are multilinear and agree on $\mathcal{B}_{21}$.

Succinctness. The shift indicator $\widetilde{\text{shift-ind}}_{\text{op}}$ is *succinct* for each of the eight $\text{op} \in \text{ShiftOps}$: any point of K^{18} can be evaluated in $O(1)$ field operations (concretely, at most 84 multiplications in K plus a 6-way product). The explicit arithmetizations, with auxiliary functions $\mathbf{s}_k, \mathbf{s}'_k, \mathbf{a}_k$ for the recursive constructions of the 64-bit and 32-bit-parallel variants, are deferred to Appendix B. The interpolated indicator $\widetilde{\text{shift-ind}}$ is succinct in turn: by (3), a point of K^{21} costs the eight per-op evaluations plus the $O(1)$ weights $\text{TensorExpand}_3(O)$.

4.3 The BitAnd Reduction

The BitAnd reduction transforms the satisfaction of the n_{and} BitAnd constraints (§3.5) into evaluation claims on the oblong-multilinearizations of the BitAnd constraint arrays. It is a univariate-skip variant of the sumcheck-based zerocheck [Gru24], in which the first sumcheck round is replaced by a univariate polynomial send. To minimize prover work, we also use deterministic challenges sampled from $\mathbb{F}_{2^8}$ in the first three sumcheck rounds, adapting an idea of Dao and Thaler [DT24].

Recall that satisfaction of the BitAnd constraints is the claim that, for each $x \in \mathcal{B}_{\ell_{\text{and}}}$, $a[x] \wedge b[x] = c[x]$, where the 64-bit constraint arrays a, b, c are defined in §3.5 and the bitwise AND is just per-bit field multiplication. Equivalently, the polynomial

$$f(I, X) := \tilde{a}(I, X) \cdot \tilde{b}(I, X) - \tilde{c}(I, X)$$

on $\mathcal{B}_6 \times \mathcal{B}_{\ell_{\text{and}}}$ vanishes identically. We work with the oblong-multilinearizations of the constraint arrays: writing $\hat{a}(\hat{I}, X), \hat{b}(\hat{I}, X), \hat{c}(\hat{I}, X)$ for the oblong-multilinearizations (§2.2.1) with $k = 6$, so each

is of degree < 64 in $\widehat{I}$ and multilinear in $X_0, \dots, X_{\ell_{\text{and}}-1}$, the constraint reduces to the vanishing of

$$\widehat{f}(\widehat{I}, X) := \widehat{a}(\widehat{I}, X) \cdot \widehat{b}(\widehat{I}, X) - \widehat{c}(\widehat{I}, X)$$

on $D \times \mathcal{B}_{\ell_{\text{and}}}$.

4.3.1 The Rijndael Field and Deterministic Zerocheck

For the prover algorithm to run mostly over a small field, we fix the Rijndael field $\mathbb{F}_{2^8} \cong \mathbb{F}_2[X]/(X^8 + X^4 + X^3 + X + 1)$ (the AES base field) and embed it into K . We assume $D \subset \mathbb{F}_{2^8}$ (a 6-dimensional $\mathbb{F}_2$ -subspace of the 8-dimensional $\mathbb{F}_2$ -space $\mathbb{F}_{2^8}$); using the fixed embedding $\iota: \mathbb{F}_{2^8} \hookrightarrow K$ from §2.1, this gives the chain $D \subset \mathbb{F}_{2^8} \subset K$.

Tensor basis. We select three elements $\rho_0, \rho_1, \rho_2 \in \mathbb{F}_{2^8}$ such that

$$(\widetilde{\text{eq}}_3(\rho_0, \rho_1, \rho_2, u_0, u_1, u_2))_{u \in \mathcal{B}_3}$$

is an $\mathbb{F}_2$ -basis of $\mathbb{F}_{2^8}$. The choices $\rho_0 := X$, $\rho_1 := X^2$, and $\rho_2 := X^4$ work: their subproducts are $1, X, X^2, \dots, X^7$, which form an $\mathbb{F}_2$ -basis of $\mathbb{F}_{2^8}$, so the tensor expansion $\text{TensorExpand}_3(\rho_0, \rho_1, \rho_2)$ (§2) differs from this monomial list by an invertible $\mathbb{F}_2$ -linear map. We write $\sigma_j := \iota(\rho_j) \in K$ for $j = 0, 1, 2$; since ι is an $\mathbb{F}_2$ -linear injection that commutes with $\widetilde{\text{eq}}$ (which has $\mathbb{F}_2$ -coefficients), $(\widetilde{\text{eq}}_3(\sigma, u))_{u \in \mathcal{B}_3}$ is also an $\mathbb{F}_2$ -linearly independent list in K .

Deterministic zerocheck. The standard zerocheck on an $\mathbb{F}_2$ -valued polynomial $t(X)$ on $\mathcal{B}_\ell$ that is claimed to vanish samples $r_x \leftarrow K^\ell$ and sumchecks $0 \stackrel{?}{=} \sum_{x \in \mathcal{B}_\ell} t(x) \cdot \widetilde{\text{eq}}_\ell(r_x, x)$, with soundness error $\ell/|K|$. The *deterministic-zerocheck* variant samples only $\ell - 3$ challenges and fixes the first three to the deterministic constants $\sigma_0, \sigma_1, \sigma_2$:

$$r_x := (\sigma_0, \sigma_1, \sigma_2, \bar{r}_{x,0}, \dots, \bar{r}_{x,\ell-4}), \quad \bar{r}_x \leftarrow K^{\ell-3}.$$

A direct calculation gives

$$\sum_{x \in \mathcal{B}_\ell} t(x) \cdot \widetilde{\text{eq}}_\ell(r_x, x) = \sum_{v \in \mathcal{B}_{\ell-3}} \bar{t}(v) \cdot \widetilde{\text{eq}}_{\ell-3}(\bar{r}_x, v),$$

where

$$\bar{t}(X_0, \dots, X_{\ell-4}) := \sum_{u \in \mathcal{B}_3} t(u_0, u_1, u_2, X_0, \dots, X_{\ell-4}) \cdot \widetilde{\text{eq}}_3(\sigma, u).$$

Since t is $\mathbb{F}_2$ -valued on $\mathcal{B}_\ell$ and $(\widetilde{\text{eq}}_3(\sigma, u))_{u \in \mathcal{B}_3}$ is $\mathbb{F}_2$ -linearly independent, t vanishes on $\mathcal{B}_\ell$ iff $\bar{t}$ vanishes on $\mathcal{B}_{\ell-3}$, and the standard zerocheck soundness applied to $\bar{t}$ gives soundness error $(\ell - 3)/|K|$.

4.3.2 The Univariate-Skip Protocol

The BitAnd reduction runs a deterministic-zerocheck on $\widehat{f}(\widehat{I}, X)$ over $D \times \mathcal{B}_{\ell_{\text{and}}}$, but with the inner sum over $\widehat{i} \in D$ handled by a univariate polynomial send (Gruen [Gru24, §5.1]) rather than additional sumcheck rounds:

1. The verifier samples $\bar{r}_x \leftarrow K^{\ell_{\text{and}}-3}$ and sends it to the prover. Both parties set $r_x := (\sigma_0, \sigma_1, \sigma_2, \bar{r}_{x,0}, \dots, \bar{r}_{x,\ell_{\text{and}}-4})$.
2. The prover sends the univariate polynomial

$$g(\widehat{I}) := \sum_{x \in \mathcal{B}_{\ell_{\text{and}}}} \widehat{f}(\widehat{I}, x) \cdot \widetilde{\text{eq}}_{\ell_{\text{and}}}(r_x, x) \in K[\widehat{I}],$$

by sending its evaluations on 64 points outside D . The polynomial has degree at most $2 \cdot (2^6 - 1) = 126$, and these 64 values together with the 64 prescribed zeros on D determine g uniquely.

3. The verifier checks that g vanishes on D (using the prescribed zeros). It then samples $r_{\widehat{i}} \leftarrow K$ and sends it to the prover.

4. The parties run an ℓ_{and} -round sumcheck on the claim

$$g(r_{\hat{i}}) \stackrel{?}{=} \sum_{x \in \mathcal{B}_{\ell_{\text{and}}}} \hat{f}(r_{\hat{i}}, x) \cdot \tilde{\mathbf{eq}}_{\ell_{\text{and}}}(r_x, x).$$

At the end, the sumcheck produces a challenge $r'_x \in K^{\ell_{\text{and}}}$ and a target s , with the assertion that $s \stackrel{?}{=} \hat{f}(r_{\hat{i}}, r'_x) \cdot \tilde{\mathbf{eq}}_{\ell_{\text{and}}}(r_x, r'_x)$.

5. The prover sends scalars $\alpha_a, \alpha_b, \alpha_c \in K$, claimed to equal $\hat{a}(r_{\hat{i}}, r'_x), \hat{b}(r_{\hat{i}}, r'_x), \hat{c}(r_{\hat{i}}, r'_x)$ respectively. The verifier evaluates $\tilde{\mathbf{eq}}_{\ell_{\text{and}}}(r_x, r'_x)$ on its own and checks

$$s \stackrel{?}{=} (\alpha_a \cdot \alpha_b - \alpha_c) \cdot \tilde{\mathbf{eq}}_{\ell_{\text{and}}}(r_x, r'_x).$$

The protocol's output is the triple of evaluation claims $(\hat{a}(r_{\hat{i}}, r'_x) = \alpha_a, \hat{b}(r_{\hat{i}}, r'_x) = \alpha_b, \hat{c}(r_{\hat{i}}, r'_x) = \alpha_c)$, to be discharged by the shift reduction (§4.7).

Soundness. A dishonest prover violating $a[x] \wedge b[x] = c[x]$ on some constraint induces a nonzero $\hat{f}$ on $D \times \mathcal{B}_{\ell_{\text{and}}}$. By the deterministic-zerocheck analysis of §4.3.1, there exists $\hat{i}^* \in D$ such that $\bar{g}(\hat{i}^*) \neq 0$ except with probability $(\ell_{\text{and}} - 3)/|K|$ over $\bar{r}_x$, where $\bar{g}(\hat{I}) := \sum_x \hat{f}(\hat{I}, x) \cdot \tilde{\mathbf{eq}}_{\ell_{\text{and}}}(r_x, x)$ is the polynomial the honest prover would send. Conditioned on this, the verifier's check that g vanishes on D forces $g \neq \bar{g}$ as polynomials, so by univariate Schwartz–Zippel, $g(r_{\hat{i}}) = \bar{g}(r_{\hat{i}})$ except with probability $2 \cdot (2^6 - 1)/|K|$ over $r_{\hat{i}}$. The sumcheck on the (now-wrong) claim adds another $2\ell_{\text{and}}/|K|$ (round polynomials have individual degree ≤ 2). Soundness of the final triple-claim follows by univariate Schwartz–Zippel on $\alpha_a, \alpha_b, \alpha_c$, contributing $O(1)/|K|$. Total soundness error is $O(\ell_{\text{and}})/|K|$.

4.3.3 Prover Algorithm

The dominant cost is the univariate send: the prover must evaluate $g(\hat{I})$ at 64 points outside D . We sketch the algorithm, called the *Rijndael zerocheck*, which performs most of the inner work entirely in $\mathbb{F}_{2^8}$ and embeds into K only once every 8 cube points. We refine the parameter D : we fix a 7-dimensional $\mathbb{F}_2$ -subspace $D' \supset D$ with $D' \subset \mathbb{F}_{2^8}$, so $|D' \setminus D| = 64$. The prover evaluates g on $D' \setminus D$.

Lookup-based extrapolation. The core primitive is a precomputed map $\text{Extrap}: \mathbb{F}_2^{64} \rightarrow \mathbb{F}_{2^8}^{64}$ that, on input a bit-vector $p = (p_0, \dots, p_{63})$ viewed as the values on D of a univariate polynomial $P \in K[\hat{I}]$ of degree < 64 , returns $(\iota^{-1}(P(\hat{i})))_{\hat{i} \in D' \setminus D}$ (which lies in $\mathbb{F}_{2^8}^{64}$ because $D' \subset \mathbb{F}_{2^8}$ and P is $\mathbb{F}_2$ -valued on D). Since Extrap is $\mathbb{F}_2$ -linear, it can be implemented by splitting the 64-bit input into eight 8-bit chunks, precomputing for each chunk a length-256 table of Extrap outputs on inputs nonzero on that chunk alone, and combining via 8 table lookups and 7 length-64 byte-wise XORs per invocation, following the lookup-table extrapolation technique of Hu et al. [Hu+25]. Total table size is $8 \cdot 256 \cdot 64$ bytes = 128 KiB.

Inner-outer structure. The prover tensor-expands $\bar{r}_x$ once into the table $\text{TensorExpand}_{\ell_{\text{and}}-3}(\bar{r}_x) = (\tilde{\mathbf{eq}}_{\ell_{\text{and}}-3}(\bar{r}_x, v))_{v \in \mathcal{B}_{\ell_{\text{and}}-3}}$. Then it iterates over outer index $v \in \mathcal{B}_{\ell_{\text{and}}-3}$, and for each v accumulates a length-64 vector temp in $\mathbb{F}_{2^8}$ by iterating $u \in \mathcal{B}_3$ and computing, for $x := u \parallel v$ and each $\hat{i} \in D' \setminus D$,

$$\text{temp}[\hat{i}] += (\text{Extrap}(a[x])[\hat{i}] \cdot \text{Extrap}(b[x])[\hat{i}] - \text{Extrap}(c[x])[\hat{i}]) \cdot \tilde{\mathbf{eq}}_3(\rho, u).$$

At the end of the inner loop, the entries of temp are inverse-embeddings of $\sum_{u \in \mathcal{B}_3} \hat{f}(\hat{i}, u \parallel v) \cdot \tilde{\mathbf{eq}}_3(\sigma, u)$. The prover then embeds via ι , scales by $\tilde{\mathbf{eq}}_{\ell_{\text{and}}-3}(\bar{r}_x, v)$, and accumulates into a length-64 outer accumulator over K . The final accumulator contains $(g(\hat{i}))_{\hat{i} \in D' \setminus D}$. The asymptotic cost is dominated by the inner $\mathbb{F}_{2^8}$ work, with $64 \mathbb{F}_{2^8} \rightarrow K$ embeddings and K -multiplications per 8 cube points. Further speedups are available via lookup-table-based additive-NTT techniques.

Preparing the sumcheck. For the sumcheck in step 4 of §4.3.2, the prover needs the value tables of $\hat{a}(r_{\hat{i}}, X), \hat{b}(r_{\hat{i}}, X), \hat{c}(r_{\hat{i}}, X)$ on $\mathcal{B}_{\ell_{\text{and}}}$. By the partial-specialization formula of §2.2.1, each table is obtained by precomputing the 64 Lagrange weights $(\delta_D(r_{\hat{i}}, \hat{i}))_{i \in \mathcal{B}_6}$ once and then, for each constraint $x \in \mathcal{B}_{\ell_{\text{and}}}$, taking the $\mathbb{F}_2$ -coefficient subset sum of these weights with bits drawn from $a[x], b[x], c[x]$ respectively.

4.4 The Zero Reduction

The Zero reduction transforms the satisfaction of the n_{zero} Zero constraints (§3.4) into a single evaluation claim on the oblong-multilinearization $\hat{d}$ of the Zero constraint array. Alone among the four constraint reductions it costs nothing: it adds no prover message and no sumcheck round, and it draws no challenges of its own beyond an occasional extension of the BitAnd reduction's. Its whole content is the observation that a Zero constraint is *linear*, so that the polynomial whose vanishing is at stake is $\hat{d}$ itself rather than a product of witness multilinear. A multilinear that vanishes on the cube is the zero polynomial; a single evaluation at a point the prover could not predict therefore certifies the constraint outright, with no sumcheck needed to get there. The reduction's only cost is one further constraint array in the batch consumed by the shift reduction (§4.7).

Write $d[i, x] \in \mathbb{F}_2$ for bit i of the word $d[x]$, let $\hat{d}(I, X)$ be the multilinear extension of d on $\mathcal{B}_6 \times \mathcal{B}_{\ell_{\text{zero}}}$, and let $\hat{d}(\hat{I}, X)$ be its oblong-multilinearization (§2.2.1) with $k = 6$, of degree < 64 in $\hat{I}$ and multilinear in $X_0, \dots, X_{\ell_{\text{zero}}-1}$. Satisfaction of the Zero constraints is exactly the claim that $\hat{d}$ vanishes on $D \times \mathcal{B}_{\ell_{\text{zero}}}$, and since $\hat{d}$ is determined by its values there, this is in turn equivalent to $\hat{d}$ being the zero polynomial.

The evaluation point. The reduction runs directly after the BitAnd reduction (§4.3.2) and evaluates $\hat{d}$ at the point that reduction has just produced: the univariate-skip challenge $r'_{\hat{i}} \in K$ of step 3, and the sumcheck output challenge $r'_x \in K^{\ell_{\text{and}}}$ of step 4. The verifier additionally samples $\bar{r}_d \leftarrow K^{\max(0, \ell_{\text{zero}} - \ell_{\text{and}})}$, and both parties set $\rho \in K^{\ell_{\text{zero}}}$ to be the length- ℓ_{zero} prefix of

$$(r'_{x,0}, \dots, r'_{x,\ell_{\text{and}}-1}, \bar{r}_d, 0, \dots),$$

that is, a prefix of r'_x when $\ell_{\text{zero}} \leq \ell_{\text{and}}$ and r'_x extended by fresh challenges otherwise. Every coordinate of ρ is uniform on K and independent of the others: those inherited from the BitAnd sumcheck are its per-round verifier challenges, and $\bar{r}_d$ is freshly sampled.

The reduction's output is the single evaluation claim

$$\hat{d}(r_{\hat{i}}, \rho) = 0,$$

which joins the other reductions' outputs in the batch of §4.7.1. It is already stated at the shared oblong point $r_{\hat{i}}$, so the fold of §4.7.1 does not apply to it. The prover sends nothing: the claimed value is the constant 0, and it is the shift reduction, running against the committed witness, that certifies it.

Soundness. Suppose the prover data violates some Zero constraint, so that $d[i^*, x^*] = 1$ for some $i^* \in \mathcal{B}_6$ and $x^* \in \mathcal{B}_{\ell_{\text{zero}}}$. Then $\hat{d}$ is not the zero polynomial: it takes the value $d[i^*, x^*] = 1$ at the cube point $(\hat{i}^*, x^*) \in D \times \mathcal{B}_{\ell_{\text{zero}}}$.

Moreover $\hat{d}$ is fixed before any challenge is drawn. The array d is a function of the prover data w , which the packed-witness oracle of phase 1 pins down before the first challenge is sampled, so a prover cannot adapt d to the point at which it will be evaluated. As $\hat{d}$ has degree < 64 in $\hat{I}$ and degree ≤ 1 in each of $X_0, \dots, X_{\ell_{\text{zero}}-1}$, and $(r_{\hat{i}}, \rho)$ is uniform on $K^{1+\ell_{\text{zero}}}$, Schwartz–Zippel gives

$$\Pr[\hat{d}(r_{\hat{i}}, \rho) = 0] \leq \frac{63 + \ell_{\text{zero}}}{|K|}.$$

Conditioned on $\hat{d}(r_{\hat{i}}, \rho) \neq 0$, the random batching of §4.7.1 preserves a false claim except with probability $O(1)/|K|$, and the shift reduction then rejects it. Total soundness error is $O(\ell_{\text{zero}})/|K|$.

On reusing the BitAnd point. Sharing (r_i, r'_i) with the BitAnd reduction is sound despite the resulting correlation. The bound above is a statement about a bad event for the Zero constraints, and it appeals only to the marginal distribution of (r_i, ρ) , which reuse leaves untouched; the analysis of §4.3.2 does the same for its own bad event, and neither conditions on the other. Unsatisfiable prover data violates a constraint of some class, and the reduction for that class catches it with the probability computed for it in isolation. What both analyses do require is that every challenge be drawn after w is committed, which the phase ordering of §4.1 guarantees.

The BitAnd reduction’s *output* point is what makes the argument this short. Its input challenge r_x would serve too, but it opens with the three deterministic coordinates $\sigma_0, \sigma_1, \sigma_2$ of §4.3.1; those axes carry no randomness, and bounding the error would first require absorbing them by the $\mathbb{F}_2$ -independence argument of that section—legitimate here, since a constraint array is an array of *bits* and $\hat{d}$ is therefore $\mathbb{F}_2$ -valued on $D \times \mathcal{B}_{\ell_{\text{zero}}}$, but an extra step, and one that would force $\ell_{\text{zero}} \geq 3$. The output point is uniform on all ℓ_{and} coordinates, so plain Schwartz–Zippel suffices.

4.5 The IntMul Reduction

The IntMul reduction transforms the satisfaction of the n_{mul} IntMul constraints (§3.6) into per-bit evaluation claims on the multilinear extensions $\tilde{a}, \tilde{b}, \tilde{c}_{\text{lo}}, \tilde{c}_{\text{hi}}$ of the IntMul constraint arrays—the 64 values $\tilde{a}(r_x, i), \dots$ at every bit-position $i \in \mathcal{B}_6$ of a shared constraint point $r_x \in K^{\ell_{\text{mul}}}$. Unlike the BitAnd reduction, which exploits that bitwise AND is just per-bit field multiplication, IntMul concerns an integer-arithmetic relation that has no direct expression in K ; we instead encode it as a multiplicative-group identity in K^* and reduce that identity via a tree of small-degree sumchecks.

4.5.1 Multiplying in the Exponent

Fix a generator $g \in K^*$, so g has order $2^\lambda - 1$. Recall the integer encoding $\llbracket a \rrbracket[x] = \sum_{i=0}^{63} 2^i \cdot a[i, x]$ of a constraint word (§3.6). For each $x \in \mathcal{B}_{\ell_{\text{mul}}}$, the integer relation $\llbracket a \rrbracket[x] \cdot \llbracket b \rrbracket[x] = 2^{64} \cdot \llbracket c_{\text{hi}} \rrbracket[x] + \llbracket c_{\text{lo}} \rrbracket[x]$ can be converted into the multiplicative-group identity

$$(g^{\llbracket a \rrbracket[x]})^{\llbracket b \rrbracket[x]} \stackrel{?}{=} (g^{2^{64}})^{\llbracket c_{\text{hi}} \rrbracket[x]} \cdot g^{\llbracket c_{\text{lo}} \rrbracket[x]}, \quad (4)$$

both sides taken in K^* . By the basic property of cyclic groups, (4) holds iff the integer relation holds modulo the order $2^\lambda - 1$.

Wraparound. The order of K^* is $2^\lambda - 1$. Assuming $\lambda \geq 128$, the integer quantities in (4) are bounded by

$$\llbracket a \rrbracket[x] \cdot \llbracket b \rrbracket[x] \leq (2^{64} - 1)^2 < 2^{128} - 1, \quad 2^{64} \cdot \llbracket c_{\text{hi}} \rrbracket[x] + \llbracket c_{\text{lo}} \rrbracket[x] \leq 2^{128} - 1,$$

where the right-hand bound is attained exactly when $\llbracket c_{\text{hi}} \rrbracket[x] = \llbracket c_{\text{lo}} \rrbracket[x] = 2^{64} - 1$. Since $\llbracket a \rrbracket[x] \cdot \llbracket b \rrbracket[x] < 2^{128} - 1 \leq 2^\lambda - 1$, the left-hand side never wraps modulo the group order.

When $\lambda > 128$ strictly, the right-hand side is likewise strictly below the order, $2^{64} \cdot \llbracket c_{\text{hi}} \rrbracket[x] + \llbracket c_{\text{lo}} \rrbracket[x] \leq 2^{128} - 1 < 2^\lambda - 1$, so neither side overflows and (4) holds if and only if the integer relation does; no further condition is required.

When $\lambda = 128$, the system default, the order is exactly $2^{128} - 1$, which the right-hand side can equal. A dishonest prover could exploit this by setting $\llbracket a \rrbracket[x] \cdot \llbracket b \rrbracket[x] = 0$ and $\llbracket c_{\text{hi}} \rrbracket[x] = \llbracket c_{\text{lo}} \rrbracket[x] = 2^{64} - 1$, since then both sides of (4) equal $1 \in K^*$ even though the integer relation fails. In this case $\llbracket a \rrbracket[x] \cdot \llbracket b \rrbracket[x]$ is even while $2^{64} \cdot \llbracket c_{\text{hi}} \rrbracket[x] + \llbracket c_{\text{lo}} \rrbracket[x]$ is odd, so the spurious case is ruled out by additionally requiring

$$\llbracket a \rrbracket[x] \cdot \llbracket b \rrbracket[x] \equiv \llbracket c_{\text{lo}} \rrbracket[x] \pmod{2}, \quad (5)$$

a condition on a single bit per constraint. Together, (4) and (5) are then the two conditions whose simultaneous satisfaction is equivalent to the IntMul relation $\llbracket a \rrbracket[x] \cdot \llbracket b \rrbracket[x] = 2^{64} \cdot \llbracket c_{\text{hi}} \rrbracket[x] + \llbracket c_{\text{lo}} \rrbracket[x]$: the multiplicative-group identity captures it modulo $2^\lambda - 1$, and the parity condition pins down the one residue the cyclic group cannot see.

4.5.2 Exponentiation Subprotocols

Identity (4) is verified by composing two generic exponentiation subprotocols. Throughout, fix the generator $g \in K^*$ and let $z \in \mathbb{F}_2^{\mathcal{B}_{\ell_{\text{mul}}} \times \mathcal{B}_k}$ be a bit vector, with $k = 6$ and entries $z[x, i] \in \mathbb{F}_2$, written with the constraint index first and the bit position last—the transpose of the bit indexing of §3.6, to which the shift reduction (§4.7) reverts—so that the bit index occupies the high variables, matching the layer structure of the product trees below and the implementation’s data layout. We index the $2^k = 64$ bit positions by $i \in \{0, \dots, 2^k - 1\}$, identified with the hypercube $\mathcal{B}_k$ via binary representation. Its multilinear extension $\tilde{z}(X, I) \in K[X_0, \dots, X_{\ell_{\text{mul}}-1}, I_0, \dots, I_{k-1}]^{\leq 1}$ has $\tilde{z}(X, i)$ for the multilinear section at the i -th hypercube point, with $\tilde{\text{eq}}_k(\cdot, i)$ the corresponding Lagrange basis function. The integer encoded at each constraint index is $\llbracket z \rrbracket[x] = \sum_{i=0}^{2^k-1} 2^i \cdot z[x, i]$, with $\llbracket \cdot \rrbracket$ as in §3.6.

The two operators. The *fixed-base* operator raises the public generator to the encoded exponents,

$$\text{gen-exp}(z)[x] := g^{\llbracket z \rrbracket[x]} \in K^*, \quad x \in \mathcal{B}_{\ell_{\text{mul}}},$$

while the *variable-base* operator raises a multilinear base $\tilde{f}(X) \in K[X_0, \dots, X_{\ell_{\text{mul}}-1}]^{\leq 1}$, whose cube values $f[x] := \tilde{f}(x)$ all lie in K^* , to the same exponents,

$$\text{var-exp}(f, z)[x] := f[x]^{\llbracket z \rrbracket[x]} \in K^*, \quad x \in \mathcal{B}_{\ell_{\text{mul}}}.$$

We denote by $\widetilde{\text{gen-exp}}(z)$ and $\widetilde{\text{var-exp}}(f, z)$ their multilinear extensions. The fixed-base subprotocol (§4.5.3) reduces an evaluation claim on $\widetilde{\text{gen-exp}}(z)$ to the 2^k per-bit evaluation claims $\tilde{z}(r'_x, i)$; the variable-base subprotocol (§4.5.4) reduces an evaluation claim on $\widetilde{\text{var-exp}}(f, z)$ to one evaluation claim on f together with the 2^k per-bit claims $\tilde{z}(r'_x, i)$. Both run the same GKR product tree that splits the exponent into its 2^k bits, differing only in how the product-tree leaves are discharged into the per-bit claims.

The product tree. Define the *selector* $\text{sel}(s, v) := 1 + (v - 1)s$, which returns v when the bit $s = 1$ and 1 when $s = 0$. Collect the 2^k leaves into a vector $L \in K^{\mathcal{B}_{\ell_{\text{mul}}} \times \mathcal{B}_k}$ with entries

$$L[x, i] := (f[x]^{2^i})^{z[x, i]} = \text{sel}(z[x, i], f[x]^{2^i}), \quad x \in \mathcal{B}_{\ell_{\text{mul}}}, \quad i \in \mathcal{B}_k,$$

where the second equality uses that $z[x, i] \in \{0, 1\}$. The operator output is the product of the leaves over the bit index,

$$\text{var-exp}(f, z)[x] = \prod_{i \in \mathcal{B}_k} L[x, i]$$

(for the fixed-base operator, $f[x] \equiv g$ and the product is $\text{gen-exp}(z)[x]$). We evaluate this product by a balanced binary tree of depth k , whose layers are vectors $L^{(j)} \in K^{\mathcal{B}_{\ell_{\text{mul}}} \times \mathcal{B}_{k-j}}$, for $j = 0, \dots, k$, with leaf layer $L^{(0)} := L$ and each node the product of its two children,

$$L^{(j)}[x, i] = L^{(j-1)}[x, (i, 0)] \cdot L^{(j-1)}[x, (i, 1)], \quad x \in \mathcal{B}_{\ell_{\text{mul}}}, \quad i \in \mathcal{B}_{k-j},$$

the trailing bit selecting the child; the root is $L^{(k)} = \text{var-exp}(f, z)$. Write $\tilde{L}^{(j)}$ for the multilinear extension of the layer vector $L^{(j)}$.

Starting from the root claim $s^{(k)} \stackrel{?}{=} \tilde{L}^{(k)}(r^{(k)})$ (the root layer has no node index, so we write $r_x^{(k)} := r^{(k)}$), the parties run the binary-tree GKR grand-product argument [Tha13] for $j = k, k-1, \dots, 1$. At layer j the parties hold a claim $\tilde{L}^{(j)}(r_x^{(j)}, \rho^{(j)})$ at an x -point $r_x^{(j)} \in K^{\ell_{\text{mul}}}$ and a folded node-index point $\rho^{(j)} \in K^{k-j}$. Expanding the product relation,

$$\tilde{L}^{(j)}(r_x^{(j)}, \rho^{(j)}) = \sum_{x \in \mathcal{B}_{\ell_{\text{mul}}}} \sum_{i \in \mathcal{B}_{k-j}} \tilde{\text{eq}}_{\ell_{\text{mul}}}(r_x^{(j)}, x) \cdot \tilde{\text{eq}}_{k-j}(\rho^{(j)}, i) \cdot \tilde{L}^{(j-1)}(x, i, 0) \cdot \tilde{L}^{(j-1)}(x, i, 1),$$

the parties run a $(\ell_{\text{mul}} + (k-j))$ -round MLE-check, whose composition polynomials have individual degree ≤ 2 . It produces the two child evaluations $\tilde{L}^{(j-1)}(r_x^{(j-1)}, \rho', 0)$ and $\tilde{L}^{(j-1)}(r_x^{(j-1)}, \rho', 1)$, which a line reduction [GKR15] collapses into the single layer- $(j-1)$ claim $\tilde{L}^{(j-1)}(r_x^{(j-1)}, \rho^{(j-1)})$ with $\rho^{(j-1)} \in$

K^{k-j+1} , the line challenge appended as the new trailing node coordinate. After k layers the parties hold a single leaf claim

$$s^{(0)} \stackrel{?}{=} \tilde{L}(r_x, \rho), \quad (r_x, \rho) \in K^{\ell_{\text{mul}}} \times K^k,$$

at the final x -point $r_x := r_x^{(0)}$ and fully folded node-index $\rho := \rho^{(0)}$. Soundness error is the standard GKR sum over layers, $O(k(k + \ell_{\text{mul}}))/|K|$.

Leaf extrapolation. Each subprotocol discharges the leaves one bit at a time, re-exposing them from the single folded claim $s^{(0)}$. In the general case—when the base is a witness value, as in the variable-base subprotocol—the prover sends the $2^k = 64$ leaf evaluations

$$s_i^{(0)} \stackrel{?}{=} \tilde{L}(r_x, i), \quad i \in \mathcal{B}_k,$$

and the verifier checks that they interpolate to the folded claim,

$$\sum_{i \in \mathcal{B}_k} \tilde{\text{eq}}_k(\rho, i) \cdot s_i^{(0)} \stackrel{?}{=} s^{(0)} = \tilde{L}(r_x, \rho).$$

The 2^k per-bit claims, all at the common x -point r_x , are the input to the leaf discharge. The interpolation check binds them to the GKR-verified product value $s^{(0)}$; a prover that sends inconsistent values is caught downstream, where each subprotocol reduces the $s_i^{(0)}$ against the committed witness with fresh randomness. When the base is public (the fixed-base subprotocol, §4.5.3), the prover instead re-exposes the exponent bits and the verifier reconstructs the leaves itself, folding this step into the same interpolation check.

4.5.3 Fixed-Base Exponentiation

The fixed-base subprotocol reduces a claim $s^{(k)} \stackrel{?}{=} \widetilde{\text{gen-exp}}(z)(r^{(k)})$ to the 2^k per-bit claims on $\tilde{z}$. With the constant base $f[x] \equiv g$, the leaf vector is affine in the exponent bit, so its multilinear extension is already linear,

$$\tilde{L}(X, i) = \text{sel}(\tilde{z}(X, i), g^{2^i}) = 1 + (g^{2^i} - 1) \cdot \tilde{z}(X, i),$$

since g^{2^i} is constant and $\tilde{z}(X, i)$ is multilinear in X .

The parties run the product-tree GKR loop (§4.5.2) on the root claim $s^{(k)} \stackrel{?}{=} \widetilde{\text{gen-exp}}(z)(r^{(k)})$, folding to the single leaf claim $s^{(0)} \stackrel{?}{=} \tilde{L}(r_x, \rho)$ at the common point $r'_x := r_x$. Because the powers g^{2^i} are public, the verifier can evaluate the leaf selector itself, so the prover re-exposes the exponent rather than the leaves: it sends the 2^k values

$$\alpha_{z,i} \stackrel{?}{=} \tilde{z}(r'_x, i), \quad i \in \mathcal{B}_k,$$

and the verifier reconstructs each leaf evaluation locally as $\text{sel}(\alpha_{z,i}, g^{2^i})$ and checks that the reconstructed leaves interpolate to the folded claim,

$$\sum_{i \in \mathcal{B}_k} \tilde{\text{eq}}_k(\rho, i) \cdot \text{sel}(\alpha_{z,i}, g^{2^i}) \stackrel{?}{=} s^{(0)} = \tilde{L}(r_x, \rho).$$

The 2^k per-bit claims $\alpha_{z,i}$, all at the common point r'_x , are the subprotocol output; a prover that sends inconsistent values is caught downstream, where they are reduced against the committed witness with fresh randomness. The subprotocol is therefore the depth- k GKR tree and one interpolation check, with total soundness error $O(k(k + \ell_{\text{mul}}))/|K|$.

The subprotocol never uses that the base is the generator: for any public base $h \in K^*$ it applies verbatim to the vector with entries $h^{\llbracket z \rrbracket [x]}$, reducing an evaluation claim on it to the per-bit claims on $\tilde{z}$, with the leaf constants g^{2^i} replaced by h^{2^i} . The combined protocol invokes this with $h = g^{2^{64}}$ for the high-half factor (§4.5.5).

4.5.4 Variable-Base Exponentiation

The variable-base subprotocol reduces a claim $s^{(k)} \stackrel{?}{=} \widetilde{\text{var-exp}}(f, z)(r^{(k)})$ to one claim on $\tilde{f}$ and the 2^k per-bit claims on $\tilde{z}$. The parties first run the product-tree GKR loop and leaf extrapolation (§4.5.2), obtaining 2^k leaf claims $s_i^{(0)} \stackrel{?}{=} \tilde{L}(r_x, i)$ at the common point r_x . Unlike the fixed-base case, the leaves couple the base and the exponent: the entry $L[x, i] = \text{sel}(z[x, i], f[x]^{2^i}) = 1 + z[x, i] \cdot (f[x]^{2^i} - 1)$. Naively reducing the leaf claim to a claim on $\tilde{f}$ would involve the polynomial $\tilde{f}(X)^{2^i}$, whose degree blows up; we sidestep this using the Frobenius endomorphism $\varphi(\alpha) = \alpha^2$ (§2.1). Applying φ^{-i} (well-defined because φ is invertible on K , with inverse $\varphi^{\lambda-1}$) to the leaf claim:

$$\varphi^{-i}(s_i^{(0)}) \stackrel{?}{=} \sum_{x \in \mathcal{B}_{\ell_{\text{mul}}}} \tilde{\text{eq}}_{\ell_{\text{mul}}}(\varphi^{-i}(r_x), x) \cdot (1 + \tilde{z}(x, i) \cdot (\tilde{f}(x) - 1)),$$

using that φ^{-i} is a ring automorphism that fixes the $\mathbb{F}_2$ -coefficients of $\tilde{\text{eq}}$ and of $\tilde{z}(\cdot, i)$, and that $\varphi^{-i}(\tilde{f}(x)^{2^i}) = \tilde{f}(x)$.

The right-hand side is now of degree ≤ 2 in $\tilde{f}$ and $\tilde{z}(\cdot, i)$. The verifier samples a single vector $\gamma \leftarrow K^k$ and the parties run a batched ℓ_{mul} -round sumcheck, using the multilinear Lagrange weights $\tilde{\text{eq}}_k(\gamma, i)$ as batching coefficients in place of 2^k independent ones:

$$\sum_{i=0}^{2^k-1} \tilde{\text{eq}}_k(\gamma, i) \cdot \varphi^{-i}(s_i^{(0)}) \stackrel{?}{=} \sum_{x \in \mathcal{B}_{\ell_{\text{mul}}}} \sum_{i=0}^{2^k-1} \tilde{\text{eq}}_k(\gamma, i) \cdot \tilde{\text{eq}}_{\ell_{\text{mul}}}(\varphi^{-i}(r_x), x) \cdot (1 + \tilde{z}(x, i) \cdot (\tilde{f}(x) - 1)).$$

A single γ suffices because the weights $\tilde{\text{eq}}_k(\gamma, i)$ are the values at γ of the k -variate multilinear Lagrange basis, so a false claim survives with probability at most $k/|K|$ by Schwartz-Zippel. At the end, the parties hold a final sumcheck challenge $r'_x \in K^{\ell_{\text{mul}}}$ and claimed evaluations $\alpha_f \stackrel{?}{=} \tilde{f}(r'_x)$ and $\alpha_{z,i} \stackrel{?}{=} \tilde{z}(r'_x, i)$ for $i \in \{0, \dots, 2^k - 1\}$. The verifier evaluates each $\tilde{\text{eq}}_{\ell_{\text{mul}}}(\varphi^{-i}(r_x), r'_x)$ on its own, costing at most λ squarings per i to compute $\varphi^{-i}(r_x)$, and closes out the batched sumcheck. The claim $\alpha_f \stackrel{?}{=} \tilde{f}(r'_x)$ is the output base claim, and the 2^k per-bit claims $\alpha_{z,i} \stackrel{?}{=} \tilde{z}(r'_x, i)$ are the output exponent claims.

4.5.5 Combined Protocol

The IntMul reduction verifies the multiplicative-group identity (4) and the wraparound parity (5) for all $x \in \mathcal{B}_{\ell_{\text{mul}}}$ simultaneously, producing, for each of $\tilde{a}, \tilde{b}, \tilde{c}_{\text{lo}}, \tilde{c}_{\text{hi}}$, its 64 per-bit evaluations at a shared constraint point. Write

$$\tilde{A} := \widetilde{\text{gen-exp}}(a), \quad \tilde{C}_{\text{lo}} := \widetilde{\text{gen-exp}}(c_{\text{lo}}),$$

the multilinear extensions of the base- g fixed-base exponentiation vectors A, C_{lo} , whose entries $A[x] = g^{\llbracket a \rrbracket [x]}$ and $C_{\text{lo}}[x] = g^{\llbracket c_{\text{lo}} \rrbracket [x]}$ lie in K^* . The high half enters the integer relation weighted by 2^{64} , so its factor is a fixed-base exponentiation with base $g^{2^{64}}$: write C_{hi} for the vector with entries

$$C_{\text{hi}}[x] := (g^{2^{64}})^{\llbracket c_{\text{hi}} \rrbracket [x]} = g^{2^{64} \cdot \llbracket c_{\text{hi}} \rrbracket [x]} \in K^*,$$

and $\tilde{C}_{\text{hi}}$ for its multilinear extension. The fixed-base subprotocol (§4.5.3) applies to C_{hi} verbatim, with the public leaf constants g^{2^i} replaced by $(g^{2^{64}})^{2^i} = g^{2^{64+i}}$; folding the high half straight into a base- $g^{2^{64}}$ exponentiation avoids ever forming the base- g vector $g^{\llbracket c_{\text{hi}} \rrbracket [x]}$ and its 2^{64} -th power. In this notation identity (4) becomes the pointwise relation

$$\text{var-exp}(A, b)[x] = C_{\text{lo}}[x] \cdot C_{\text{hi}}[x], \quad x \in \mathcal{B}_{\ell_{\text{mul}}},$$

since $\text{var-exp}(A, b)[x] = A[x]^{\llbracket b \rrbracket [x]} = g^{\llbracket a \rrbracket [x] \cdot \llbracket b \rrbracket [x]}$ while $C_{\text{lo}}[x] \cdot C_{\text{hi}}[x] = g^{\llbracket c_{\text{lo}} \rrbracket [x] + 2^{64} \cdot \llbracket c_{\text{hi}} \rrbracket [x]}$. The base A of the variable-base exponentiation is itself the output of a fixed-base exponentiation, so verifying the relation chains the two subprotocols of §4.5.2.

We organize the reduction into six phases (0 through 5), writing r_k for the principal verifier challenge of phase k where unambiguous.

Phase 0 (setup). The verifier samples $r_0 \leftarrow K^{\ell_{\text{mul}}}$ and sends it to the prover, who returns a single value $y_0 \in K$. This y_0 is claimed to equal both readings of the relation at r_0 :

$$y_0 \stackrel{?}{=} \widetilde{\text{var-exp}}(A, b)(r_0), \quad y_0 \stackrel{?}{=} \sum_{x \in \mathcal{B}_{\ell_{\text{mul}}}} \tilde{\mathbf{eq}}_{\ell_{\text{mul}}}(r_0, x) \cdot C_{\text{lo}}[x] \cdot C_{\text{hi}}[x],$$

the right-hand sum being the multilinear extension of the relation's right-hand side evaluated at r_0 . The first reading is discharged by phases 1–4, the second by the *deferred right-hand-side sumcheck* opened in phase 2. A single y_0 satisfies both readings iff the two multilinear sides agree at r_0 , which by Schwartz–Zippel certifies the pointwise relation except with probability $\ell_{\text{mul}}/|K|$.

Phase 1 (variable-base tree and Frobenius twist). The parties run the product-tree GKR loop and leaf extrapolation (§4.5.2) of the variable-base subprotocol on the root claim $y_0 \stackrel{?}{=} \widetilde{\text{var-exp}}(A, b)(r_0)$, with base $\tilde{A}$ and exponent b . This reduces it to 64 leaf claims

$$s_i^{(0)} \stackrel{?}{=} \tilde{L}(r_1, i), \quad L[x, i] = 1 + (A[x]^{2^i} - 1) \cdot b[x, i], \quad i \in \mathcal{B}_6,$$

all at the common leaf x -point $r_1 \in K^{\ell_{\text{mul}}}$, identifying the product tree's leaf x -point r_x with the phase-1 challenge r_1 . Because the leaves couple the base and the exponent, the parties then apply the Frobenius twist φ^{-i} to each leaf claim and batch across i with weights $\tilde{\mathbf{eq}}_6(\gamma, i)$ for a verifier challenge $\gamma \leftarrow K^6$ (§4.5.4). Phase 1 closes with the resulting variable-base sumcheck claim

$$\sum_{i \in \mathcal{B}_6} \tilde{\mathbf{eq}}_6(\gamma, i) \cdot \varphi^{-i}(s_i^{(0)}) \stackrel{?}{=} \sum_{x \in \mathcal{B}_{\ell_{\text{mul}}}} \sum_{i \in \mathcal{B}_6} \tilde{\mathbf{eq}}_6(\gamma, i) \cdot \tilde{\mathbf{eq}}_{\ell_{\text{mul}}}(\varphi^{-i}(r_1), x) \cdot (1 + (\tilde{A}(x) - 1) \cdot \tilde{b}(x, i)),$$

a degree-2 sumcheck in $\tilde{A}$ and $\tilde{b}$.

Phase 2 (batched sumcheck). The parties run the variable-base sumcheck claim that phase 1 set up, batching into the same ℓ_{mul} -round sumcheck the *deferred right-hand-side reading* of phase 0,

$$y_0 \stackrel{?}{=} \sum_{x \in \mathcal{B}_{\ell_{\text{mul}}}} \tilde{\mathbf{eq}}_{\ell_{\text{mul}}}(r_0, x) \cdot \tilde{C}_{\text{lo}}(x) \cdot \tilde{C}_{\text{hi}}(x),$$

a degree-2 sumcheck in $\tilde{C}_{\text{lo}}$ and $\tilde{C}_{\text{hi}}$. Sharing the common challenge $r_2 \in K^{\ell_{\text{mul}}}$, the sumchecks output the three root claims $\tilde{A}(r_2)$, $\tilde{C}_{\text{lo}}(r_2)$, and $\tilde{C}_{\text{hi}}(r_2)$, the last a base- $g^{2^{64}}$ exponentiation claim. The variable-base subprotocol also outputs the 64 per-bit exponent claims $\tilde{b}(r_2, i)$ at the constraint point r_2 ; the verifier recombines them into the single claim $\tilde{b}(r_2, r_I^b) = \sum_{i \in \mathcal{B}_6} \tilde{\mathbf{eq}}_6(r_I^b, i) \tilde{b}(r_2, i)$ by sampling $r_I^b \leftarrow K^6$, carrying one exponent claim rather than 64 into the later phases. The phase hands the three root claims to phase 3 and the single recombined exponent claim $\tilde{b}(r_2, r_I^b)$ to phase 4.

Phase 3 (fixed-base trees). The parties run the three fixed-base product trees on $\tilde{A}(r_2)$, $\tilde{C}_{\text{lo}}(r_2)$, $\tilde{C}_{\text{hi}}(r_2)$, all rooted at the common point r_2 (the $\tilde{A}$ and $\tilde{C}_{\text{lo}}$ trees with base g , the $\tilde{C}_{\text{hi}}$ tree with base $g^{2^{64}}$). The binary-tree grand-product loop (§4.5.2) is run down through *all but the last layer*, folding both the bit-index and the constraint variable and batching the three trees to a common point r_3 . Phase 3 ends holding, for each of $a, c_{\text{lo}}, c_{\text{hi}}$, an MLE-check at r_3 on the product of the two layer-0 child *selectors*: writing $\tilde{L}_{c_{\text{lo}}}$ for the gen-exp leaf with boolean values $\tilde{L}_{c_{\text{lo}}}[x, i] = \text{sel}(c_{\text{lo}}[x, i], g^{2^i})$, the claim is on $\tilde{L}_{c_{\text{lo}}}(r_3, \rho, 0) \cdot \tilde{L}_{c_{\text{lo}}}(r_3, \rho, 1)$ with $\rho \in K^5$ the folded lower bit-index, and analogously for a and for c_{hi} (whose leaf uses the high-half constants, $\tilde{L}_{c_{\text{hi}}}[x, i] = \text{sel}(c_{\text{hi}}[x, i], g^{2^{64+i}})$). No local inversion is performed: with the bit-index folded, the selectors are kept symbolic and the conversion to $\tilde{c}_{\text{lo}}$ is deferred to phase 5.

Phase 4 (last layer, re-randomization, and wraparound). A single batched sumcheck over the constraint variable runs three checks at once:

- the last product-tree layer of each of $a, c_{\text{lo}}, c_{\text{hi}}$ —the MLE-check of the layer-0 selector products at r_3 ;

- the re-randomization of the exponent: an MLE-check point-switching the single claim $\tilde{b}(r_2, r_I^b)$ from the phase-2 constraint point r_2 to the shared point r_x ;
- the *wraparound parity* zerocheck (5), $\sum_{x \in \mathcal{B}_{\ell_{\text{mul}}}} \tilde{\text{eq}}_{\ell_{\text{mul}}}(r_2, x) [\tilde{a}(x, 0) \tilde{b}(x, 0) - \tilde{c}_{\text{lo}}(x, 0)] \stackrel{?}{=} 0$, reusing the phase-2 challenge r_2 —available for free because the $\tilde{b}$ re-randomization already evaluates at constraint r_2 .

The batch folds the constraint variable to the shared point $r_x \in K^{\ell_{\text{mul}}}$ and the tree bit-index to $r_I \in K^6$. Its outputs are the three leaf-selector claims $\tilde{L}_a(r_x, r_I)$, $\tilde{L}_{c_{\text{lo}}}(r_x, r_I)$, $\tilde{L}_{c_{\text{hi}}}(r_x, r_I)$, the exponent claim $\tilde{b}(r_x, r_I^b)$, and the bit-0 wraparound claims $\tilde{a}(r_x, 0)$, $\tilde{b}(r_x, 0)$, $\tilde{c}_{\text{lo}}(r_x, 0)$.

Phase 5 (leaf reconstruction). The prover sends the 4×64 per-bit values $\tilde{a}(r_x, i)$, $\tilde{b}(r_x, i)$, $\tilde{c}_{\text{lo}}(r_x, i)$, $\tilde{c}_{\text{hi}}(r_x, i)$ for $i \in \mathcal{B}_6$; these are the IntMul output. The verifier binds them to the phase-4 claims rather than collapsing them: for each of $a, c_{\text{lo}}, c_{\text{hi}}$ it forms the selector recombination

$$\sum_{i \in \mathcal{B}_6} \tilde{\text{eq}}_6(r_I, i) \text{sel}(\tilde{a}(r_x, i), g^{2^i}) \stackrel{?}{=} \tilde{L}_a(r_x, r_I),$$

using the leaf constant g^{2^i} for a and c_{lo} and the high-half constant $g^{2^{64+i}}$ for c_{hi} ; for b it tests $\sum_{i \in \mathcal{B}_6} \tilde{\text{eq}}_6(r_I^b, i) \tilde{b}(r_x, i)$ against $\tilde{b}(r_x, r_I^b)$, and it tests the $i = 0$ values against the wraparound claims. The sent values are thereby bound to the GKR-verified products, and to the committed witness by the shift reduction downstream.

The 4×64 output claims, all at the shared constraint point $r_x \in K^{\ell_{\text{mul}}}$, are passed to the shift reduction (§4.7). Total soundness error is $O(\ell_{\text{mul}})/|K|$, dominated by the depth-6 GKR trees and the batched sumchecks of phases 1–4, the last also carrying the wraparound parity zerocheck and the leaf-reconstruction binding.

4.5.6 Prover Algorithm

The dominant cost is preparing and maintaining tables of values for the various exponentiated multilinearars at the relevant cube points; this involves substantial work in K . We highlight a few implementation considerations.

Computing the leaf tables. For each of the four exponentiations, the prover initializes the leaf tables $(L[x, i])_{x \in \mathcal{B}_{\ell_{\text{mul}}}}$ for $i = 0, \dots, 63$ from the base values $f[x]$ and the bits $z[x, i]$. Since each $z[x, i] \in \mathbb{F}_2$, the entry $L[x, i]$ is either 1 or $f[x]^{2^i} = \varphi^i(f[x])$. The A and C_{lo} exponentiations use base g , so the powers g^{2^i} are precomputed once into a table indexed by i ; the C_{hi} exponentiation uses base $g^{2^{64}}$, whose powers $g^{2^{64+i}}$ form a second such table. In each case the leaves are obtained by a bit-controlled select, so no per-entry exponentiation or Frobenius application is needed. For the variable-base exponentiation (base $\tilde{A}$, exponent b), the prover applies φ^i to each entry of the $\tilde{A}$ -table, which costs i squarings per entry.

Sumcheck round computation. Each layer of the GKR loop runs a batched sumcheck with composing polynomials of individual degree 2 (plus the equality indicator), over ℓ_{mul} rounds. Standard sumcheck-prover algorithms (e.g. Thaler [Tha22, §4.1]) apply directly. The Frobenius step’s sumcheck has composing polynomials of degree ≤ 2 in $\tilde{f}$ and $\tilde{z}(\cdot, i)$ (plus the equality indicator), so the per-round prover work is also linear in the table size.

Frobenius application. Computing $\varphi^{-i}(r_x)$ for $i = 0, \dots, 63$, needed by the verifier in the variable-base Frobenius step, costs at most $\lambda \cdot 64$ field squarings in total. The prover precomputes these quantities once and reuses them across the batched sumcheck.

4.6 The BinMul Reduction

The BinMul reduction transforms the satisfaction of the n_{binmul} BinMul constraints (§3.7) into evaluation claims on the multilinear extensions of the six BinMul constraint arrays. The BitAnd and IntMul constraints act on $\mathbb{F}_2$ -valued arrays that carry one relation per bit; a BinMul constraint instead asserts a single multiplicative relation in the GHASH field $\mathbb{F}_{2^{128}}$ per constraint. Packing each operand's two words into one field element (§3.7) turns the constraint into a product relation between $\mathbb{F}_{2^{128}}$ -valued multilinear, which an ordinary zerocheck discharges.

4.6.1 Packing the Operands

Recall that $\mathbb{F}_{2^{128}} \cong \mathbb{F}_2[X]/(X^{128} + X^7 + X^2 + X + 1)$ is the GHASH field (§2.1) and write $\gamma \in \mathbb{F}_{2^{128}}$ for the class of X , so that $(1, \gamma, \dots, \gamma^{127})$ is the monomial $\mathbb{F}_2$ -basis in which $\langle\langle \cdot \rangle\rangle$ encodes operands. In the system default $\lambda = 128$ we have $\mathbb{F}_{2^{128}} = K$; when $128 \nmid \lambda$ strictly, the reduction runs over K with the operand values confined to the subfield $\mathbb{F}_{2^{128}} \subseteq K$ and γ taken as the image of X under the fixed embedding.

For each of the six constraint arrays write $\tilde{a}_{\text{lo}}(I, X), \tilde{a}_{\text{hi}}(I, X), \dots, \tilde{c}_{\text{hi}}(I, X)$ for the multilinear extensions, with bit-index $I \in \mathcal{B}_6$ and constraint-index $X \in \mathcal{B}_{\ell_{\text{binmul}}}$. Define the packed operand multilinear $\tilde{A}, \tilde{B}, \tilde{C} \in K[X_0, \dots, X_{\ell_{\text{binmul}}-1}]^{\leq 1}$ by

$$\tilde{A}(X) := \sum_{i \in \mathcal{B}_6} \gamma^i \cdot \tilde{a}_{\text{lo}}(i, X) + \sum_{i \in \mathcal{B}_6} \gamma^{64+i} \cdot \tilde{a}_{\text{hi}}(i, X),$$

and analogously $\tilde{B}$ from $(b_{\text{lo}}, b_{\text{hi}})$ and $\tilde{C}$ from $(c_{\text{lo}}, c_{\text{hi}})$, where in each case $i \in \mathcal{B}_6$ is identified with $\{0, \dots, 63\}$ via binary representation. Each is multilinear in X and $\mathbb{F}_{2^{128}}$ -valued, and at a hypercube point $x \in \mathcal{B}_{\ell_{\text{binmul}}}$ it evaluates to the encoded field element

$$\tilde{A}(x) = \langle\langle a_{\text{lo}}[x], a_{\text{hi}}[x] \rangle\rangle = A[x] \in \mathbb{F}_{2^{128}},$$

since $\tilde{a}_{\text{lo}}(i, x) = a_{\text{lo}}[i, x]$ is the i -th bit of the word $a_{\text{lo}}[x]$, and likewise for B and C .

4.6.2 The Zerocheck

Satisfaction of the BinMul constraints is the claim that $A[x] \cdot B[x] = C[x]$ for every $x \in \mathcal{B}_{\ell_{\text{binmul}}}$, i.e., that

$$f(X) := \tilde{A}(X) \cdot \tilde{B}(X) - \tilde{C}(X)$$

vanishes on $\mathcal{B}_{\ell_{\text{binmul}}}$. The operands are already K -valued, so, unlike the BitAnd reduction (§4.3), no univariate skip or Rijndael-field specialization applies; the reduction is an ordinary product-check zerocheck.

1. The verifier samples $r_z \leftarrow K^{\ell_{\text{binmul}}}$ and sends it to the prover.
2. The parties run an ℓ_{binmul} -round sumcheck on

$$0 \stackrel{?}{=} \sum_{x \in \mathcal{B}_{\ell_{\text{binmul}}}} \tilde{\mathbf{eq}}_{\ell_{\text{binmul}}}(r_z, x) \cdot (\tilde{A}(x) \tilde{B}(x) - \tilde{C}(x)).$$

The composing polynomial $\tilde{\mathbf{eq}}_{\ell_{\text{binmul}}}(r_z, X) \cdot (\tilde{A}(X) \tilde{B}(X) - \tilde{C}(X))$ has individual degree ≤ 3 , so the prover sends a degree-3 round polynomial each round. The sumcheck produces a challenge $r_x \in K^{\ell_{\text{binmul}}}$ and a target s , with the assertion that $s \stackrel{?}{=} \tilde{\mathbf{eq}}_{\ell_{\text{binmul}}}(r_z, r_x) \cdot (\tilde{A}(r_x) \tilde{B}(r_x) - \tilde{C}(r_x))$.

3. The prover sends the 6×64 per-bit values $\tilde{a}_{\text{lo}}(r_x, i), \tilde{a}_{\text{hi}}(r_x, i), \tilde{b}_{\text{lo}}(r_x, i), \tilde{b}_{\text{hi}}(r_x, i), \tilde{c}_{\text{lo}}(r_x, i), \tilde{c}_{\text{hi}}(r_x, i)$ for $i \in \mathcal{B}_6$. The verifier recombines them into the packed evaluations

$$\alpha_A := \sum_{i \in \mathcal{B}_6} \gamma^i \cdot \tilde{a}_{\text{lo}}(r_x, i) + \gamma^{64+i} \cdot \tilde{a}_{\text{hi}}(r_x, i),$$

and α_B, α_C likewise, evaluates $\tilde{\mathbf{eq}}_{\ell_{\text{binmul}}}(r_z, r_x)$ on its own, and checks

$$s \stackrel{?}{=} (\alpha_A \cdot \alpha_B - \alpha_C) \cdot \tilde{\mathbf{eq}}_{\ell_{\text{binmul}}}(r_z, r_x).$$

The reduction’s output is the 6×64 evaluation claims $\tilde{a}_{\text{lo}}(r_x, i), \dots, \tilde{c}_{\text{hi}}(r_x, i)$ at the shared constraint point $r_x \in K^{\ell_{\text{binmul}}}$. These are in the same per-bit form as the IntMul output (§4.5) and are passed to the shift reduction (§4.7). Soundness is that of the standard zerocheck, with error $O(\ell_{\text{binmul}})/|K|$.

4.6.3 Prover Algorithm

The prover forms the value tables of $\tilde{A}, \tilde{B}, \tilde{C}$ on $\mathcal{B}_{\ell_{\text{binmul}}}$ by packing the bit-columns of the constraint arrays: each entry $\tilde{A}(x) = A[x]$ is assembled from the 128 bits of $(a_{\text{lo}}[x], a_{\text{hi}}[x])$ by a bit-controlled XOR of the precomputed basis constants $\gamma^0, \dots, \gamma^{127}$, exactly as the IntMul leaf tables are formed by bit-controlled selects (§4.5.6). With the three tables in hand, the zerocheck sumcheck runs by standard algorithms (Thaler [Tha22, §4.1]), with per-round work linear in $2^{\ell_{\text{binmul}}}$ and a composing polynomial of individual degree ≤ 3 . The constraint arrays are themselves computed from the witness via the shift structure shared with the other reductions.

4.7 The Shift Reduction

The shift reduction consumes the evaluation claims produced by the Zero, BitAnd, IntMul, and BinMul reductions and transforms all of them into a single multilinear-evaluation claim on the witness $\tilde{w}$. It is the largest single component of the LIOP, with a sumcheck whose underlying domain contains the witness; the prover algorithm therefore has to exploit the sparsity of the constraint system to run in time close to the witness size.

For exposition we describe the reduction for a single oblong-multilinearized constraint array $\hat{z}(\hat{I}, X)$ over $D \times \mathcal{B}_{\ell_{\text{cons}}}$, with a single input evaluation claim $\alpha_z \stackrel{?}{=} \hat{z}(r'_i, r'_x)$. The protocol extends to multiple constraint arrays by random batching, with the verifier sampling fresh batching coefficients before the sumcheck begins. In the full Binius64 LIOP, the outputs of all four constraint reductions are first brought to a common oblong point r'_i over the bit-index and batched, as described in §4.7.1; a single shift reduction then processes the batched claim, over constraint arrays padded to a common $\ell_{\text{cons}} := \max(\ell_{\text{zero}}, \ell_{\text{and}}, \ell_{\text{mul}}, \ell_{\text{binmul}})$.

Only the oblong point must be shared. The reductions close at different constraint points— r'_x for BitAnd, ρ for Zero, and each multiplication reduction’s own final challenge—and a constraint point enters the derivation below solely through the constraint matrix $\tilde{Z}_{\text{cons}}$ of §4.7.2, which is a per-array object to begin with. Batching arrays z at points ρ_z with coefficients η_z therefore just replaces the single term $\tilde{Z}_{\text{cons}}(r'_x, y, S_1, O_1, S_2, O_2)$ by $\sum_z \eta_z \cdot \tilde{Z}_{\text{cons}}^z(\rho_z, y, S_1, O_1, S_2, O_2)$ throughout: the factorization of §4.7.3 is untouched, the prover accumulates the sparse index there from one tensor expansion per distinct point, and the verifier evaluates the per-array contributions separately at the end. The BitAnd and Zero points are moreover nested: when $\ell_{\text{zero}} \leq \ell_{\text{and}}$, the point ρ of §4.4 is a prefix of r'_x , so $\text{TensorExpand}_{\ell_{\text{zero}}}(\rho)$ is an intermediate stage of $\text{TensorExpand}_{\ell_{\text{and}}}(r'_x)$ and the two expansions cost no more than the larger one alone. The oblong point, by contrast, is shared out of necessity—it is carried by the $\tilde{\delta}_D$ and shift-indicator factors, which the sumcheck binds jointly for all arrays.

4.7.1 Unifying the Output Claims

The four constraint reductions present their outputs in two different shapes over the bit-index axis $\mathcal{B}_6$. The BitAnd reduction (§4.3), through its univariate skip (§4.3.2), already collapses the bit-index into a single oblong coordinate: its outputs are the oblong evaluations $\hat{a}(r'_i, r'_x), \hat{b}(r'_i, r'_x), \hat{c}(r'_i, r'_x)$, where $r'_i \in K$ is the univariate-skip challenge. The Zero reduction (§4.4) is likewise already in oblong form, its output being $\hat{d}(r'_i, \rho) = 0$ at the same r'_i . The IntMul and BinMul reductions instead leave the bit-index free: after its final sumcheck the prover computes and sends, for each output column z , the 64 per-bit evaluations $\tilde{z}(r_x, i)$ over $i \in \mathcal{B}_6$ at the reduction’s constraint point r_x —the four columns of IntMul phase 5 (§4.5) and the six columns of BinMul (§4.6.2), sixty-four coordinates apiece.

We unify the two shapes by folding each free bit-index against r'_i . From the 64 per-bit claims of a column, the oblong evaluation at r'_i is the Lagrange recombination

$$\hat{z}(r'_i, r_x) = \sum_{i \in \mathcal{B}_6} \delta_D(r'_i, \hat{i}) \cdot \tilde{z}(r_x, i),$$

which the verifier forms on its own from the sent values using the 64 oblong weights $\delta_D(r'_i, \hat{i})$ (§2.2.1). This places every IntMul and BinMul column on the same oblong footing as the BitAnd and Zero

columns, after which the verifier random-combines all columns into the single batched claim consumed below.

Ordering. The fold reuses the BitAnd univariate-skip challenge r_i as the common oblong point, so the two multiplication reductions must run *before* the BitAnd reduction. Their per-bit evaluations are sent to the verifier first, and only then does the BitAnd univariate skip sample r_i (§4.3.2, step 3). Drawing r_i after the multiplication outputs are committed is what binds those outputs: a prover who learned r_i beforehand could choose per-bit values folding to an arbitrary oblong claim. The LIOP therefore sequences the reductions as IntMul, BinMul, then BitAnd (phases 2–4 of §4.1).

4.7.2 Mathematizing Constraint Arrays

We first re-express the oblong-multilinearization $\widehat{z}(\widehat{I}, X)$ in terms of $\widetilde{w}$ and the shift indicators.

Constraint matrix. Define the six-dimensional $\mathbb{F}_2$ -valued matrix Z_{cons} of shape $n_{\text{cons}} \times n_{\text{words}} \times 64 \times 8 \times 64 \times 8$ by

$$Z_{\text{cons}}[x, y, s_1, o_1, s_2, o_2] := \mathbb{K}[(y, \text{op}(o_1), s_1, \text{op}(o_2), s_2) \in L_x^z],$$

where L_x^z is the list of shifted value indices generating $z[x]$ and $o_1, o_2 \in \mathcal{B}_3$ index ShiftOps as in §3.2. The constraint array can be re-expressed as

$$z[x] = \sum_{o_1, o_2 \in \mathcal{B}_3} \sum_{s_1, s_2 \in \mathcal{B}_6} \sum_{y \in \mathcal{B}_{\ell_{\text{words}}}} Z_{\text{cons}}[x, y, s_1, o_1, s_2, o_2] \cdot \text{op}(o_2)(\text{op}(o_1)(w[y], s_1), s_2).$$

Writing $\widetilde{Z}_{\text{cons}}$ for the multilinear extension of Z_{cons} , taking multilinear extensions and unrolling the composition identity (2) gives

$$\begin{aligned} \widehat{z}(I, X) = & \sum_{o_1, o_2 \in \mathcal{B}_3} \sum_{s_1, s_2 \in \mathcal{B}_6} \sum_{y \in \mathcal{B}_{\ell_{\text{words}}}} \sum_{k, j \in \mathcal{B}_6} \widetilde{Z}_{\text{cons}}(X, y, s_1, o_1, s_2, o_2) \\ & \cdot \widetilde{\text{shift-ind}}(I, k, s_2, o_2) \cdot \widetilde{\text{shift-ind}}(k, j, s_1, o_1) \cdot \widetilde{w}(j, y), \end{aligned}$$

where the shift indicators have been assembled into the interpolated indicator of (3), legitimately so because o_1 and o_2 are summed over the cube. Applying the oblong-multilinearization construction of §2.2.1 (introducing $\delta_D(\widehat{I}, \widehat{i})$ and summing $i \in \mathcal{B}_6$) yields the analogous expression for $\widehat{z}(\widehat{I}, X)$. Specializing at (r_i, r'_x) produces the explicit form

$$\begin{aligned} \alpha_z \stackrel{?}{=} & \sum_{o_1, o_2 \in \mathcal{B}_3} \sum_{s_1, s_2 \in \mathcal{B}_6} \sum_{j, k, i \in \mathcal{B}_6} \sum_{y \in \mathcal{B}_{\ell_{\text{words}}}} \delta_D(r_i, \widehat{i}) \cdot \widetilde{Z}_{\text{cons}}(r'_x, y, s_1, o_1, s_2, o_2) \\ & \cdot \widetilde{\text{shift-ind}}(i, k, s_2, o_2) \cdot \widetilde{\text{shift-ind}}(k, j, s_1, o_1) \cdot \widetilde{w}(j, y). \end{aligned} \quad (6)$$

This nested sum is over $\ell_{\text{words}} + 36$ variables.

The sumcheck. The parties discharge (6) by a single sumcheck over its variables $(O_1, S_1, O_2, S_2, J, K, I, Y)$.

Write $\widetilde{\delta}_D(r_i, I)$ for the multilinear extension in I of the 64 oblong weights $i \mapsto \delta_D(r_i, \widehat{i})$; it agrees with them on $\mathcal{B}_6$, so putting it in place of the first factor leaves (6) unchanged. Each of the five factors is then multilinear in the summation variables, and each variable occurs in exactly two of them— I in $\widetilde{\delta}_D$ and the outer indicator; K in the outer and the inner indicator; J in the inner indicator and $\widetilde{w}$; S_1 and O_1 in $\widetilde{Z}_{\text{cons}}$ and the inner indicator; S_2 and O_2 in $\widetilde{Z}_{\text{cons}}$ and the outer indicator; Y in $\widetilde{Z}_{\text{cons}}$ and $\widetilde{w}$ —so the summand has degree 2 in every variable, and the prover sends a degree-2 round polynomial in each of the $36 + \ell_{\text{words}}$ rounds.

The variables are bound in the order

$$O_2, \quad S_2, \quad O_1, \quad S_1, \quad J, \quad K, \quad I, \quad Y,$$

the outer shift ahead of the inner one, producing challenges $r_{o_2}, r_{o_1} \in K^3$, $r_{s_2}, r_{s_1} \in K^6$, $r_j, r_k, r_i \in K^6$, and $r_y \in K^{\ell_{\text{words}}}$. This order is what admits the efficient multi-phase prover algorithm of §4.7.3, which peels the shifts from the output end inward. The sumcheck closes with the claim

$$\gamma \stackrel{?}{=} \tilde{\delta}_D(r_i, r_i) \cdot \tilde{Z}_{\text{cons}}(r'_x, r_y, r_{s_1}, r_{o_1}, r_{s_2}, r_{o_2}) \\ \cdot \widetilde{\text{shift-ind}}(r_i, r_k, r_{s_2}, r_{o_2}) \cdot \widetilde{\text{shift-ind}}(r_k, r_j, r_{s_1}, r_{o_1}) \cdot \tilde{w}(r_j, r_y).$$

The verifier evaluates the first four factors on its own and outputs the single witness-evaluation claim $\tilde{w}(r_j, r_y)$. This bit-level claim on the $\mathbb{F}_2$ -valued witness is reduced to a linear query on the packed K -valued oracle π in the ring-switching step of §4.8.

Verifier evaluations. The factor $\tilde{\delta}_D(r_i, r_i)$ is the inner product of the 64 oblong weights $(\delta_D(r_i, i))_{i \in \mathcal{B}_6}$ (§2.2.1) with $\text{TensorExpand}_6(r_i)$. The two shift-indicator factors are evaluations of the interpolated indicator, which (3) expands into the eight succinct arithmetizations of §4.2, weighted by TensorExpand_3 of the operation challenge in the corresponding slot. For $\tilde{Z}_{\text{cons}}(r'_x, r_y, r_{s_1}, r_{o_1}, r_{s_2}, r_{o_2})$ the straightforward sparse-evaluation algorithm runs in $O(n_{\text{cons}}) + O(\text{circuit size})$ time by iterating over the constraint lists; succinct verification is left for future work, following e.g. the Spark compiler of Setty [Set20].

4.7.3 Prover Algorithm

We sketch the prover-side data structures and the main loops. The binding order of §4.7.2 cuts the sumcheck into six phases—nine rounds on (O_2, S_2) , nine on (O_1, S_1) , then six each on J , K , and I , and finally ℓ_{words} on Y —of which only the last touches a table indexed by the witness. The goal is to make the prover’s work close to linear in the witness size despite the formal $\ell_{\text{words}} + 36$ -dimensional sum.

Factorization. In (6), no single factor depends on both i and y : the oblong weights and the two shift indicators reach from i down to j , whereas $\tilde{Z}_{\text{cons}}$ and $\tilde{w}$ reach from y up to j . The two sides meet only at J . Through the first three phases, while the shift variables and J are being bound, i , k , and y all still range over the cube, so the sums over (i, k) and over y factor and the running claim reads

$$\alpha_z \stackrel{?}{=} \sum_{o_1, o_2 \in \mathcal{B}_3} \sum_{s_1, s_2 \in \mathcal{B}_6} \sum_{j \in \mathcal{B}_6} h(j, s_2, o_2, s_1, o_1) \cdot g(j, s_1, o_1, s_2, o_2),$$

where

$$h(J, S_2, O_2, S_1, O_1) := \sum_{i, k \in \mathcal{B}_6} \delta_D(r_i, i) \cdot \widetilde{\text{shift-ind}}(i, k, S_2, O_2) \cdot \widetilde{\text{shift-ind}}(k, J, S_1, O_1), \\ g(J, S_1, O_1, S_2, O_2) := \sum_{y \in \mathcal{B}_{\ell_{\text{words}}}} \tilde{Z}_{\text{cons}}(r'_x, y, S_1, O_1, S_2, O_2) \cdot \tilde{w}(J, y).$$

Both are multilinear in their 24 variables: in each summand the factors depend multilinearly on disjoint blocks of them, the shared indices i, k, y being summed over the cube. A value table of h would have 2^{24} entries, too large to form. The prover never materializes it, working instead from two successive 2^{15} tables built by the operator below.

The shift operator. For a vector $\psi \in K^{\mathcal{B}_6}$ write $T[\psi]$ for the 2^{15} -entry table

$$T[\psi](J, S, O) := \sum_{k \in \mathcal{B}_6} \psi(k) \cdot \widetilde{\text{shift-ind}}(k, J, S, O).$$

Each (s, o) slice of $T[\psi]$ is a shift of ψ . For srl the indicator $\text{shift-ind}_{\text{srl}}(k, j, s)$ is $\mathbb{K}[j = k + s]$ on the cube, so the slice holds $\psi(j - s)$ where that is valid and 0 elsewhere; similar shape patterns apply to sll, sra, ror, and their 32-bit variants. For sra the entry at $j = 63$ collects several values of k , and these are accumulated in a running prefix sum over s . The whole table costs $O(2^{15})$ field operations, a single slice $O(2^6)$.

Writing $d \in K^{\mathcal{B}_6}$ for the oblong weights $d(i) := \delta_D(r_i, \hat{i})$, precomputed once (§2.2.1), both contractions in h are applications of T . Setting $\eta := T[d]$ and naming its first index K ,

$$\eta(K, S_2, O_2) = \sum_{i \in \mathcal{B}_6} d(i) \cdot \widetilde{\text{shift-ind}}(i, K, S_2, O_2), \quad h(J, s_2, o_2, s_1, o_1) = T[\eta(\cdot, s_2, o_2)](J, s_1, o_1).$$

So h is reached by applying T to the oblong weights, taking the outer slice, and applying T again.

Active shift quadruples. A constraint system draws on only a few of the 2^{18} quadruples (s_1, o_1, s_2, o_2) . Hash and cipher circuits fix a handful of shift and rotation amounts and reuse them throughout, so the count

$$n_{\text{shift}} := |\{(s_1, o_1, s_2, o_2) : (y, \text{op}(o_1), s_1, \text{op}(o_2), s_2) \in L_x^z \text{ for some } x \text{ and some array } z \text{ of the batch}\}|$$

of *active* quadruples—taken across the Zero, BitAnd, IntMul, and BinMul arrays that the batched reduction consumes—is typically in the tens. For an inactive quadruple the entire slice $Z_{\text{cons}}[\cdot, \cdot, s_1, o_1, s_2, o_2]$ vanishes, hence so does $g(J, s_1, o_1, s_2, o_2)$. The prover accordingly carries g on the active quadruples alone.

Constraint-column index. Most shifted value indices in practice are sparse: each constraint list L_x^z has $O(1)$ entries on average. The prover builds, in a single linear pass over the constraint system, a sparse index data structure $\text{index}: \mathcal{B}_{\ell_{\text{words}}} \rightarrow (\mathcal{B}_6 \times \mathcal{B}_3 \times \mathcal{B}_6 \times \mathcal{B}_3 \rightarrow K)$ such that

$$\text{index}[y][(s_1, o_1, s_2, o_2)] = \tilde{Z}_{\text{cons}}(r'_x, y, s_1, o_1, s_2, o_2)$$

exactly for those keys at which the value is nonzero (otherwise the key is absent). The build is

precompute $\text{TensorExpand}_{\ell_{\text{cons}}}(r'_x)$; then for each $x \in \mathcal{B}_{\ell_{\text{cons}}}$ and each $(y, \text{op}(o_1), s_1, \text{op}(o_2), s_2) \in L_x^z$, add $\text{TensorExpand}(r'_x)[x]$ to $\text{index}[y][(s_1, o_1, s_2, o_2)]$.

Total work and space are $O(\text{circuit size}) + O(n_{\text{words}})$ field elements.

Computing g . Using index , the prover forms the $2^6 \cdot n_{\text{shift}}$ -entry table $g[j][q] = \sum_y \tilde{Z}_{\text{cons}}(r'_x, y, q) \cdot \tilde{w}(j, y)$, indexed by the bit position and by an active quadruple q , by iterating over y and, for each populated key q in $\text{index}[y]$, adding $\text{index}[y][q]$ into $g[j][q]$ for each bit j where $w[y]_j = 1$. Total work is proportional to the (sparse) circuit size. This loop is the reduction's one pass over the whole circuit, and the accumulator it scatters into stays cache-resident throughout.

Phase 1: binding the outer shift. The prover materializes $\eta = T[d]$ and folds it over (O_2, S_2) as the nine rounds proceed, at 2^{15} multiplications in total. In round t it needs, for each active quadruple whose outer pair is still live, the 2^6 -vector

$$h_t(\cdot) = T[\eta_t(\cdot, u)](\cdot, s_1, o_1),$$

where u is the partially bound outer index and η_t the folded table: one slice of T applied to one stride of η_t , hence $O(2^6)$ apiece. Pairing these against the sparse g costs another $O(2^6)$ per active combination, so a round costs $O(2^6 \cdot n_{\text{shift}})$.

The inner pair (s_1, o_1) is still a cube index throughout this phase, and that is what keeps the slice of T a shift. Under the opposite order the outer indicator would be folded to a dense $2^6 \times 2^6$ matrix by the time the inner pair was reached, and each active combination would cost a full matrix–vector product in place of a shift.

Phase 2: binding the inner shift. Phase 1 leaves η folded to the single 2^6 -vector $\psi := \eta(\cdot, r_{s_2}, r_{o_2})$. The prover forms $T[\psi]$, whose slices are the concatenated shifts of ψ , and runs the nine rounds binding (O_1, S_1) against it. This is an ordinary sparse–dense sumcheck. Let L_t denote the image of the active inner pairs under the map forgetting the t coordinates bound so far; since binding a variable merges two indices into one, $|L_t| \leq \min(n_{\text{shift}}, 2^{9-t})$. The folded g is supported on L_t , so the prover folds and forms round polynomials over those entries alone, at $O(2^6 \cdot |L_t|)$ field operations in round t and $O(2^6 \cdot n_{\text{shift}} \cdot \log_2(2^9/n_{\text{shift}}))$ across the phase. The table $T[\psi]$ is folded in full: its folded value at an active pair is a weighted sum over that index's whole subtree, inactive pairs included, so the dense table is what supplies it. This costs at most 2^{15} multiplications, a fixed charge independent of the circuit.

Phase 3: binding J . Both tables now have length 2^6 in J alone, and the six rounds binding J are a dense sumcheck on their product, at $O(2^6)$ field operations. The phase closes with

$$\beta \stackrel{?}{=} h(r_j, r_{s_2}, r_{o_2}, r_{s_1}, r_{o_1}) \cdot G, \quad G := g(r_j, r_{s_1}, r_{o_1}, r_{s_2}, r_{o_2}),$$

both factors being the terminal entries of the two folds. In particular G carries the summation over y of $\tilde{Z}_{\text{cons}} \cdot \tilde{w}$, which is what built the table of g in the first place.

Phase 4: binding K . Unrolling h at the bound shift point exposes the six rounds binding K as a sumcheck on

$$\beta \stackrel{?}{=} \sum_{k \in \mathcal{B}_6} \psi(k) \cdot \widetilde{\text{shift-ind}}(k, r_j, r_{s_1}, r_{o_1}) \cdot G.$$

The first factor is the vector ψ that phase 1 left behind: the terminal fold of η is exactly the partial evaluation $\sum_{i \in \mathcal{B}_6} d(i) \cdot \widetilde{\text{shift-ind}}(i, K, r_{s_2}, r_{o_2})$, the multilinear extension commuting with the finite sum over i . The second the prover tabulates as $\chi(k) := \widetilde{\text{shift-ind}}(k, r_j, r_{s_1}, r_{o_1})$, from 2^6 evaluations of the succinct arithmetizations of §4.2. Keeping ψ and χ apart is what makes the round polynomials degree 2; the phase costs $O(2^6)$ field operations.

Phase 5: binding I . The same step once more, one link further up the chain. The prover holds d from the outset and tabulates $v(i) := \widetilde{\text{shift-ind}}(i, r_k, r_{s_2}, r_{o_2})$ from another 2^6 succinct evaluations, then runs six rounds on

$$\sum_{i \in \mathcal{B}_6} d(i) \cdot v(i) \cdot \Lambda, \quad \Lambda := \widetilde{\text{shift-ind}}(r_k, r_j, r_{s_1}, r_{o_1}) \cdot G,$$

where Λ collects the scalars already fixed, the first being the terminal entry of the χ fold. Neither of the last two phases touches a table indexed by y .

Phase 6: binding Y . For the ℓ_{words} rounds binding Y the prover needs the value tables of the multilinear $\tilde{w}(r_j, Y)$ (a standard partial specialization of $\tilde{w}$, computed via $\text{TensorExpand}_6(r_j)$) and $\tilde{Z}_{\text{cons}}(r'_x, Y, r_{s_1}, r_{o_1}, r_{s_2}, r_{o_2})$. The second again uses `index`: precompute the four tensor expansions $\text{TensorExpand}_6(r_{s_1})$, $\text{TensorExpand}_3(r_{o_1})$, $\text{TensorExpand}_6(r_{s_2})$, $\text{TensorExpand}_3(r_{o_2})$, then accumulate, for each y and each populated $q = (s_1, o_1, s_2, o_2) \in \text{index}[y]$, the product of $\text{index}[y][q]$ with $\tilde{\text{eq}}_6(r_{s_1}, s_1) \cdot \tilde{\text{eq}}_3(r_{o_1}, o_1) \cdot \tilde{\text{eq}}_6(r_{s_2}, s_2) \cdot \tilde{\text{eq}}_3(r_{o_2}, o_2)$ into a length- n_{words} vector. The scalars fixed by the earlier phases ride along as a constant factor.

4.8 Ring-Switching to the Packed Witness

The shift reduction (§4.7) closes by establishing a single evaluation claim

$$\tilde{w}(r_j, r_y) = t, \quad r_j \in K^6, \quad r_y \in K^{\ell_{\text{words}}}, \quad t \in K,$$

on the multilinear extension of the $\mathbb{F}_2$ -valued witness w . Our LIOP framework (§2.3) requires the prover oracle to be K -valued and the final witness-side check to be a single linear query against that oracle. Ring-switching, due to Diamond and Posen [DP24], bridges the two by packing the $\mathbb{F}_2$ -valued witness into a K -valued oracle and reducing the bit-level evaluation claim to a linear query on the packed oracle. We summarize the construction at a level sufficient for the LIOP specification and defer protocol details and proofs to [DP24].

The packed witness. We view the witness as a length- $64 \cdot n_{\text{words}}$ bit-string indexed by $\mathcal{B}_{6+\ell_{\text{words}}}$. Set the packing parameter

$$\kappa := \log_2 \lambda,$$

the $\mathbb{F}_2$ -dimension of K . Our padding assumption $\lambda \mid 64 \cdot n_{\text{words}}$ (§2) ensures $6 + \ell_{\text{words}} \geq \kappa$, so we can group the $6 + \ell_{\text{words}}$ Boolean dimensions of w into the first κ (“packed” bit dimensions, indexed by $i \in \mathcal{B}_\kappa$) and the remaining

$$\ell_{\text{pack}} := 6 + \ell_{\text{words}} - \kappa$$

(“unpacked” dimensions, indexed by $y \in \mathcal{B}_{\ell_{\text{pack}}}$). Each “cell” indexed by y contains λ witness bits; with $\lambda = 128$ this packs two consecutive 64-bit witness words into a single K -element.

Fix once and for all an $\mathbb{F}_2$ -basis $(\beta_i)_{i \in \mathcal{B}_\kappa}$ of K , indexed by $\mathcal{B}_\kappa$ for notational convenience. The *packing map* pack_λ takes the witness $w: \mathcal{B}_{6+\ell_{\text{words}}} \rightarrow \mathbb{F}_2$ to the function

$$\text{pack}_\lambda(w): \mathcal{B}_{\ell_{\text{pack}}} \rightarrow K, \quad \text{pack}_\lambda(w)(y) := \sum_{i \in \mathcal{B}_\kappa} w(i, y) \cdot \beta_i,$$

i.e., each entry of $\text{pack}_\lambda(w)$ is the inner product of λ witness bits with the chosen basis. The LIOP commits to the oracle $\pi := \text{pack}_\lambda(w)$, of length $2^{\ell_{\text{pack}}}$ over K .

Ring-switching reduction. The reduction follows Diamond and Posen [DP24, Construction 3.1]. We split the shift reduction’s evaluation point $(r_j, r_y) \in K^{6+\ell_{\text{words}}}$ into a packed part $r_{\text{pack}} \in K^\kappa$ and an unpacked part $r_{\text{rest}} \in K^{\ell_{\text{pack}}}$, reinterpreting (r_j, r_y) under the packing layout above.

1. The prover sends the vector of partial evaluations

$$\hat{s} := (\hat{s}_i)_{i \in \mathcal{B}_\kappa}, \quad \hat{s}_i := \tilde{w}(i, r_{\text{rest}}),$$

one K -element per index $i \in \mathcal{B}_\kappa$.

2. The verifier checks consistency with the input claim t by verifying the inner product

$$t \stackrel{?}{=} \sum_{i \in \mathcal{B}_\kappa} \tilde{\text{eq}}_\kappa(i, r_{\text{pack}}) \cdot \hat{s}_i.$$

3. The verifier samples $r' \leftarrow K^{\ell_{\text{pack}}}$.
4. The verifier specializes the *ring-switching indicator multilinear* of [DP24, Construction 3.1] at (r_{rest}, r') to obtain a multilinear *ring-switch-ind* $_{r_{\text{rest}}, r'}(Y) \in K[Y_0, \dots, Y_{\ell_{\text{pack}}-1}]^{\leq 1}$, and derives a target value

$$s' \in K$$

as a fixed K -linear combination of the entries of $\hat{s}$ whose coefficients depend on r_{rest} , r' , and the basis (β_i) . The exact functional form of both *ring-switch-ind* $_{r_{\text{rest}}, r'}$ and s' is given by [DP24, Construction 3.1]; the verifier evaluates both in $O(\ell_{\text{pack}} \cdot \lambda)$ operations.

The output of ring-switching is the pair $(\text{ring-switch-ind}_{r_{\text{rest}}, r'}, s')$, which defines the linear oracle query on π :

$$\sum_{y \in \mathcal{B}_{\ell_{\text{pack}}}} \text{ring-switch-ind}_{r_{\text{rest}}, r'}(y) \cdot \pi(y) \stackrel{?}{=} s'.$$

Soundness of this reduction is established in [DP24, §3]; the error is $O(\ell_{\text{pack}})/|K|$ over r' .

For notational uniformity with the rest of the document, we abbreviate this output query as $(t_{\text{rs}}, s_{\text{rs}}) := (\text{ring-switch-ind}_{r_{\text{rest}}, r'}, s')$. It is one of the two linear queries on π produced by the LIOP; it is random-combined with the public-input-check query of §4.9 to yield the single final query of §2.3.

4.9 Public Input Check

The public input check verifies that the first $n_{\text{public}} := n_{\text{const}} + n_{\text{inout}}$ words of w equal the verifier-held public data: $w[y] = C(y)$ for $y \in \{0, \dots, n_{\text{const}} - 1\}$ and $w[n_{\text{const}} + y] = u(y)$ for $y \in \{0, \dots, n_{\text{inout}} - 1\}$. We concatenate the constants and the statement into a single vector

$$\text{public}: \mathcal{B}_6 \times \mathcal{B}_{\ell_{\text{public}}} \rightarrow \mathbb{F}_2, \quad \text{public} := C \parallel u,$$

under the identification of $\mathcal{B}_{\ell_{\text{public}}}$ with $\{0, \dots, n_{\text{public}} - 1\}$. The padding requirement $\lambda \mid 64 \cdot n_{\text{public}}$ (§2) lets us apply the packing map pack_λ (§4.8) to public in the same way as to w , producing

$$\text{pack}_\lambda(\text{public}): \mathcal{B}_{\ell_{\text{public-pack}}} \rightarrow K, \quad \ell_{\text{public-pack}} := \ell_{\text{public}} - (\log_2 \lambda - 6).$$

The check the verifier wants to enforce is that π agrees with $\text{pack}_\lambda(\text{public})$ on the first $2^{\ell_{\text{public-pack}}}$ entries: for all $y \in \mathcal{B}_{\ell_{\text{public-pack}}}$,

$$\pi(y, 0, \dots, 0) \stackrel{?}{=} \text{pack}_\lambda(\text{public})(y),$$

where the index $(y, 0, \dots, 0) \in \mathcal{B}_{\ell_{\text{pack}}}$ pads y with $\ell_{\text{pack}} - \ell_{\text{public-pack}}$ zeros on the high coordinates. Equivalently, the polynomial

$$\widetilde{\pi}(Y_0, \dots, Y_{\ell_{\text{public-pack}}-1}, 0, \dots, 0) - \widetilde{\text{pack}_\lambda(\text{public})}(Y_0, \dots, Y_{\ell_{\text{public-pack}}-1})$$

vanishes on $\mathcal{B}_{\ell_{\text{public-pack}}}$.

Reduction to a linear query. The verifier samples $r_p \leftarrow K^{\ell_{\text{public-pack}}}$ and checks

$$\widetilde{\pi}(r_p, 0, \dots, 0) \stackrel{?}{=} \widetilde{\text{pack}_\lambda(\text{public})}(r_p).$$

The right-hand side is computable by the verifier on its own in $O(n_{\text{public}}/\lambda)$ time via the standard TensorExpand-based multilinear-evaluation algorithm. The left-hand side is an evaluation of the oracle's multilinear extension, which is exactly the multilinear-evaluation linear query on π at the point $(r_p, 0, \dots, 0) \in K^{\ell_{\text{pack}}}$:

$$t_{\text{pub}}(Y) := \widetilde{\text{eq}}_{\ell_{\text{pack}}}((r_p, 0, \dots, 0), Y) = \widetilde{\text{eq}}_{\ell_{\text{public-pack}}}(r_p, Y_{<\ell_{\text{public-pack}}}) \cdot \prod_{i=\ell_{\text{public-pack}}}^{\ell_{\text{pack}}-1} (1 - Y_i),$$

with target $s_{\text{pub}} := \widetilde{\text{pack}_\lambda(\text{public})}(r_p)$. The check passes iff $\sum_{y \in \mathcal{B}_{\ell_{\text{pack}}}} t_{\text{pub}}(y) \cdot \pi(y) = s_{\text{pub}}$.

By the standard zerocheck soundness, the reduction has error $\ell_{\text{public-pack}}/|K|$ over the choice of r_p .

Batching with the ring-switching query. The LIOP could output the two queries $(t_{\text{rs}}, s_{\text{rs}})$ from ring-switching and $(t_{\text{pub}}, s_{\text{pub}})$ from the public input check independently. We instead random-combine them into a single linear query on π . The verifier samples a batching coefficient $\xi \leftarrow K$ and outputs

$$t(Y) := t_{\text{rs}}(Y) + \xi \cdot t_{\text{pub}}(Y), \quad s := s_{\text{rs}} + \xi \cdot s_{\text{pub}},$$

so that the LIOP's final linear query is $\sum_{y \in \mathcal{B}_{\ell_{\text{pack}}}} t(y) \cdot \pi(y) = s$. Both t_{rs} and t_{pub} are succinct (evaluable in $O(\ell_{\text{pack}})$ field operations), so t is too. The random batching contributes an additional $1/|K|$ to the soundness error.

5 Basic SNARK Compiler

This section presents a generic compiler from any LIOP (§2.3) to a SNARK. The compiler is a composition of three standard transformations,

$$\text{LIOP} \xrightarrow{\text{BaseFold}} \text{IOP} \xrightarrow{\text{BCS}} \text{IP} \xrightarrow{\text{Fiat-Shamir}} \text{SNARK},$$

each of which is independent of the LIOP and applies uniformly to LIOPs with any number of oracles and any number of linear queries. Applying the compiler to the Binius64 LIOP of §4 yields the Binius64 SNARK (§5.5).

5.1 Compiling LIOPs to SNARKs

Stage 1: LIOP $\rightarrow$ IOP via BaseFold. An LIOP's verifier closes by issuing linear queries, each specified by a multilinear polynomial t_j and target s_j against an oracle π_{i_j} (§2.3). To turn this into a standard IOP, in which the verifier issues point queries, we encode each LIOP oracle $\pi_i \in K^{2^{n_i}}$ as a codeword of a Reed-Solomon code over K . Each linear query then becomes an additional linear constraint on the codeword: the codeword is required not only to be a valid Reed-Solomon codeword but also to satisfy $\sum_{w \in \mathcal{B}_{n_i}} t_j(w) \cdot \pi_{i_j}(w) = s_j$. The pair (Reed-Solomon code, linear constraint) is

a *constrained Reed–Solomon code* in the sense of Arnon, Chiesa, Fenzi, and Ygeev [ACFY25, §4.1]. BaseFold (§5.2) is an *interactive oracle proof of proximity* (IOPP) for constrained Reed–Solomon codes: given oracle access to a candidate codeword, BaseFold accepts iff the codeword is close to a true codeword of the constrained code, using a polylogarithmic number of point queries. Composing one BaseFold instance per LIOP linear query yields a public-coin IOP whose oracles are the original π_i together with the folded codewords introduced by BaseFold.

Stage 2: IOP $\rightarrow$ IP via BCS. The BCS transform of Ben-Sasson, Chiesa, and Spooner [BCS16] replaces each oracle message of an IOP with the Merkle-tree root of its content; subsequent point queries are answered by the prover with the corresponding leaf and Merkle authentication path. In the random oracle model, this transformation preserves soundness up to a tight multiplicative loss in the adversary’s query budget, and the result is a public-coin interactive proof (no oracles) of the same statement.

Stage 3: IP $\rightarrow$ SNARK via Fiat–Shamir. The Fiat–Shamir transform makes the public-coin IP non-interactive by deriving each verifier challenge as a hash of the running transcript. The resulting non-interactive argument is a SNARK in the random oracle model.

Succinctness. The compiled SNARK is succinct (proof size and verification time polylogarithmic in the witness size) provided the source LIOP is succinct in the sense of §2.3: when each linear query polynomial t_j is evaluable in $\text{poly}(n_{i_j})$ time, BaseFold’s verifier uses polylogarithmically many point queries, the Merkle openings are polylog-sized, and the LIOP’s own non-oracle messages are short.

5.2 BaseFold

BaseFold is an IOPP for constrained Reed–Solomon codes (Arnon, Chiesa, Fenzi, and Ygeev [ACFY25, §4.1]). The original construction is due to Zeilberger, Chen, and Fisch [ZCF24]; we use the binary-field adaptation of Diamond and Posen [DP24], with encoding via the additive number-theoretic transform (NTT) of Lin, Chung, and Han [LCH14]. Tightened soundness analyses are given by Haböck [Hab24].

Setup. Fix a power-of-two rate parameter $\mathcal{R} \geq 2$ and a nested chain of additive $\mathbb{F}_2$ -linear subspaces $\{0\} = E_0 \subset E_1 \subset \dots \subset E_n \subset K$, where each E_k has dimension $k + \log_2 \mathcal{R}$.

The encoding map $\text{Enc}_k: K^{2^k} \rightarrow K^{|E_k|}$ takes an oracle $\pi: \mathcal{B}_k \rightarrow K$ to its evaluation on E_k of the polynomial

$$f_\pi(X) := \sum_{i=0}^{2^k-1} \pi_{\text{rev}(i)} \cdot X_i(X),$$

where $X_0, \dots, X_{2^k-1}$ is the novel polynomial basis underlying the Lin–Chung–Han additive NTT [LCH14] (the basis of monic polynomials with $\deg X_i = i$ that diagonalizes the additive-NTT butterfly tree), and $\text{rev}(i)$ is the k -bit reversal of $i \in \{0, \dots, 2^k - 1\}$. The bit-reversed indexing is a deliberate choice with two consequences. First, it aligns hypercube zero-padding on the message side with codeword duplication on the encoding side, which we use in §5.3 to handle oracles of unequal sizes. Second, it causes BaseFold’s sumcheck to specialize high-indexed variables of π ’s multilinear extension first, which yields a more SIMD-friendly prover algorithm than specializing low-indexed variables first.

The constrained Reed–Solomon code at level n is parameterized by a multilinear *multiplicand* $t: \mathcal{B}_n \rightarrow K$ and a target $s \in K$; it is the set of codewords $c = \text{Enc}_n(\pi)$ for $\pi: \mathcal{B}_n \rightarrow K$ satisfying

$$\pi \cdot t = s \quad \text{i.e.,} \quad \sum_{w \in \mathcal{B}_n} \pi(w) \cdot t(w) = s.$$

Protocol. On input an oracle $c_n \in K^{|E_n|}$ and a constraint (t, s) , BaseFold proceeds in two phases:

1. *Folding phase.* For each round $k = n - 1, \dots, 0$, the parties interleave one round of a sumcheck on the constraint $\sum_{w \in \mathcal{B}_n} \pi(w) \cdot t(w) = s$ with a single fold of the codeword. The prover sends a sumcheck round polynomial; the verifier samples $\rho_k \leftarrow K$ and updates the running sumcheck claim; both parties fold the codeword from $c_{k+1} \in K^{|E_{k+1}|}$ to $c_k \in K^{|E_k|}$ using ρ_k . The prover provides c_k as a new oracle each round.

2. *Query phase.* After n rounds, the codeword $c_0 \in K^{|E_0|}$ is short (of length $\mathcal{R}$) and the sumcheck claim reduces to a single multilinear-extension evaluation that the verifier can check against the prover's claimed reduced codeword. The verifier additionally samples μ random query positions in E_n , and for each position requests the corresponding entries of $c_n, c_{n-1}, \dots, c_0$ from the prover oracles. The verifier checks two things at each query position: that the entries are consistent with each successive fold given the challenges ρ_k , and that the chain bottoms out in the sent c_0 .

The verifier accepts iff all sumcheck round-polynomial checks pass, the final reduced claim matches, and all μ fold-consistency checks pass.

Parameters. For target soundness $2^{-\lambda}$, the query count μ is chosen logarithmic in λ and in the inverse proximity gap of the underlying code; concrete settings are given in [Hab24]. The prover work is $O(2^n \cdot \mathcal{R})$ in K , dominated by the initial encoding; the verifier work is $\text{poly}(n) + O(\mu \cdot n)$.

5.3 Lifted BaseFold

The Batched BaseFold protocol of §5.4 requires the oracles to share a common variable count $\mathbf{n}$. When the LIOP commits oracles of unequal sizes, we lift each smaller oracle's codeword to the level- $\mathbf{n}$ encoding via the *Lifted BaseFold* (a.k.a. Lifted FRI, after the Fast Reed–Solomon IOPP) construction [HMH].

We use three operators on K -vectors. For $v \in K^N$ with $N = 2^\mu$, and $\eta \geq 0$, define

$$\begin{aligned} \text{ZeroPadLSB}_\eta(v) &\in K^{N \cdot 2^\eta}, & \text{ZeroPadLSB}_\eta(v)_i &= \begin{cases} v_{i/2^\eta} & \text{if } 2^\eta \mid i, \\ 0 & \text{otherwise,} \end{cases} \\ \text{ZeroPadMSB}_\eta(v) &\in K^{N \cdot 2^\eta}, & \text{ZeroPadMSB}_\eta(v)_i &= \begin{cases} v_i & \text{if } i < N, \\ 0 & \text{otherwise,} \end{cases} \\ \text{Duplicate}_\eta(v) &\in K^{N \cdot 2^\eta}, & \text{Duplicate}_\eta(v)_i &= v_{\lfloor i/2^\eta \rfloor}. \end{aligned}$$

The two zero-padding operators differ in which index bits they zero: ZeroPadLSB_η keeps entries only at positions whose low η bits vanish, while ZeroPadMSB_η keeps entries only at the bottom N indices (positions whose high η bits vanish).

The Lin–Chung–Han additive NTT exhibits a clean zero-pad/duplicate correspondence on the LSB side:

Theorem 5.1 (LCH NTT zero-pad correspondence). *For all $\mu, \eta \geq 0$ and any coefficient vector $a \in K^{2^\mu}$ with respect to the novel polynomial basis of §5.2,*

$$\text{NTT}_{\mu+\eta}(\text{ZeroPadLSB}_\eta(a)) = \text{Duplicate}_\eta(\text{NTT}_\mu(a)).$$

Now fix an oracle $\pi: \mathcal{B}_n \rightarrow K$ with $n \leq \mathbf{n}$, identified with its value vector in K^{2^n} when applying the operators. Its lift $\text{ZeroPadMSB}_{\mathbf{n}-n}(\pi): \mathcal{B}_{\mathbf{n}} \rightarrow K$ has multilinear extension

$$\widetilde{\text{ZeroPadMSB}}_{\mathbf{n}-n}(\pi)(X_0, \dots, X_{\mathbf{n}-1}) = \tilde{\pi}(X_0, \dots, X_{n-1}) \cdot \tilde{\mathbf{eq}}_{\mathbf{n}-n}(0^{\mathbf{n}-n}, X_n, \dots, X_{\mathbf{n}-1}), \quad (7)$$

where $\tilde{\mathbf{eq}}_{\mathbf{n}-n}(0^{\mathbf{n}-n}, Y_0, \dots, Y_{\mathbf{n}-n-1}) = \prod_j (1 - Y_j)$ vanishes off the all-zeros high block.

The bit-reversed indexing of §5.2 turns end-padding on the hypercube into LSB zero-padding on the coefficient side: the coefficient vector of $\text{ZeroPadMSB}_{\mathbf{n}-n}(\pi)$ in the novel basis is exactly $\text{ZeroPadLSB}_{\mathbf{n}-n}$ applied to the coefficient vector of π . Combining with Theorem 5.1, we obtain the central identity of Lifted BaseFold:

$$\text{Enc}_{\mathbf{n}}(\text{ZeroPadMSB}_{\mathbf{n}-n}(\pi)) = \text{Duplicate}_{\mathbf{n}-n}(\text{Enc}_n(\pi)). \quad (8)$$

The lifted codeword is therefore the original codeword with each entry duplicated $2^{\mathbf{n}-n}$ times. Given Merkle access to the original codeword $c = \text{Enc}_n(\pi)$, the verifier emulates oracle access to the lifted codeword by translating a query at position $e \in E_{\mathbf{n}}$ into a query at $\lfloor e \cdot |E_n| / |E_{\mathbf{n}}| \rfloor \in E_n$.

Using this lift, every LIOP oracle is virtually placed at a common level $\mathbf{n} = \max_i n_i$ (where n_i is the variable count of the i -th oracle) before the parties run Batched BaseFold (§5.4). Soundness of the lift requires a refined proximity-gap analysis (the lifted codeword's proximity to the level- $\mathbf{n}$ code is constrained by the original's proximity to the level- n code, with a scaling factor of $2^{\mathbf{n}-n}$); see [HMH] for details.

5.4 Batched BaseFold

The compiler of §5.1 runs one BaseFold instance per LIOP linear query. When the LIOP commits multiple oracles with exactly one opening claim per oracle, the openings can be discharged with a single BaseFold instance via two preliminary reductions. We present this batched variant, which is the setting needed for the IronSpartan LIOP of §6. (The Binius64 LIOP itself uses only one oracle and one opening, so it does not need this machinery.)

Suppose the LIOP holds opening claims

$$\langle \pi_i, t_i \rangle = s_i, \quad i = 0, \dots, k-1,$$

where $\pi_i: \mathcal{B}_{n_i} \rightarrow K$ is the i -th oracle, $t_i: \mathcal{B}_{n_i} \rightarrow K$ is the *transparent* operand—a multilinear the verifier can evaluate on its own, with no help from the prover—and $s_i \in K$ is the target.

Step 1: Batched sumcheck. The verifier samples a single batching challenge $\alpha \leftarrow K$. Let $\pi_i^{(\uparrow)}, t_i^{(\uparrow)}: \mathcal{B}_{\mathbf{n}} \rightarrow K$ denote the ZeroPadMSB lifts (§5.3) of π_i, t_i to the common variable count $\mathbf{n} := \max_i n_i$; the lifts vanish off the original domains, so $\sum_{x \in \mathcal{B}_{\mathbf{n}}} \tilde{\pi}_i^{(\uparrow)}(x) \tilde{t}_i^{(\uparrow)}(x) = s_i$. The parties run a sumcheck on

$$\sum_{i=0}^{k-1} \alpha^i s_i \stackrel{?}{=} \sum_{x \in \mathcal{B}_{\mathbf{n}}} \sum_{i=0}^{k-1} \alpha^i \cdot \tilde{\pi}_i^{(\uparrow)}(x) \cdot \tilde{t}_i^{(\uparrow)}(x).$$

After $\mathbf{n}$ rounds with sumcheck challenges $r \in K^{\mathbf{n}}$, the batched claim reduces to $\sum_{i=0}^{k-1} \alpha^i \cdot \tilde{\pi}_i^{(\uparrow)}(r) \cdot \tilde{t}_i^{(\uparrow)}(r)$. The verifier evaluates each $\tilde{t}_i^{(\uparrow)}(r)$ on its own (via the factorization (7)). The remaining unknowns are the prover-claimed evaluations

$$\alpha_i := \tilde{\pi}_i^{(\uparrow)}(r), \quad i = 0, \dots, k-1.$$

Step 2: Random reduction to a piecewise-concatenated oracle. To verify the k evaluation claims α_i via a single BaseFold instance, we treat the $\pi_i^{(\uparrow)}$ as restrictions of a single *piecewise concatenated* multilinear

$$\pi(X, Y) := \sum_{i=0}^{k-1} \tilde{\pi}_i^{(\uparrow)}(X) \cdot \tilde{\mathbf{e}}_{\log_2 k}(i, Y),$$

in $X \in K^{\mathbf{n}}$ and $\log_2 k$ fresh oracle-index variables $Y \in K^{\log_2 k}$, so that $\pi(X, i) = \tilde{\pi}_i^{(\uparrow)}(X)$ for each $i \in \mathcal{B}_{\log_2 k}$.

The verifier samples $r' \leftarrow K^{\log_2 k}$ and computes the target

$$s' := \sum_{i=0}^{k-1} \alpha_i \cdot \tilde{\mathbf{e}}_{\log_2 k}(i, r') = \pi(r, r').$$

The parties then run a single BaseFold instance on π at the point (r, r') with target s' . The k separately-committed codewords of $\pi_0, \dots, \pi_{k-1}$ from the LIOP together form the virtual oracle for π : a BaseFold query at position $(e, y) \in E_{\mathbf{n}} \times \mathcal{B}_{\log_2 k}$ is answered by querying the codeword of π_y at position e , with the Merkle authentication path drawn from the original π_y commitment.

5.5 The Binius64 SNARK

The Binius64 SNARK is the result of applying the compiler of §5.1 to the Binius64 LIOP of §4. The LIOP has a single oracle $\pi = \text{pack}_\lambda(w): \mathcal{B}_{\ell_{\text{pack}}} \rightarrow K$ and ends with a single succinct linear query (t, s) , namely the random combination of the ring-switching and public-input-check queries (§4.9). The compiler instantiates this LIOP query with a single BaseFold IOPP instance on the constrained Reed–Solomon code (RS code over K with the constraint $\sum_y t(y) \cdot \pi(y) = s$), and applies BCS and Fiat–Shamir on top.

Concretely, the non-interactive proof consists of:

- the Merkle root of $\text{Enc}_{\ell_{\text{pack}}}(\pi)$, sent at the start (commitment to the oracle);

- the LIOP’s non-oracle messages (the univariate polynomial $g(\widehat{T})$ of the BitAnd reduction, GKR-step claimed evaluations and Frobenius-step values of the IntMul reduction, the per-bit column values of the BinMul reduction, the round polynomials of the various sumchecks, the vector $\hat{s}$ of the ring-switching step), with verifier challenges Fiat–Shamir-derived from the running transcript;
- the BaseFold transcript verifying the LIOP’s final linear query: Merkle roots of the folded code-words, the sumcheck round polynomials of BaseFold’s folding phase, and the Merkle openings answering the μ query positions.

The verifier reconstructs each LIOP and BaseFold challenge by hashing the transcript prefix in the canonical Fiat–Shamir order, runs each LIOP verifier-side check, and runs the BaseFold verifier on the IOPP transcript.

Soundness. Combining the soundness errors of the LIOP, BaseFold, and BCS gives an overall soundness error of $O(\ell_{\text{words}} + \ell_{\text{cons}}) / |K| + 2^{-\lambda}$ in the random oracle model, where the first term collects the various sumcheck and Schwartz–Zippel errors of §4.3–§4.9 and the second term is the BaseFold soundness floor. Knowledge soundness follows from the same compilation chain with knowledge extraction at each stage.

6 The IronSpartan LIOP

The Binius64 LIOP of §4 is specialized to the BitAnd, IntMul, and BinMul constraint system. We now specify a second LIOP, *IronSpartan*, for the simple language of rank-one constraint systems (R1CS). IronSpartan adapts the Spartan PIOP of Setty [Set20] to the LIOP framework of §2.3, and presents it as a *commit-and-prove* argument: the assignment is split into a public, a precommit, and a witness segment, and the prover commits the precommit segment as an oracle *before* the public instance is fixed. Where Spartan answers its single witness-evaluation query with a polynomial commitment scheme, IronSpartan phrases the analogous claim as linear queries on the committed segment oracles, to be discharged by the batched compiler of §5. The construction is moreover honest-verifier zero-knowledge (HVZK), so composed with the Batched ZK BaseFold compiler of §7.2 it yields a zero-knowledge commit-and-prove SNARK for R1CS.

6.1 R1CS and the Spartan PIOP

IronSpartan proves satisfaction of a rank-one constraint system in *commit-and-prove* form, over an assignment partitioned into three *data segments*

$$z = (z_{\text{pub}}, z_{\text{pre}}, z_{\text{wit}}),$$

indexed by $\sigma \in \{\text{pub}, \text{pre}, \text{wit}\}$ – the *public*, *precommit*, and *witness* segments, of sizes $n_{\text{pub}}, n_{\text{pre}}, n_{\text{wit}}$. The public segment z_{pub} carries the constant 1 together with the public inputs and is held by the verifier; the precommit and witness segments are private and are committed as oracles. The defining feature of the commit-and-prove setting is that the prover sends the precommit oracle z_{pre} *before* the public instance z_{pub} is fixed: the precommitted data cannot depend on the statement, and one precommitment may be reused across many statements.

The constraint system is specified by *nine wiring matrices* $W_{\text{op}, \sigma} \in K^{n_M \times n_\sigma}$, one per operator $\text{op} \in \{A, B, C\}$ and segment σ . Concatenating the segments columnwise recovers the usual R1CS matrices, $A = [W_{A, \text{pub}} \mid W_{A, \text{pre}} \mid W_{A, \text{wit}}]$ and likewise for B and C , so that each matrix–vector product splits across segments as

$$\text{op} \cdot z = \sum_{\sigma} W_{\text{op}, \sigma} z_{\sigma}, \quad \text{op} \in \{A, B, C\}.$$

The assignment *satisfies* the instance if

$$(Az) \circ (Bz) = Cz,$$

where $\circ$ denotes the entry-wise product. We pad n_M and each segment size n_σ to powers of two, $n_M = 2^{\ell_M}$ and $n_\sigma = 2^{\ell_\sigma}$, indexing constraints (matrix rows) by $\mathcal{B}_{\ell_M}$ and the entries of segment σ (matrix columns) by $\mathcal{B}_{\ell_\sigma}$.

Multilinear encoding. Each wiring matrix $W_{\text{op},\sigma} : \mathcal{B}_{\ell_M} \times \mathcal{B}_{\ell_\sigma} \rightarrow K$ and each segment $z_\sigma : \mathcal{B}_{\ell_\sigma} \rightarrow K$ has a multilinear extension, written $\widetilde{W}_{\text{op},\sigma}(X, Y)$ and $\widetilde{z}_\sigma(Y)$. For $M \in \{A, B, C\}$, write

$$\widetilde{(Mz)}(X) := \sum_{\sigma} \sum_{y \in \mathcal{B}_{\ell_\sigma}} \widetilde{W}_{M,\sigma}(X, y) \widetilde{z}_\sigma(y)$$

for the multilinear extension of the product vector Mz . The relation holds iff $\widetilde{(Az)}(x) \cdot \widetilde{(Bz)}(x) = \widetilde{(Cz)}(x)$ for every constraint $x \in \mathcal{B}_{\ell_M}$.

The Spartan PIOP. After committing the assignment, Spartan [Set20] verifies this relation in two sumcheck stages.

1. **Constraint check.** The verifier samples $\tau \leftarrow K^{\ell_M}$, and the parties run an MLE-check (§2) on

$$\sum_{x \in \mathcal{B}_{\ell_M}} \widetilde{\text{eq}}_{\ell_M}(\tau, x) \cdot [\widetilde{(Az)}(x) \widetilde{(Bz)}(x) - \widetilde{(Cz)}(x)] = 0.$$

This reduces to a random point $r_x \in K^{\ell_M}$ at which the prover sends the three claimed products $a = \widetilde{(Az)}(r_x)$, $b = \widetilde{(Bz)}(r_x)$, $c = \widetilde{(Cz)}(r_x)$, for which the verifier checks $a \cdot b = c$.

2. **Wiring check.** The verifier samples $\gamma \leftarrow K$ and a second MLE-check batches the three products into a single evaluation query on the committed assignment.

Spartan answers the resulting evaluation query with a polynomial commitment scheme; IronSpartan replaces this stage, as we now describe.

6.2 IronSpartan Overview

IronSpartan departs from Spartan in how the wiring check is discharged, exploiting the segment structure to keep the public data out of the committed oracles. For the batching challenge γ , define for each segment the *batched wiring operand*

$$t_\sigma(Y) := \widetilde{W}_{A,\sigma}(r_x, Y) + \gamma \widetilde{W}_{B,\sigma}(r_x, Y) + \gamma^2 \widetilde{W}_{C,\sigma}(r_x, Y), \quad \sigma \in \{\text{pub}, \text{pre}, \text{wit}\},$$

a multilinear on $\mathcal{B}_{\ell_\sigma}$ that the verifier evaluates from the public wiring matrices and the challenges r_x, γ . Batching the products a, b, c with γ and splitting across segments,

$$a + \gamma b + \gamma^2 c = \sum_{\sigma} \langle z_\sigma, t_\sigma \rangle = \langle z_{\text{pub}}, t_{\text{pub}} \rangle + \langle z_{\text{pre}}, t_{\text{pre}} \rangle + \langle z_{\text{wit}}, t_{\text{wit}} \rangle.$$

The public summand the verifier computes on its own; the precommit and witness summands are *linear queries* (§2.3) on the committed oracles z_{pre} and z_{wit} , with operands t_{pre} and t_{wit} . As in the Binius64 case these operands are *transparent* – the verifier evaluates each t_σ from the public wiring matrices – but, unlike there, they are not succinct: evaluating t_σ reads the wiring matrices. IronSpartan does not require succinctness. It therefore omits Spartan’s second sumcheck entirely, emitting the wiring claim as linear queries that the compiler (§5) verifies against the Reed–Solomon encodings of the segment oracles. The constraint check is retained as in Spartan.

Splitting the target. The constraint check fixes only the total $s := a + \gamma b + \gamma^2 c$, whereas the compiler needs a separate target for each oracle query. Once γ is sampled, the prover sends a single field element, the precommit contribution

$$s_{\text{pre}} := \langle z_{\text{pre}}, t_{\text{pre}} \rangle.$$

The verifier computes the public contribution $s_{\text{pub}} := \langle z_{\text{pub}}, t_{\text{pub}} \rangle$ on its own and recovers the witness target as the residual

$$s_{\text{wit}} := s - s_{\text{pub}} - s_{\text{pre}}.$$

IronSpartan then emits two linear queries, $(t_{\text{pre}}, s_{\text{pre}})$ on z_{pre} and $(t_{\text{wit}}, s_{\text{wit}})$ on z_{wit} . Together with the verifier’s public computation these pin the full inner product: a prover satisfying both queries satisfies $\sum_{\sigma} \langle z_\sigma, t_\sigma \rangle = s$. The extra element s_{pre} costs no soundness, as it is itself certified by the precommit query.

Toward zero knowledge. Three leakage sources must be closed to make IronSpartan honest-verifier zero-knowledge:

- the *constraint-check sumcheck*, whose round polynomials depend on the committed segments; we mask it with a Libra-style masking polynomial committed as an auxiliary oracle π_g (§6.3);
- the *oracle openings* of z_{pre} , z_{wit} , and π_g performed by the compiler’s BaseFold instance; these are masked by the randomizable support and mask oracles of the Batched ZK BaseFold compiler (§7.2);
- the revealed products (a, b, c) , witness-dependent linear images of the assignment; we mask these with dummy constraints (§6.4).

The first and third are intrinsic to the LIOP and are treated below; the second is supplied by the compiler. Instantiated over binary fields via the binary BaseFold of §5.2 (FRI-Binius [DP25]), the result runs over the same fields as the rest of this document.

6.3 Zero-Knowledge Sumcheck Machinery

IronSpartan’s constraint check (§6.5.1) is a sumcheck, and the standard sumcheck protocol is not zero-knowledge: each round polynomial is a partial sum of the witness-dependent summand and leaks information about it. We recall the masking technique of Libra [Xie+19], which restores zero knowledge in odd characteristic but *fails* in characteristic 2 – the case of interest, since K has characteristic 2 – and then show that the *multilinear-extension-check* form of IronSpartan’s constraint check repairs the masking in every characteristic. The honest-verifier zero-knowledge statement is Theorem 6.3, proved in Appendix D.

Throughout, $f \in K[X_0, \dots, X_{n-1}]$ is an n -variate polynomial of individual degree at most d , and we treat the sumcheck claim $\sum_{x \in \mathcal{B}_n} f(x) = s_f$. In round $i = 0, \dots, n-1$ the prover binds X_{n-i-1} and sends the univariate *round polynomial*

$$R_f^{(i)}(X) := \sum_{v \in \mathcal{B}_{n-i-1}} f(v, X, r_{n-i}, \dots, r_{n-1}), \quad \deg R_f^{(i)} \leq d,$$

where $r_{n-i}, \dots, r_{n-1}$ are the prior challenges; the verifier checks $R_f^{(i)}(0) + R_f^{(i)}(1) = s^{(i)}$ against the running claim (with $s^{(0)} = s_f$), samples $r_{n-i-1} \leftarrow K$, and sets $s^{(i+1)} := R_f^{(i)}(r_{n-i-1})$. Transmitting d of the $d+1$ coefficients of each $R_f^{(i)}$ suffices, the last being fixed by the consistency check; these coefficients, with s_f , are the messages a zero-knowledge variant must hide.

6.3.1 Libra and Its Failure in Characteristic 2

Libra masks the sumcheck by additively blending in a random polynomial. The prover samples a *masking polynomial*

$$g(X_0, \dots, X_{n-1}) = g_c + \sum_{j=0}^{n-1} g_j(X_j) \cdot X_j, \quad g_j(X) = \sum_{k=0}^{d-1} g_{j,k} X^k,$$

with constant $g_c \in K$ and random univariates g_j of degree $\leq d-1$, so g has individual degree $\leq d$ and $1 + nd$ coefficients. It announces $s_g := \sum_{v \in \mathcal{B}_n} g(v)$; the verifier samples a blending challenge $\alpha \leftarrow K$; and the parties sumcheck the combined claim

$$s_f + \alpha s_g = \sum_{x \in \mathcal{B}_n} (f + \alpha g)(x),$$

with round polynomials $R^{(i)} = R_f^{(i)} + \alpha R_g^{(i)}$. After the final challenge the prover reveals $g^* := g(r)$ at $r = (r_0, \dots, r_{n-1})$, and the verifier recovers the unmasked value $f^* := s^{(n)} - \alpha g^*$.

Committing the mask. Since g has individual degree $d > 1$ it is not multilinear and cannot be an LIOP oracle directly; the prover commits a multilinear proxy for its coefficients. Put

$$\nu := \lceil \log_2 n \rceil, \quad \delta := \lceil \log_2(d+1) \rceil,$$

and let $g': \mathcal{B}_\nu \times \mathcal{B}_\delta \rightarrow K$ be a uniformly random multilinear, committed as the oracle $\pi_g := g'$. Identifying $\mathcal{B}_\nu \cong \{0, \dots, 2^\nu - 1\}$ and $\mathcal{B}_\delta \cong \{0, \dots, 2^\delta - 1\}$ (§2), the prover reads the mask coefficients off g' ,

$$g_c := \sum_{j \in \mathcal{B}_\nu} g'(j, 0), \quad g_{j,k-1} := g'(j, k) \quad (j < n, 1 \leq k \leq d),$$

the other entries of g' remaining slack. To evaluate g at the sumcheck point r , define the multilinear $\text{libra-eval}_r: \mathcal{B}_\nu \times \mathcal{B}_\delta \rightarrow K$ by

$$\text{libra-eval}_r(j, k) := \begin{cases} r_j^k & \text{if } j < n \text{ and } k \leq d, \\ 0 & \text{otherwise,} \end{cases}$$

with r_j the j -th coordinate of r . Expanding,

$$\langle g', \text{libra-eval}_r \rangle = \sum_{(j,k) \in \mathcal{B}_{\nu+\delta}} g'(j, k) \text{libra-eval}_r(j, k) = g_c + \sum_{j=0}^{n-1} g_j(r_j) r_j = g(r),$$

so the claim $g^* = g(r)$ is a linear query on π_g with operand libra-eval_r and target g^* , which the verifier evaluates in $O(nd)$ operations.

PROTOCOL 6.1 (Zero-knowledge sumcheck via Libra masking). *Claim:* $\sum_{x \in \mathcal{B}_n} f(x) = s_f$, for f of individual degree $\leq d$.

1. *Masking.* The prover samples $g' \leftarrow K^{\mathcal{B}_{\nu+\delta}}$, commits $\pi_g := g'$, derives g as above, and sends $s_g = \sum_{v \in \mathcal{B}_n} g(v)$. The verifier samples $\alpha \leftarrow K$ and sets $s^{(0)} := s_f + \alpha s_g$.
2. *Rounds.* For $i = 0, \dots, n-1$: the prover sends all but one coefficient of $R^{(i)} = R_f^{(i)} + \alpha R_g^{(i)}$; the verifier recovers the last from $R^{(i)}(0) + R^{(i)}(1) = s^{(i)}$, samples $r_{n-i-1} \leftarrow K$, and sets $s^{(i+1)} := R^{(i)}(r_{n-i-1})$.
3. *Reduction.* The prover reveals $g^* = g(r)$. The verifier forms $f^* := s^{(n)} - \alpha g^*$ and outputs the evaluation claim $f^* = f(r)$ on f and the linear query $g^* = \langle g', \text{libra-eval}_r \rangle$ on π_g .

Failure in characteristic 2. The masking messages s_g and the transmitted coefficients of the $R_g^{(i)}$ – form a fixed K -linear image L_r of the $1 + nd$ coefficients of g . Expanding over the hypercube,

$$s_g = 2^n g_c + 2^{n-1} \sum_{j=0}^{n-1} g_j(1), \quad R_g^{(i)}(X) = 2^{n-i-1} g_{n-i-1}(X) X + (\text{terms in } g_c \text{ and the } g_j, j \neq n-i-1),$$

so the message block carrying the fresh univariate g_{n-i-1} is scaled by 2^{n-i-1} , and L_r is block lower-triangular with these powers of two on its diagonal. In odd characteristic the diagonal entries are units, L_r is invertible, and a uniform mask yields uniform messages (Appendix D). In characteristic 2, however, $2^k = 0$ for $k > 0$: the diagonal collapses for every round $i < n-1$, the rank of L_r falls to $1 + d$, and the non-constant coefficients of the first $n-1$ round polynomials are left unmasked. Libra masking thus gives no zero knowledge over K .

6.3.2 Zero-Knowledge Multilinear-Extension Check

IronSpartan's constraint check is not a bare sumcheck but a *multilinear-extension check* (§2): the summand carries an explicit equality-indicator factor, the claim being $\sum_{x \in \mathcal{B}_n} \tilde{\text{eq}}_n(\tau, x) f(x) = s_f$ for a verifier challenge $\tau \in K^n$. Masking it with the same g, g', α – now summing the $\tilde{\text{eq}}_n(\tau, \cdot)$ -weighted

combination – repairs the masking in every characteristic, following Gruen [Gru24]. The equality-indicator factor is pulled out of the round polynomials, so each $R^{(i)}$ stays of degree $\leq d$ and the round relation becomes the $\tilde{\text{eq-weighted}}$ consistency

$$s^{(i)} = (1 - \tau_{n-i-1}) R^{(i)}(0) + \tau_{n-i-1} R^{(i)}(1), \quad s^{(i+1)} := R^{(i)}(r_{n-i-1}),$$

in place of the unweighted $R^{(i)}(0) + R^{(i)}(1) = s^{(i)}$ of the bare sumcheck.

PROTOCOL 6.2 (Zero-knowledge multilinear-extension check). *Claim:* $\sum_{x \in \mathcal{B}_n} \tilde{\text{eq}}_n(\tau, x) f(x) = s_f$, for a challenge $\tau \in K^n$ and f of individual degree $\leq d$.

The protocol is Protocol 6.1 with every hypercube sum weighted by $\tilde{\text{eq}}_n(\tau, \cdot)$: the prover sends $s_g = \sum_{v \in \mathcal{B}_n} \tilde{\text{eq}}_n(\tau, v) g(v)$, and $R_f^{(i)}, R_g^{(i)}$ are the equality-indicator-factored round polynomials of the weighted summand, each of degree $\leq d$, computed by the MLE-check prover of [Gru24]. In round i the prover sends the degree- ≥ 1 coefficients of $R^{(i)} = R_f^{(i)} + \alpha R_g^{(i)}$; the verifier recovers the constant coefficient from $s^{(i)} = (1 - \tau_{n-i-1}) R^{(i)}(0) + \tau_{n-i-1} R^{(i)}(1)$, samples $r_{n-i-1} \leftarrow K$, and sets $s^{(i+1)} := R^{(i)}(r_{n-i-1})$. The reduction is unchanged: it outputs the evaluation claim $f^* = f(r)$ on f and the linear query $g^* = \langle g', \text{libra-eval}_r \rangle$ on π_g .

Why it works. With the equality-indicator factor pulled out, the as-yet-unbound low variables are summed against $\tilde{\text{eq}}_{n-i-1}(\tau_{<n-i-1}, \cdot)$, contributing the factor $\sum_{x_{<n-i-1}} \tilde{\text{eq}}_{n-i-1}(\tau_{<n-i-1}, x_{<n-i-1}) = 1$ – in place of the $\sum_{x_{<n-i-1}} 1 = 2^{n-i-1}$ of the bare sumcheck. The masking contributions are therefore

$$s_g = g_c + \sum_{j=0}^{n-1} \tau_j g_j(1), \quad R_g^{(i)}(X) = g_c + \sum_{j>n-i-1} g_j(r_j) r_j + \sum_{j<n-i-1} \tau_j g_j(1) + g_{n-i-1}(X) X.$$

The fresh univariate appears as $g_{n-i-1}(X) X$ with *unit* coefficient, rather than scaled by the vanishing power of two 2^{n-i-1} . Its degree- ≥ 1 coefficients are exactly those of g_{n-i-1} , so the round message hides the corresponding coefficients of f in every characteristic; the masking is in fact perfect once the blending challenge α is nonzero (Appendix D).

Theorem 6.3. *The zero-knowledge multilinear-extension check (Protocol 6.2) is honest-verifier zero-knowledge: there is an efficient simulator whose transcript, given the public claim (τ, s_f) and the reduced value f^* , is statistically $O(1)/|K|$ -close to an honest execution. The same masking applied to the bare sumcheck (Protocol 6.1) is honest-verifier zero-knowledge only when $\text{char}(K) \neq 2$.*

The proof is given in Appendix D. This masked multilinear-extension check is what IronSpartan uses for its constraint check (§6.5.1).

6.4 Dummy-Constraint Masking

Masking the sumcheck and the oracle openings still leaves one leak: the constraint check reveals the products (a, b, c) , which are witness-dependent linear images of the assignment evaluated at the random point r_x . We neutralize this by enlarging the instance with *dummy constraints* carrying fresh random values, in the manner of Diamond [Dia25]. Concretely, we append two multiplication constraints whose operands are fresh witness-segment variables assigned uniformly random field elements consistent with the multiplicative relation, occupying two otherwise-unused matrix rows and six fresh columns of the witness segment (two operands and one product per dummy constraint). The wiring extensions $\widetilde{W}_{A,\text{wit}}(r_x, \cdot), \widetilde{W}_{B,\text{wit}}(r_x, \cdot), \widetilde{W}_{C,\text{wit}}(r_x, \cdot)$ then carry random components supported on the dummy columns, rendering the revealed (a, b, c) uniform subject to $a \cdot b = c$ and independent of the real witness.

6.5 The IronSpartan LIOP

Setup. The setup pads the constraint count and each segment to power-of-two dimensions, reserving room for the masking machinery. Writing q for the compiler’s query count (§7.2), it chooses ℓ_M and the ℓ_σ with

$$2^{\ell_M} \geq n_M + 2, \quad 2^{\ell_{\text{pub}}} \geq n_{\text{pub}}, \quad 2^{\ell_{\text{pre}}} \geq n_{\text{pre}} + (q + 1), \quad 2^{\ell_{\text{wit}}} \geq n_{\text{wit}} + 6 + (q + 1).$$

The two extra constraint rows hold the dummy constraints of §6.4; the six extra witness-segment columns hold their fresh variables; and each committed segment, z_{pre} and z_{wit} , reserves a size- $(q + 1)$ randomizable support (§2.3) for the ZK compiler. The randomizable-support columns are all-zero across the wiring matrices, so every operand t_σ vanishes on them, meeting the well-formedness condition of §2.3; the dummy columns, by contrast, are used by the dummy rows. Remaining padding entries are zero.

Protocol. In keeping with the commit-and-prove setting, the prover first commits the precommit oracle z_{pre} (with its randomizable support filled uniformly). Once the public instance z_{pub} is fixed, the prover commits the witness oracle z_{wit} (likewise) and the Libra oracle π_g (the masking coefficients g' of §6.3.2). The parties then run the constraint-check reduction (§6.5.1) followed by the reduction to inner-product claims (§6.5.2), terminating in linear queries on z_{pre} , z_{wit} , and π_g – three oracles with one query each, exactly the setting of Batched BaseFold (§5.4).

6.5.1 Constraint-Check Reduction

The constraint-check reduction is the zero-knowledge MLE-check of §6.3.2 applied to the R1CS constraint polynomial.

1. The verifier samples $\tau \leftarrow K^{\ell_M}$.
2. The parties run a zero-knowledge MLE-check, masked by the Libra polynomial committed in π_g , on

$$\sum_{x \in \mathcal{B}_{\ell_M}} \widetilde{\text{eq}}_{\ell_M}(\tau, x) \cdot [\widetilde{(Az)}(x) \widetilde{(Bz)}(x) - \widetilde{(Cz)}(x)] = 0.$$

3. The check reduces to a random point $r_x \in K^{\ell_M}$, at which the prover sends the claimed products $a = \widetilde{(Az)}(r_x)$, $b = \widetilde{(Bz)}(r_x)$, $c = \widetilde{(Cz)}(r_x)$ of §6.1; the verifier checks $a \cdot b = c$ and samples a batching challenge $\gamma \leftarrow K$.

The reduction outputs the claimed products (a, b, c) , the point r_x , the challenge γ , and a Libra evaluation claim – a linear query on π_g (operand libra-eval_r , target g^*).

6.5.2 Reduction to Inner-Product Claims on Matrix Extensions

It remains to certify the three products (a, b, c) . As in §6.2, IronSpartan batches them with γ and splits the inner product across segments using the operands t_σ . The prover sends the precommit contribution $s_{\text{pre}} = \langle z_{\text{pre}}, t_{\text{pre}} \rangle$; the verifier computes the public contribution $s_{\text{pub}} = \langle z_{\text{pub}}, t_{\text{pub}} \rangle$ and forms the witness target $s_{\text{wit}} = (a + \gamma b + \gamma^2 c) - s_{\text{pub}} - s_{\text{pre}}$. The LIOP thus terminates with three linear queries – $(t_{\text{pre}}, s_{\text{pre}})$ on z_{pre} , $(t_{\text{wit}}, s_{\text{wit}})$ on z_{wit} , and the Libra query on π_g – whose transparent operands the verifier evaluates from the public matrices. The compiler of §5 discharges all three with a single Batched BaseFold instance. Replacing it with the Batched ZK BaseFold compiler of §7.2, and using the randomizable supports and dummy-constraint masking established above, makes the compiled argument honest-verifier zero-knowledge; the full simulator is given in §7.

7 Zero-Knowledge Compiler

7.1 Proof Composition for ZK-SNARKs

Our zero-knowledge SNARK is obtained by *proof composition*: an arbitrary, not-necessarily-zero-knowledge LIOP for a relation R is wrapped inside the zero-knowledge commit-and-prove IronSpartan LIOP of §6, yielding a zero-knowledge LIOP for the same relation R . The strategy instantiates the classical paradigm of Ben-Or et al. [Ben+90] – every interactive proof can be made zero-knowledge by having the prover commit (encrypt) its messages and then prove, in zero knowledge, that the verifier’s acceptance predicate holds on the committed messages. We follow the concrete realizations of Frigo and shelat [Fs24] and of VEIL [DHRR26], which specialize this paradigm to the hash-based multilinear

proof systems compiled in §5. IronSpartan is the outer, zero-knowledge proof: its R1CS commit-and-prove interface is exactly what is needed to certify the inner verifier’s predicate without revealing the inner prover’s messages.

We call the wrapped protocol the *inner* LIOP, and its prover and verifier the inner prover and inner verifier. The composition encrypts every value the inner prover would reveal under a one-time pad, so it requires the number of such values to be *predetermined* – a fixed function of the relation R rather than of the witness. There are two kinds, each needing its own OTP key: the non-oracle field elements the inner prover sends during the protocol, and the targets of the linear claims its verifier reduces to. Both counts are predetermined for the LIOPs of this document. The two protocols are then interleaved as follows.

1. **Precommit.** Before the public instance is fixed, the composed prover commits IronSpartan’s precommit segment z_{pre} , which holds one independent uniform *one-time-pad* (OTP) key for each non-oracle field element the inner prover will send. (The OTP keys for the linear-claim targets are supplied separately, by the randomizable supports of the inner oracles; see below.) Because the count is predetermined and the precommit segment may be fixed ahead of the statement (§6), this commitment is well defined.
2. **Run the inner LIOP.** The composed prover runs the inner prover against the public-coin verifier. Whenever the inner prover would send a non-oracle field element m , the composed prover instead sends the ciphertext $c := m + k$, where k is the OTP key precommitted for that element. Since each k is uniform and used once, every c is uniform and independent of m , so the inner prover’s non-oracle messages leak nothing. The inner oracles, meanwhile, are committed as oracles of the composed LIOP, to be discharged by the compiler of §7.2.
3. **Run IronSpartan.** Finally, the composed prover invokes the IronSpartan LIOP to prove that the plaintexts $m = c - k$ – with the ciphertexts c public and the keys k read from the precommit segment – satisfy exactly the algebraic relations that the inner verifier would check on the inner prover’s messages. These checks are low-degree in the transmitted elements and the (public) challenges, so they form an R1CS instance over $(z_{\text{pre}}, z_{\text{wit}})$ that IronSpartan discharges in zero knowledge.

The inner verifier’s predicate ultimately reduces to a set of linear claims on the inner oracles – the inner LIOP’s terminal linear queries (§2.3). The target $s = \langle \pi_i, t \rangle$ of each such claim is a witness-dependent linear image of an inner oracle, so it too is hidden under a one-time pad. Here the key is drawn from the oracle itself: the targeted oracle π_i carries one additional index in its randomizable support (§2.3), whose uniform value k is the claim’s OTP key. The composed prover reveals only the ciphertext $c := s + k$, which is uniform; IronSpartan then certifies, against c , that the underlying claim is the one the inner verifier would check. Thus each reduced linear claim consumes exactly one OTP key, just as each non-oracle field element does – the difference being that the claim keys live in the inner oracles’ randomizable supports rather than in z_{pre} .

7.2 Batched ZK BaseFold

The compiler of §5.1 is not zero-knowledge: BaseFold’s query phase reveals codeword symbols of the committed oracles, and its folding phase reveals sumcheck round polynomials and folded codewords that depend on those oracles. We now give an *alternate* compiler, *Batched ZK BaseFold*, with the stronger guarantee that whenever the source LIOP is honest-verifier zero-knowledge (HVZK), the compiled interactive proof is HVZK as well – and hence, after Fiat–Shamir, a non-interactive zero-knowledge argument in the random oracle model. It follows the construction of Diamond [Dia25], itself inspired by the low-degree-test masking technique of Aurora [Ben+19], adapted to the batched setting of §5.4. It replaces the Batched BaseFold instantiation of Stage 1 in the compiler of §5.1, leaving the BCS and Fiat–Shamir stages untouched.

Two leakage channels must be closed, and we close them with two separate mechanisms. The *query phase* opens each committed codeword at the verifier’s chosen positions, directly exposing symbols of $\text{Enc}(\pi_i)$; these we hide with the randomizable support of §2.3. The *folding phase* reveals round polynomials and intermediate codewords that depend on the oracles; these we hide by combining each oracle with an independent random mask codeword, in the manner of Aurora.

Randomizable-support requirement. Write q for the number of query positions sampled in BaseFold’s query phase (the parameter μ of §5.2). We require each LIOP oracle $\pi_i: \mathcal{B}_{n_i} \rightarrow K$ to carry a randomizable support (§2.3) $I_i \subseteq \mathcal{B}_{n_i}$ of size $|I_i| = q + 1$. Of these coordinates, q absorb the q codeword symbols of $\text{Enc}_{n_i}(\pi_i)$ exposed in the query phase, and the remaining one supplies the randomness that makes the Merkle commitment hiding, whose role we defer to the detailed zero-knowledge analysis below. By the well-formedness condition of §2.3, every query operand t_i targeting π_i vanishes on I_i , so filling I_i with randomness leaves the opening targets s_i – and thus completeness and soundness – unchanged.

Commit phase. When the LIOP prover would send an oracle $\pi_i: \mathcal{B}_{n_i} \rightarrow K$, the IOP prover instead proceeds as follows. It first fills the randomizable support, drawing $\pi_i(w) \leftarrow K$ uniformly and independently for each $w \in I_i$ and leaving the protocol-determined coordinates intact. It then samples a uniformly random *mask* $\omega_i: \mathcal{B}_{n_i} \rightarrow K$ of the same length, and commits to the *interleaved* Reed–Solomon codeword whose two rows are $\text{Enc}_{n_i}(\pi_i)$ and $\text{Enc}_{n_i}(\omega_i)$: the Merkle commitment is taken over the columns of this $2 \times |E_{n_i}|$ array, so that opening a single codeword position e exposes both $\text{Enc}_{n_i}(\pi_i)[e]$ and $\text{Enc}_{n_i}(\omega_i)[e]$ under one authentication path.

Opening phase. The LIOP concludes with opening claims $\langle \pi_i, t_i \rangle = s_i$ for $i = 0, \dots, k - 1$ (§5.4), which the parties discharge in three steps.

1. For each i , the prover sends the masked inner product $\sigma_i := \langle \omega_i, t_i \rangle = \sum_{w \in \mathcal{B}_{n_i}} t_i(w) \cdot \omega_i(w)$.
2. The verifier samples a masking challenge $\gamma \leftarrow K$.
3. Both parties form, for each i , the virtual oracle $\pi_i^\gamma := \pi_i + \gamma \cdot \omega_i$ with target $s_i^\gamma := s_i + \gamma \cdot \sigma_i$, so that $\langle \pi_i^\gamma, t_i \rangle = s_i^\gamma$, and run Batched BaseFold (§5.4) on the claims $\langle \pi_i^\gamma, t_i \rangle = s_i^\gamma$. The verifier emulates oracle access to the codeword $\text{Enc}_{n_i}(\pi_i^\gamma) = \text{Enc}_{n_i}(\pi_i) + \gamma \cdot \text{Enc}_{n_i}(\omega_i)$ by reading both rows of the interleaved commitment at the queried column and combining them with γ (lifting to the common level $\mathbf{n}$ as in §5.3 where needed).

Because γ scales a genuine codeword $\text{Enc}_{n_i}(\omega_i)$, the combination $\text{Enc}_{n_i}(\pi_i^\gamma)$ is again a codeword of the same code, and the masked sumcheck constraint $\langle \pi_i^\gamma, t_i \rangle = s_i^\gamma$ is exactly the constraint Batched BaseFold expects.

Soundness. Fix the committed rows π_i, ω_i . The masked identity $\langle \pi_i^\gamma, t_i \rangle = s_i^\gamma$ is an affine identity in γ that holds for a uniformly random γ only if both $\langle \pi_i, t_i \rangle = s_i$ and $\langle \omega_i, t_i \rangle = \sigma_i$, except with probability $1/|K|$ by Schwartz–Zippel; sending σ_i before γ is sampled prevents the prover from adapting it. The masking step therefore reduces verifying the original claims to verifying the masked claims, adding $O(1)/|K|$ to the soundness error of Batched BaseFold. Knowledge soundness is preserved: the extractor recovers each π_i from the first row of its interleaved commitment exactly as in §5.4.

Zero knowledge. The mask $\gamma \cdot \omega_i$ is a uniformly random codeword (for $\gamma \neq 0$), so the folding-phase transcript of Batched BaseFold run on π_i^γ – its sumcheck round polynomials, intermediate folded codewords, and final reduced codeword – is distributed independently of the witness, as is the revealed σ_i . Independently, the q exposed symbols of each $\text{Enc}_{n_i}(\pi_i)$ are rendered jointly uniform by the random fill on I_i , so the query-phase openings leak nothing. Composed with the HVZK simulator for the source LIOP – which produces the opening targets s_i and the non-oracle transcript without the witness – these two mechanisms let a simulator reproduce the entire compiled transcript. We defer the full simulator, including the treatment of the Merkle commitments, to the detailed analysis below.

7.3 Compiling Binius64 to a ZK-SNARK

We assemble the zero-knowledge SNARK for the Binius64 constraint system from three components: the Binius64 LIOP of §4, the IronSpartan LIOP of §6, and the Batched ZK BaseFold compiler of §7.2. The Binius64 LIOP is complete and knowledge-sound but not zero-knowledge: its sumcheck round polynomials and its terminal evaluation claim are witness-dependent. We take it as the *inner* LIOP of the proof composition of §7.1 and discharge its verifier’s predicate with IronSpartan as the outer zero-knowledge proof.

Recall (§4.1) that the Binius64 LIOP commits a single oracle – the packed witness $\pi = \text{pack}_\lambda(w)$ – runs the IntMul, BinMul, BitAnd, shift, ring-switching, and public-input reductions, and terminates in a single linear query on π . Apart from this committed oracle, the prover transmits a predetermined number N of non-oracle field elements: the sumcheck round-polynomial coefficients, the claimed evaluations passed between reductions, and the GKR layer claims, all fixed in count by the padded constraint-system parameters. This is the predetermined- N condition the composition of §7.1 requires.

Following that composition, the composed prover first precommits, in IronSpartan’s precommit segment z_{pre} , one uniform OTP key per non-oracle element. It then runs the Binius64 prover, sending each of the N elements masked by its key, so the transcript reveals only uniform ciphertexts. The Binius64 verifier’s acceptance is a conjunction of low-degree checks – the per-round sumcheck consistency relations $R^{(i)}(0) + R^{(i)}(1) = s^{(i)}$, the products checked between the reductions, and the derivation of the final query target – each a low-degree function of the transmitted elements and the public Fiat–Shamir challenges. The composed prover encodes this conjunction as an R1CS instance whose public segment z_{pub} holds the ciphertexts, the challenges, and the Binius64 public statement u ; whose precommit segment holds the OTP keys; and whose witness segment z_{wit} holds the plaintext elements and the intermediate values of the checks. Since the Binius64 verifier is succinct, this R1CS is small – its size is that of the verifier’s computation, polylogarithmic in the witness – so IronSpartan proves it cheaply, in zero knowledge.

The one place the inner oracle survives into the compiled argument is the Binius64 LIOP’s terminal linear query $\langle \pi, t \rangle = s$ – the combined ring-switching and public-input query of §4.1. As s is a witness-dependent linear image of π , the packed-witness oracle carries one index in its randomizable support beyond the $q + 1$ required by Batched ZK BaseFold (§7.2); its uniform value is the OTP key k for this single claim. The composed prover reveals only the ciphertext $c = s + k$, IronSpartan certifies the underlying claim against c , and the query itself is discharged against the committed π by the compiler.

The composed LIOP is thus honest-verifier zero-knowledge and commits four oracles, each with a single opening claim: the packed witness π and IronSpartan’s z_{pre} , z_{wit} , and Libra oracle π_g . This is precisely the input shape of Batched ZK BaseFold; compiling Stage 1 with it and leaving the BCS and Fiat–Shamir stages of §5.1 untouched yields a non-interactive zero-knowledge argument for the Binius64 relation in the random oracle model.

Security. The argument is honest-verifier zero-knowledge: a simulator with no access to the witness produces the transcript in three layers. The N non-oracle messages are reproduced as uniform ciphertexts, matching the honest distribution because each OTP key is uniform and used once. IronSpartan’s portion is produced by its HVZK simulator (§6.3, Appendix D), which simulates the masked constraint check, the dummy-masked products (§6.4), and the segment-query targets. Finally, the Batched ZK BaseFold simulator (§7.2) reproduces the commit, folding, and query phases of all four oracles from the opening targets alone, using the randomizable supports – including the extra index on π – to render the exposed codeword symbols and the revealed claim ciphertext c uniform. After Fiat–Shamir this is a non-interactive zero-knowledge argument in the random oracle model. Knowledge soundness and the soundness error are inherited from the inner LIOP, IronSpartan, and the compiler, increased only by the $O(1)/|K|$ terms of the masking and composition steps.

A Expressing Common Operations

The constraint system is far more expressive than its primitive forms suggest; bitwise NOT, OR, integer addition, and integer subtraction are all expressible using Zero and BitAnd constraints with appropriate shifted value indices. We sketch the encodings below, motivated by the RV64I instruction set.

Bitwise AND, XOR, NOT, OR. Bitwise AND and bitwise XOR are both native: AND is a BitAnd constraint, and $z = x \oplus y$ is the Zero constraint $x \oplus y \oplus z = 0$. To constrain that an unshifted value equals $w[y]$, one uses the shifted index $(y, \text{sll}, 0, \text{sll}, 0)$, which acts as the identity. Bitwise NOT is a Zero constraint naming the constant $0\text{xFFFFFFFFFFFFFFFF}$ (available because that constant is placed in the constant stretch): the constraint $x \oplus y \oplus 0\text{xFFFFFFFFFFFFFFFF} = 0$ forces $y = \bar{x}$. Bitwise OR follows from the identity $x \vee y = (x \wedge y) \oplus x \oplus y$: the BitAnd constraint $x \wedge y = x \oplus y \oplus z$ forces $z = x \vee y$.

Integer addition. To constrain $z = (x + y) \bmod 2^{64}$, the prover supplies an auxiliary witness word c_{out} whose bits are the per-position carries. Writing $c_{\text{in}} := \text{sll}(c_{\text{out}}, 1)$, the BitAnd constraint

$$(x \oplus c_{\text{in}}) \wedge (y \oplus c_{\text{in}}) = c_{\text{in}} \oplus c_{\text{out}}$$

enforces correct carry propagation, and the Zero constraint $z \oplus x \oplus y \oplus c_{\text{in}} = 0$ forces $z = x \oplus y \oplus c_{\text{in}}$.

Integer subtraction. To constrain $z = (x - y) \bmod 2^{64}$, the prover supplies an auxiliary witness word b_{out} whose bits are the per-position borrows. Writing $b_{\text{in}} := \text{sll}(b_{\text{out}}, 1)$, the BitAnd constraint

$$(\bar{x} \oplus b_{\text{in}}) \wedge (y \oplus b_{\text{in}}) = b_{\text{in}} \oplus b_{\text{out}}$$

enforces correct borrow propagation, and the Zero constraint $z \oplus x \oplus y \oplus b_{\text{in}} = 0$ forces $z = x \oplus y \oplus b_{\text{in}}$.

B Arithmetization of Shift Indicators

This appendix gives the explicit arithmetizations of the eight shift indicators $\widetilde{\text{shift-ind}}_{\text{op}}$ defined in §4.2, establishing the succinctness claim stated there. The constructions generalize the shifted virtual polynomial construction of Diamond and Posen [DG25, §4.3].

B.1 Logical Shifts and Rotation

We arithmetize $\widetilde{\text{shift-ind}}_{\text{srl}}$ first. The case sll follows by transposing the first two arguments, since on the cube

$$\text{shift-ind}_{\text{sll}}(i, j, s) = \text{shift-ind}_{\text{srl}}(j, i, s),$$

and the same identity extends to the multilinear extensions.

The shift indicators of srl , sll , and ror all reduce to two auxiliary functions $\mathbf{s}_k, \mathbf{s}'_k$ on $\mathcal{B}_k^3$, defined for $k \in \{0, \dots, 6\}$ by

$$\mathbf{s}_k(i, j, s) := \begin{cases} 1 & \text{if } j = i + s \\ 0 & \text{otherwise} \end{cases}, \quad \mathbf{s}'_k(i, j, s) := \begin{cases} 1 & \text{if } j = i + s - 2^k \\ 0 & \text{otherwise} \end{cases},$$

where $i, j, s \in \mathcal{B}_k$ are read as integers (§2) and the equalities $j = i + s$ and $j = i + s - 2^k$ are taken over the integers. So $\mathbf{s}_k$ indicates that $i + s$ does not overflow modulo 2^k and equals j , while $\mathbf{s}'_k$ indicates that $i + s$ overflows by exactly 2^k and equals $j + 2^k$.

By definition,

$$\text{shift-ind}_{\text{srl}} = \mathbf{s}_6.$$

The function $\text{shift-ind}_{\text{ror}}(i, j, s)$ holds iff $j \equiv i + s \pmod{64}$; since $0 \leq i, s < 64$, this means $j = i + s$ or $j = i + s - 64$, and so

$$\text{shift-ind}_{\text{ror}} = \mathbf{s}_6 + \mathbf{s}'_6.$$

Recursive arithmetization. The functions $\mathbf{s}_k, \mathbf{s}'_k$ satisfy a recursive identity. The base case is $\mathbf{s}_0 = 1$ and $\mathbf{s}'_0 = 0$. For the inductive case, write $i = (i_{<k-1}, i_{k-1})$ for the splitting of $i \in \mathcal{B}_k$ into its low $k-1$ bits and its top bit, similarly for j and s ; then on $\mathcal{B}_k^3$,

$$\mathbf{s}_k(i, j, s) = \begin{cases} \mathbf{s}_{k-1}(i_{<k-1}, j_{<k-1}, s_{<k-1}) & \text{if } i_{k-1} + s_{k-1} = j_{k-1}, \\ \mathbf{s}'_{k-1}(i_{<k-1}, j_{<k-1}, s_{<k-1}) & \text{if } i_{k-1} + s_{k-1} + 1 = j_{k-1}, \\ 0 & \text{otherwise,} \end{cases}$$

$$\mathbf{s}'_k(i, j, s) = \begin{cases} \mathbf{s}_{k-1}(i_{<k-1}, j_{<k-1}, s_{<k-1}) & \text{if } i_{k-1} + s_{k-1} = j_{k-1} + 2, \\ \mathbf{s}'_{k-1}(i_{<k-1}, j_{<k-1}, s_{<k-1}) & \text{if } i_{k-1} + s_{k-1} + 1 = j_{k-1} + 2, \\ 0 & \text{otherwise,} \end{cases}$$

where all bit arithmetic is taken over the integers.

Each of the four bit-pattern conditions on the top bits $(i_{k-1}, j_{k-1}, s_{k-1}) \in \mathcal{B}_1^3$ admits a 2-multiplication arithmetization over $\mathbb{F}_2$:

$$\begin{aligned} i + s = j &\iff 1 + j + s + i \cdot (1 + s \cdot (1 + j)) = 1, \\ i + s + 1 = j &\iff (1 - i) \cdot j \cdot (1 - s) = 1, \\ i + s = j + 2 &\iff i \cdot (1 - j) \cdot s = 1, \\ i + s + 1 = j + 2 &\iff i + s + j \cdot (i + s \cdot (1 + i)) = 1. \end{aligned}$$

Substituting these into the recursion yields multilinear formulas for $\widetilde{s}_k$ and $\widetilde{s}'_k$ in terms of $\widetilde{s}_{k-1}$ and $\widetilde{s}'_{k-1}$. Together, the two formulas advance a pair of registers using at most 12 multiplications per level; over $k = 1, \dots, 6$ this gives at most $\widetilde{72}$ multiplications in K to evaluate the pair $(\widetilde{s}_6, \widetilde{s}'_6)$ at any point of K^{18} . In particular, $\text{shift-ind}_{\text{srl}}$, $\text{shift-ind}_{\text{sll}}$, and $\text{shift-ind}_{\text{ror}}$ are all succinct.

B.2 Arithmetic Right Shift

The arithmetic right shift sra behaves like srl except that the sign bit of v (bit 63) is replicated into the vacated high positions. Concretely, $\text{sra}(v, s)_i = v_{i+s}$ if $i + s < 64$, and $\text{sra}(v, s)_i = v_{63}$ otherwise. Hence

$$\text{shift-ind}_{\text{sra}}(i, j, s) = \text{shift-ind}_{\text{srl}}(i, j, s) + \left(\prod_{k=0}^5 j_k \right) \cdot \mathbf{a}(i, s),$$

where the indicator $\prod_{k=0}^5 j_k$ selects $j = 63$ and the auxiliary function $\mathbf{a}: \mathcal{B}_6 \times \mathcal{B}_6 \rightarrow \mathbb{F}_2$ is defined by $\mathbf{a}(i, s) = 1$ iff $i \geq 64 - s$ (equivalently, iff $i + s$ overflows modulo 64).

We arithmetize $\mathbf{a}$ via the sequence of functions $\mathbf{a}_k: \mathcal{B}_k \times \mathcal{B}_k \rightarrow \mathbb{F}_2$, $k \in \{0, \dots, 6\}$, defined by $\mathbf{a}_k(i, s) = 1$ iff $i \geq 2^k - s$, so that $\mathbf{a} = \mathbf{a}_6$. The base case is $\mathbf{a}_0 = 0$, and for $k \geq 1$,

$$\mathbf{a}_k(i, s) = \begin{cases} 1 & \text{if } i_{k-1} = s_{k-1} = 1, \\ \mathbf{a}_{k-1}(i_{<k-1}, s_{<k-1}) & \text{if } i_{k-1} \neq s_{k-1}, \\ 0 & \text{otherwise.} \end{cases}$$

The arithmetization of this recursion is the single-line update

$$\widetilde{\mathbf{a}}_k(I, S) = I_{k-1} \cdot S_{k-1} + (I_{k-1} + S_{k-1}) \cdot \widetilde{\mathbf{a}}_{k-1}(I, S),$$

costing at most two multiplications per level, hence at most 12 multiplications in K for $\widetilde{\mathbf{a}}_6$. Together with the 72 multiplications for $\widetilde{s}_6$, evaluating $\text{shift-ind}_{\text{sra}}$ at any point of K^{18} costs at most 84 multiplications plus a 6-way product for $\prod_{k=0}^5 J_k$.

B.3 Parallel 32-bit Shifts

The four parallel 32-bit operations sll32 , srl32 , sra32 , ror32 treat each 64-bit word as two independent 32-bit halves and apply the corresponding 32-bit shift to both halves. The shift amount s is taken modulo 32, i.e., only the low 5 bits $S_0, \dots, S_4$ are used; the top bit S_5 is ignored.

Each 32-bit indicator is a product of a 5-bit version of the corresponding 64-bit indicator (acting on $I_0, \dots, I_4$ and $J_0, \dots, J_4$) with the equality indicator $\widetilde{\mathbf{eq}}(I_5, J_5)$, which restricts movement to within a single half. Explicitly, writing $I_{<5} := (I_0, \dots, I_4)$, $J_{<5} := (J_0, \dots, J_4)$, and $S_{<5} := (S_0, \dots, S_4)$:

$$\begin{aligned} \widetilde{\text{shift-ind}}_{\text{srl32}}(I, J, S) &= \widetilde{s}_5(I_{<5}, J_{<5}, S_{<5}) \cdot \widetilde{\mathbf{eq}}(I_5, J_5), \\ \widetilde{\text{shift-ind}}_{\text{sll32}}(I, J, S) &= \widetilde{s}_5(J_{<5}, I_{<5}, S_{<5}) \cdot \widetilde{\mathbf{eq}}(I_5, J_5), \\ \widetilde{\text{shift-ind}}_{\text{ror32}}(I, J, S) &= (\widetilde{s}_5(I_{<5}, J_{<5}, S_{<5}) + \widetilde{s}'_5(I_{<5}, J_{<5}, S_{<5})) \cdot \widetilde{\mathbf{eq}}(I_5, J_5), \\ \widetilde{\text{shift-ind}}_{\text{sra32}}(I, J, S) &= \widetilde{\text{shift-ind}}_{\text{srl32}}(I, J, S) + \widetilde{\mathbf{eq}}(I_5, J_5) \cdot \left(\prod_{k=0}^4 J_k \right) \cdot \widetilde{\mathbf{a}}_5(I_{<5}, S_{<5}). \end{aligned}$$

The sign-bit selector $\prod_{k=0}^4 J_k$ in the sra32 formula identifies $j_{<5} = (1, 1, 1, 1, 1)$, which is bit 31 when $J_5 = 0$ and bit 63 when $J_5 = 1$, i.e., the sign bit of whichever half j lies in; the factor $\widetilde{\mathbf{eq}}(I_5, J_5)$ then forces i to lie in the same half.

Each 5-bit auxiliary function is succinct by the same recursion as in §B.1 and §B.2, truncated to $k = 5$: $\tilde{s}_5$ and $\tilde{s}'_5$ cost at most 60 multiplications, and $\tilde{a}_5$ costs at most 10 multiplications. Hence each of the four 32-bit shift indicators is succinct.

C Batched Product Checks of Unequal Depths

The binary-tree grand-product argument [Tha13] recalled in §4.5.2 reduces an evaluation claim on the root of a depth- k product tree to an evaluation claim on its leaf vector. Several such trees of a *common* depth batch readily: interpolating the trees along a selector index merges them into a single tree over an enlarged content domain, and one run of the layer loop discharges them all, at the cost of a claim-combination step at the start and a claim-separation step at the end. This appendix specifies the batched product check for trees of *unequal* depths. The device is the multiplicative analog of the variable padding of §2.2: each shallower tree is padded up to the maximal depth with constant-1 leaves, which leaves its products unchanged, and—the substance of this appendix—the padding commutes with the product-tree layer structure, so that the padded prover messages are cheap corrections of the unpadded ones. The verifier’s control flow depends only on the maximal depth, the transcript having exactly the shape of an equal-depth batch, and the prover’s extra work is a low-order term.

C.1 Setting

Fix a content dimension $\ell \geq 0$ and a selector dimension $\kappa \geq 0$. There are $N \leq 2^\kappa$ *instances*, indexed by $u \in \{0, \dots, N-1\} \subseteq \mathcal{B}_\kappa$. Instance u consists of a depth $k_u \geq 0$ and a leaf vector $L_u \in K^{\mathcal{B}_\ell \times \mathcal{B}_{k_u}}$; as in §4.5.2, its product-tree layers $L_u^{(j)} \in K^{\mathcal{B}_\ell \times \mathcal{B}_{k_u-j}}$, for $j = 0, \dots, k_u$, are defined by $L_u^{(0)} := L_u$ and

$$L_u^{(j)}[x, i] := L_u^{(j-1)}[x, (i, 0)] \cdot L_u^{(j-1)}[x, (i, 1)], \quad x \in \mathcal{B}_\ell, \quad i \in \mathcal{B}_{k_u-j},$$

the trailing bit of the node index selecting the child. We write $p_u := L_u^{(k_u)} \in K^{\mathcal{B}_\ell}$ for the root layer, whose entries are the products $p_u[x] = \prod_{z \in \mathcal{B}_{k_u}} L_u[x, z]$.

Set $n := \max_u k_u$, assumed ≥ 1 , and $\eta_u := n - k_u$. The protocol input is one evaluation claim per instance, all at a *common* content point $\tau \in K^\ell$:

$$s_u \stackrel{?}{=} \tilde{p}_u(\tau), \quad u \in \{0, \dots, N-1\};$$

its output is one evaluation claim per instance on the leaf multilinear $\tilde{L}_u$, at a family of nested points described below. (When $\ell = 0$ the content index is empty and the input claims are the products themselves.)

C.2 One-Padding

Recall the selector $\text{sel}(s, v) = 1 + (v-1)s$ of §4.5.2. For $\nu \geq 0$ and a function $f \in K^{\mathcal{B}_\ell \times \mathcal{B}_m}$, the ν -fold *one-padding* $\text{OnePad}_\nu(f) \in K^{\mathcal{B}_\ell \times \mathcal{B}_{\nu+m}}$ is defined by

$$\text{OnePad}_\nu(f)[x, (i', i)] := \text{sel}(\tilde{\text{eq}}_\nu(0^\nu, i'), f[x, i]) = \begin{cases} f[x, i] & \text{if } i' = 0^\nu, \\ 1 & \text{otherwise,} \end{cases}$$

where the node index is split into its low ν bits i' and high m bits i . Under the integer identification of node indices, one-padding places the entry at node index i of f at node index $2^\nu \cdot i$ and fills the remaining positions with ones; in particular it preserves the products over the node index. On the level of multilinear extensions,

$$\widetilde{\text{OnePad}_\nu(f)}(X, Z', Z) = \text{sel}(\tilde{\text{eq}}_\nu(0^\nu, Z'), \tilde{f}(X, Z)) = 1 + (\tilde{f}(X, Z) - 1) \cdot \tilde{\text{eq}}_\nu(0^\nu, Z'), \quad (9)$$

since the right-hand side is multilinear in all variables and agrees with $\text{OnePad}_\nu(f)$ on the cube.

Define the padded leaf vectors $L'_u := \text{OnePad}_{\eta_u}(L_u) \in K^{\mathcal{B}_\ell \times \mathcal{B}_n}$, with product-tree layers $L'^{(j)}_u$ for $j = 0, \dots, n$ as above. All padded trees have depth n , and the padding threads through their layers as follows.

Lemma C.1 (Padded layers). *For every instance u :*

1. for $0 \leq j \leq k_u$: $L'_u(j) = \text{OnePad}_{\eta_u}(L_u(j))$;
2. for $k_u \leq j \leq n$: $L'_u(j) = \text{OnePad}_{n-j}(p_u)$.

In particular the padded root agrees with the original one, $L'_u(n) = p_u$.

Proof. Upward induction on j , the base case $j = 0$ being the definition of L'_u . For $1 \leq j \leq k_u$, split the layer- $(j-1)$ node index as (i', i, b) with $i' \in \mathcal{B}_{\eta_u}$ the low bits, $i \in \mathcal{B}_{k_u-j}$ the remaining middle bits, and b the trailing bit; by the induction hypothesis,

$$L'_u(j)[x, (i', i)] = \prod_{b \in \mathcal{B}_1} \text{sel}(\tilde{\text{eq}}_{\eta_u}(0^{\eta_u}, i'), L_u^{(j-1)}[x, (i, b)]),$$

which equals 1 when $i' \neq 0^{\eta_u}$, and equals $L_u^{(j-1)}[x, (i, 0)] \cdot L_u^{(j-1)}[x, (i, 1)] = L_u^{(j)}[x, i]$ when $i' = 0^{\eta_u}$; this is $\text{OnePad}_{\eta_u}(L_u^{(j)}[x, (i', i)])$. For $k_u < j \leq n$, the layer- $(j-1)$ node index (i', b) consists of padding bits only, and

$$L'_u(j)[x, i'] = \prod_{b \in \mathcal{B}_1} \text{sel}(\tilde{\text{eq}}_{n-j+1}(0^{n-j+1}, (i', b)), p_u[x]);$$

the $b = 1$ factor is identically 1, and the $b = 0$ factor is $\text{sel}(\tilde{\text{eq}}_{n-j}(0^{n-j}, i'), p_u[x])$, as claimed. $\square$

Lemma C.1 is why the padding bits occupy the *low* end of the node index: the layer loop below runs from the root down and appends one node variable per layer at the end of the evaluation point, so the variables introduced by its first η_u reductions—which act on the pure-padding layers $\text{OnePad}_{n-j}(p_u)$ —occupy the low positions, and only afterwards does the real tree of instance u begin, wrapped throughout in the fixed one-padding $\text{OnePad}_{\eta_u}(\cdot)$.

C.3 The Protocol

Combine the padded trees along the selector index into the single function $\mathbf{L} \in K^{\mathcal{B}_\kappa \times \mathcal{B}_\ell \times \mathcal{B}_n}$,

$$\mathbf{L}[u, x, z] := \begin{cases} L'_u[x, z] & \text{if } u < N, \\ 0 & \text{otherwise,} \end{cases}$$

and let $\mathbf{L}^{(j)} \in K^{\mathcal{B}_\kappa \times \mathcal{B}_\ell \times \mathcal{B}_{n-j}}$ be its product-tree layers *over the node index only*— $\mathbf{L}^{(j)}[u, x, i] = \mathbf{L}^{(j-1)}[u, x, (i, 0)] \cdot \mathbf{L}^{(j-1)}[u, x, (i, 1)]$ —the selector and content indices being carried along passively. By construction,

$$\tilde{\mathbf{L}}^{(j)}(U, X, Z) = \sum_{u=0}^{N-1} \tilde{\text{eq}}_\kappa(u, U) \cdot \tilde{L}'_u(j)(X, Z). \quad (10)$$

Claim combination. The verifier samples $\sigma \leftarrow K^\kappa$, and both parties form the combined root claim

$$y^{(n)} := \sum_{u=0}^{N-1} \tilde{\text{eq}}_\kappa(\sigma, u) \cdot s_u \stackrel{?}{=} \tilde{\mathbf{L}}^{(n)}(\sigma, \tau),$$

the identification of the right-hand side being (10) at $j = n$ together with $L'_u(n) = p_u$ (Lemma C.1). Since the weights $\tilde{\text{eq}}_\kappa(\sigma, u)$ are the values at σ of the κ -variate multilinear Lagrange basis, a family containing a false input claim yields a true combined claim with probability at most $\kappa/|K|$ over σ (Schwartz–Zippel, as for the batching of §4.5.4).

Layer loop. For $j = n, n-1, \dots, 1$, the parties hold a claim $y^{(j)} \stackrel{?}{=} \tilde{\mathbf{L}}^{(j)}(\sigma^{(j)}, \tau^{(j)}, \rho^{(j)})$, initialized with $\sigma^{(n)} := \sigma$, $\tau^{(n)} := \tau$, and $\rho^{(n)}$ empty. Expanding the layer relation,

$$\tilde{\mathbf{L}}^{(j)}(\sigma^{(j)}, \tau^{(j)}, \rho^{(j)}) = \sum_{u \in \mathcal{B}_\kappa} \sum_{x \in \mathcal{B}_\ell} \sum_{i \in \mathcal{B}_{n-j}} \tilde{\mathbf{eq}}_\kappa(\sigma^{(j)}, u) \tilde{\mathbf{eq}}_\ell(\tau^{(j)}, x) \tilde{\mathbf{eq}}_{n-j}(\rho^{(j)}, i) \cdot \tilde{\mathbf{L}}^{(j-1)}(u, x, i, 0) \tilde{\mathbf{L}}^{(j-1)}(u, x, i, 1),$$

the parties run a $(\kappa + \ell + (n-j))$ -round MLE-check (Protocol 2.2) whose composition has individual degree ≤ 2 , exactly as in §4.5.2, producing challenges $(\sigma^{(j-1)}, \tau^{(j-1)}, \rho') \in K^\kappa \times K^\ell \times K^{n-j}$. The prover sends the two child evaluations

$$\beta_b \stackrel{?}{=} \tilde{\mathbf{L}}^{(j-1)}(\sigma^{(j-1)}, \tau^{(j-1)}, \rho', b), \quad b \in \{0, 1\};$$

the verifier checks that $\beta_0 \cdot \beta_1$ equals the MLE-check's reduced evaluation, and a line reduction [GKR15] collapses the pair: the verifier samples $r_b \leftarrow K$ and sets $y^{(j-1)} := (1-r_b)\beta_0 + r_b\beta_1$ and $\rho^{(j-1)} := (\rho', r_b)$, appending the new node coordinate at the end. Neither the round count, nor the message degrees, nor any verifier check depends on the individual depths k_u : the transcript has the shape of an equal-depth batch of depth n .

Claim separation and unpadding. After n layers the parties hold $y^{(0)} \stackrel{?}{=} \tilde{\mathbf{L}}^{(0)}(\sigma^{(0)}, \tau^{(0)}, \rho^{(0)})$ with $\rho^{(0)} \in K^n$. The prover sends the padded per-instance leaf evaluations

$$\alpha'_u \stackrel{?}{=} \tilde{L}'_u(\tau^{(0)}, \rho^{(0)}), \quad u \in \{0, \dots, N-1\},$$

and the verifier checks that they interpolate the folded claim,

$$\sum_{u=0}^{N-1} \tilde{\mathbf{eq}}_\kappa(\sigma^{(0)}, u) \cdot \alpha'_u \stackrel{?}{=} y^{(0)},$$

as in the leaf extrapolation of §4.5.2. Finally the verifier removes the padding using (9): splitting $\rho^{(0)} = (\rho_{<\eta_u}, \rho_{\geq\eta_u}) \in K^{\eta_u} \times K^{k_u}$ and writing $e_u := \tilde{\mathbf{eq}}_{\eta_u}(0^{\eta_u}, \rho_{<\eta_u})$, it computes

$$\alpha_u := 1 + (\alpha'_u - 1) \cdot e_u^{-1},$$

rejecting in the event $e_u = 0$, which requires some coordinate of $\rho^{(0)}$ to equal 1 and so occurs with probability at most $n/|K|$. The protocol output is the family of leaf claims

$$\alpha_u \stackrel{?}{=} \tilde{L}_u(\tau^{(0)}, \rho_{\geq\eta_u}), \quad u \in \{0, \dots, N-1\}.$$

The evaluation points are *nested*: every instance's node point is a suffix of the common point $\rho^{(0)}$ shared by the deepest instances, and all instances share the content point $\tau^{(0)}$.

Completeness and soundness. For an honest prover every check passes and every output claim is true, except that the verifier rejects with probability at most $n/|K|$ (the event $e_u = 0$ above). Conversely, if some input claim $s_u \neq \tilde{p}_u(\tau)$, then except with probability $O(n(\kappa + n + \ell))/|K|$ over the verifier's challenges—the claim combination, the n MLE-checks and line reductions, and the final interpolation check—either the verifier rejects or some output claim $\alpha_u \neq \tilde{L}_u(\tau^{(0)}, \rho_{\geq\eta_u})$ is false, to be caught downstream when the output claims are discharged against committed data.

C.4 Prover Algorithm

The protocol above is verbatim the equal-depth batched product check applied to the padded trees; the content of this subsection is that the prover need never materialize the padded trees. Throughout, fix a layer j and abbreviate $\eta := \eta_u$ and $m := k_u - j$.

Per-instance decomposition. The layer- j MLE-check binds variables from the highest index down: its first $n - j$ rounds bind the node variables from the top—for instance u in a layer $j \leq k_u$, first the $m := k_u - j$ real bits and then the η padding bits—followed by ℓ content rounds and finally κ selector rounds. While the selector variables are unbound, the round polynomial of Protocol 2.2 decomposes by (10) as

$$R^{(i)}(X) = \sum_{u=0}^{N-1} \tilde{\mathbf{eq}}_{\kappa}(\sigma^{(j)}, u) \cdot R_u^{(i)}(X),$$

where $R_u^{(i)}$ is the corresponding round polynomial of the (unbatched) layer MLE-check of the padded instance u ; the final κ selector rounds operate on length- 2^κ tables holding the per-instance child-evaluation pairs, costing $O(2^\kappa)$ operations per layer. It remains to describe $R_u^{(i)}$, in the three phases of a *real* layer ($1 \leq j \leq k_u$) and then in the *padding* layers ($k_u < j \leq n$). Write $\rho^{(j)} = (\rho_{\text{pa}}, \rho_{\text{re}}) \in K^\eta \times K^m$ for the split of the node claim point into padding and real coordinates, and $q_u := \tilde{\mathbf{eq}}_\eta(0^\eta, \rho_{\text{pa}})$.

Real rounds. In the first m rounds, every padding variable of the summation is still unbound. Splitting the node sum on whether the padding bits i' vanish: on the slab $i' = 0^\eta$ the two *sel*-factors of (9) reduce to the unpadded children and the padding equality-weights contribute the constant q_u ; off the slab both factors equal 1 and the weights sum to $1 - q_u$. Hence

$$R_u^{(i)}(X) = 1 + q_u \cdot (\bar{R}_u^{(i)}(X) - 1),$$

where $\bar{R}_u^{(i)}$ is the round polynomial of the *unpadded* layer- j MLE-check of instance u , run on the claim point $(\tau^{(j)}, \rho_{\text{re}})$ with the shared transcript challenges. The correction is an affine map of the message coefficients, costing $O(1)$ per round, and it commutes with the verifier's per-round consistency check.

Padding rounds. Let $r_{\text{re}} \in K^m$ denote the real-node challenges just bound, and let $g_{u,b} \in K^{\mathcal{B}_\ell}$, for $b \in \{0, 1\}$, be the partially specialized children

$$g_{u,b}[x] := \tilde{L}_u^{(j-1)}(x, (r_{\text{re}}, b)), \quad x \in \mathcal{B}_\ell,$$

which are exactly the tables the unpadded prover holds after its node rounds. At the phase boundary the prover computes three scalars: the running unpadded claim $z_u := \sum_{x \in \mathcal{B}_\ell} \tilde{\mathbf{eq}}_\ell(\tau^{(j)}, x) g_{u,0}[x] g_{u,1}[x]$ (available from the message chain), the evaluation sum $S_u := \tilde{g}_{u,0}(\tau^{(j)}) + \tilde{g}_{u,1}(\tau^{(j)})$ (two $O(2^\ell)$ evaluations), and $Q_u := z_u - S_u + 1$. In padding round $a \in \{0, \dots, \eta - 1\}$, define

$$e_{u,a} := \prod_{c=0}^{\eta-a-2} \tilde{\mathbf{eq}}(0, \rho_{\text{pa},c}), \quad E_{u,a}(X) := \tilde{\mathbf{eq}}(0, X) \cdot \prod_{a'=0}^{a-1} \tilde{\mathbf{eq}}(0, r_{a'}^{\text{pa}}),$$

the equality-weights of the still-unbound and already-bound padding coordinates respectively, with $r_{a'}^{\text{pa}}$ the challenge of padding round a' . Splitting the residual node sum on whether the unbound padding bits vanish, expanding the product $\prod_b (1 + (g_{u,b}[x] - 1) E_{u,a}(X))$ on the slab, and summing the content index against its equality-weights yields the closed form

$$R_u^{(m+a)}(X) = 1 - e_{u,a} + e_{u,a} \cdot \left[1 + (S_u - 2) E_{u,a}(X) + Q_u E_{u,a}(X)^2 \right] :$$

the coefficients of E^0, E^1, E^2 are the equality-weighted moments 1, $S_u - 2$, and Q_u of the children. Each padding round costs $O(1)$ operations, only $e_{u,a}$ and $E_{u,a}$ updating between rounds.

Content rounds. Once all node variables are bound, the factor E collapses to the constant $e_u^* := \prod_{a=0}^{\eta-1} \tilde{\mathbf{eq}}(0, r_a^{\text{pa}})$, and the residual summand is the bivariate product of the transformed tables

$$\tilde{g}_{u,b} := \text{sel}(e_u^*, g_{u,b}) = 1 + (g_{u,b} - 1) \cdot e_u^*.$$

The remaining ℓ content rounds therefore coincide with those of the unpadded MLE-check run on tables obtained by one affine pass ($O(2^\ell)$ operations), with no further per-round correction, and the layer's child evaluations are the $\text{sel}(e_u^*, \cdot)$ -images of the unpadded ones. One checks directly that the messages of the three phases chain through the verifier's per-round consistency checks; for instance, at the real-to-padding boundary both the last real-round message and the first padding-round message reduce the claim to $1 + q_u(z_u - 1)$, using $(S_u - 2) + Q_u = z_u - 1$.

Padding layers. For $k_u < j \leq n$ the layer is $\text{OnePad}_{n-j}(p_u)$ and its $b = 1$ child is identically 1 (Lemma C.1 and its proof), so the composition is *affine* in p_u and all messages have degree ≤ 1 . The three phases specialize with $m = 0$ and padding count $n - j$ in place of η : there are no real rounds; the padding rounds open the layer with the children $g_{u,0} = p_u$ and $g_{u,1}$ the all-ones table, whence $z_u = \tilde{p}_u(\tau^{(j)})$ (the instance’s running claim), $S_u = \tilde{p}_u(\tau^{(j)}) + 1$, and $Q_u = 0$; and the content rounds run the linear MLE-check on the transformed table $\text{sel}(e_u^*, p_u)$, a fresh length- 2^ℓ copy per padding layer ($O(2^\ell)$ operations). The child evaluations are $\beta_0 = 1 + (\tilde{p}_u(\tau^{(j-1)}) - 1) \cdot \tilde{\text{eq}}_{n-j}(0^{n-j}, \rho')$ and $\beta_1 = 1$.

Cost. Summing over the layers, instance u costs its unpadded prover— $O(2^{k_u+\ell})$ operations for the whole tree—plus $O(2^\ell)$ per layer (the boundary evaluations and table transformation in a real layer; the fresh table of p_u in a padding layer) and $O(1)$ per round, for a padding overhead of $O(n \cdot 2^\ell)$. The batch as a whole additionally pays $O(2^\kappa)$ per layer for the selector rounds.

D Zero-Knowledge of the Sumcheck Machinery

We prove Theorem 6.3. Both the success of the masked multilinear-extension check (§6.3.2) and the failure of Libra masking in characteristic 2 (§6.3.1) follow from the distribution of the prover’s masking messages.

The masking messages. Fix the challenges τ, r . The masking messages $-s_g$ together with, for each round i , the degree- ≥ 1 coefficients of $R_g^{(i)}$ that enter the sent $R^{(i)} = R_f^{(i)} + \alpha R_g^{(i)}$ —number $1 + nd$ and form a fixed K -linear image of the coefficient vector $c_g = (g_c, (g_{j,k})_{j < n, k < d})$ of the mask.

Lemma D.1. *For the multilinear-extension check this map is a bijection of K^{1+nd} , in every characteristic; for the bare sumcheck it is a bijection if and only if $\text{char}(K) \neq 2$. Whenever it is a bijection and g' is uniform, the masking messages are uniform on K^{1+nd} .*

Proof. By the closed form of §6.3.2, $R_g^{(i)}(X) = (g_c + \sum_{j > n-i-1} g_j(r_j)r_j + \sum_{j < n-i-1} \tau_j g_j(1)) + g_{n-i-1}(X)X$, so the degree- ≥ 1 coefficients of $R_g^{(i)}$ are exactly the coefficients $g_{n-i-1,0}, \dots, g_{n-i-1,d-1}$ of the fresh univariate g_{n-i-1} . Reading the round messages in order $i = 0, \dots, n-1$ recovers $g_{n-1}, g_{n-2}, \dots, g_0$ in turn, after which $s_g = g_c + \sum_j \tau_j g_j(1)$ recovers g_c ; the map is thus a bijection, with no condition on the challenges. For the bare sumcheck (§6.3.1) the corresponding coefficients are $2^{n-i-1}g_{n-i-1,k}$, so the round- i block is this same bijection scaled by 2^{n-i-1} —invertible exactly when $2^{n-i-1} \neq 0$ for every i , i.e. when $\text{char}(K) \neq 2$. In characteristic 2 the blocks with $i < n-1$ vanish, the rank falls to $1 + d$, and the non-constant coefficients of the first $n-1$ round polynomials lie outside the image. A bijection carries the uniform distribution on g to the uniform distribution on the messages. $\square$

Simulator.

Proof of Theorem 6.3. The simulator $\mathcal{S}$ receives the public claim (τ, s_f) and the reduced value f^* —in the IronSpartan composition f^* is itself furnished by the surrounding honest-verifier simulator (§7)—and has no access to f or the witness. It runs:

1. Sample the oracle $\pi_g = g' \leftarrow K^{\mathcal{B}_{\nu+\delta}}$ uniformly, as the honest prover does.
2. Sample $s_g \leftarrow K$; receive α ; set $s^{(0)} := s_f + \alpha s_g$.
3. For $i = 0, \dots, n-1$: sample the transmitted coefficients of $R^{(i)}$ uniformly; let the verifier recover the remaining coefficient and sample r_{n-i-1} ; set $s^{(i+1)} := R^{(i)}(r_{n-i-1})$.
4. Set $g^* := (s^{(n)} - f^*)/\alpha$ and program the response to the single query $\langle g', \text{libra-eval}_r \rangle$ to equal g^* .

In an honest run the messages are $s^{(0)} = s_f + \alpha s_g$ and $R^{(i)} = R_f^{(i)} + \alpha R_g^{(i)}$. Conditioned on the challenges, f fixes s_f and the $R_f^{(i)}$, while the mask supplies s_g and the $R_g^{(i)}$; by Lemma D.1 these are jointly uniform over the uniform choice of g' , so for $\alpha \neq 0$ the sent $(s^{(0)}, (R^{(i)})_i)$ are uniform, matching $\mathcal{S}$ exactly. The lone opening of π_g lands at the point encoded by libra-eval_r , where $\mathcal{S}$ programs the

response to satisfy $f^* = s^{(n)} - \alpha g^*$; as g' is committed and opened nowhere else, this is undetectable. The two transcripts coincide except on the event $\alpha = 0$, of probability $1/|K|$, giving statistical distance $O(1)/|K|$. The same argument fails for the bare sumcheck in characteristic 2, where by Lemma D.1 the masking messages are confined to a proper subspace and so cannot match the uniform simulation. $\square$

References

- [ACFY25] Gal Arnon, Alessandro Chiesa, Giacomo Fenzi, and Eylon Yogev. “WHIR: Reed–Solomon Proximity Testing with Super-Fast Verification”. In: *Advances in Cryptology – EUROCRYPT 2025*. Ed. by Serge Fehr and Pierre-Alain Fouque. Cham: Springer Nature Switzerland, 2025, pp. 214–243. ISBN: 978-3-031-91134-7.
- [BCS16] Eli Ben-Sasson, Alessandro Chiesa, and Nicholas Spooner. “Interactive Oracle Proofs”. In: *International Conference on Theory of Cryptography*. Vol. 9986. Berlin, Heidelberg: Springer-Verlag, 2016, pp. 31–60. ISBN: 978-3-662-53644-5. DOI: [10.1007/978-3-662-53644-5_2](https://doi.org/10.1007/978-3-662-53644-5_2).
- [Ben+19] Eli Ben-Sasson, Alessandro Chiesa, Michael Riabzev, Nicholas Spooner, Madars Virza, and Nicholas P. Ward. “Aurora: Transparent Succinct Arguments for R1CS”. In: *Advances in Cryptology – EUROCRYPT 2019*. Berlin, Heidelberg: Springer-Verlag, 2019, pp. 103–128. ISBN: 978-3-030-17652-5. DOI: [10.1007/978-3-030-17653-2_4](https://doi.org/10.1007/978-3-030-17653-2_4).
- [Ben+90] M. Ben-Or, O. Goldreich, S. Goldwasser, J. Håstad, J. Kilian, S. Micali, and P. Rogaway. “Everything provable is provable in zero-knowledge”. In: *Proceedings on Advances in Cryptology*. CRYPTO ’88. Santa Barbara, California, USA: Springer-Verlag, 1990, pp. 37–56. ISBN: 0387971963.
- [CFS17] Alessandro Chiesa, Michael A. Forbes, and Nicholas Spooner. *A Zero Knowledge Sumcheck and its Applications*. Cryptology ePrint Archive, Paper 2017/305. 2017. URL: <https://eprint.iacr.org/2017/305>.
- [CFW26] Alessandro Chiesa, Giacomo Fenzi, and Guy Weissenberg. *Zero-Knowledge IOPPs for Constrained Interleaved Codes*. Cryptology ePrint Archive, Paper 2026/391. 2026. URL: <https://eprint.iacr.org/2026/391>.
- [DG25] Benjamin E. Diamond and Angus Gruen. “Proximity Gaps in Interleaved Codes”. In: *IACR Communications in Cryptology* 1.4 (Jan. 13, 2025). ISSN: 3006-5496. DOI: [10.62056/a01jbkrz](https://doi.org/10.62056/a01jbkrz).
- [DHRR26] Rahul Dalal, Tamir Hemo, Eugene Rabinovich, and Ron Rothblum. *VEIL: Lightweight Zero-Knowledge for Hash-Based Multilinear Proof Systems*. Cryptology ePrint Archive, Paper 2026/683. 2026. URL: <https://eprint.iacr.org/2026/683>.
- [Dia25] Benjamin E. Diamond. *Zero-Knowledge Polynomial Commitment in Binary Fields*. Cryptology ePrint Archive, Paper 2025/1015. 2025. URL: <https://eprint.iacr.org/2025/1015>.
- [DP24] Benjamin E. Diamond and Jim Posen. *Polylogarithmic Proofs for Multilinears over Binary Towers*. Cryptology ePrint Archive, Paper 2024/504. 2024. URL: <https://eprint.iacr.org/2024/504>.
- [DP25] Benjamin E. Diamond and Jim Posen. “Succinct Arguments over Towers of Binary Fields”. In: *Advances in Cryptology – EUROCRYPT 2025*. Ed. by Serge Fehr and Pierre-Alain Fouque. Cham: Springer Nature Switzerland, 2025, pp. 93–122. ISBN: 978-3-031-91134-7.
- [DT24] Quang Dao and Justin Thaler. *Constraint-Packing and the Sum-Check Protocol over Binary Tower Fields*. Cryptology ePrint Archive, Paper 2024/1038. 2024. URL: <https://eprint.iacr.org/2024/1038>.
- [Fs24] Matteo Frigo and abhi shelat. *Anonymous credentials from ECDSA*. Cryptology ePrint Archive, Paper 2024/2010. 2024. URL: <https://eprint.iacr.org/2024/2010>.
- [GKR15] Shafi Goldwasser, Yael Tauman Kalai, and Guy N. Rothblum. “Delegating Computation: Interactive Proofs for Muggles”. In: *Journal of the ACM* 62.4 (Sept. 2015). ISSN: 0004-5411. DOI: [10.1145/2699436](https://doi.org/10.1145/2699436).

- [Gru24] Angus Gruen. *Some Improvements for the PIOP for ZeroCheck*. Cryptology ePrint Archive, Paper 2024/108. 2024. URL: <https://eprint.iacr.org/2024/108>.
- [Hab24] Ulrich Haböck. *Basefold in the List Decoding Regime*. Cryptology ePrint Archive, Paper 2024/1571. 2024. URL: <https://eprint.iacr.org/2024/1571>.
- [HMH] Adrian Hamelink, Andrew Milson, and Ulrich Haböck. *ZK13: Lifted FRI: A uniform multi-domain polynomial commitment scheme*. YouTube. URL: <https://www.youtube.com/watch?v=p6Z3WjRcZD0>.
- [Hu+25] Yuncong Hu, Chongrong Li, Zhi Qiu, Tiancheng Xie, Yue Ying, Jiaheng Zhang, and Zhenfei Zhang. *GKR for Boolean Circuits with Sub-linear RAM Operations*. Cryptology ePrint Archive, Paper 2025/717. 2025. URL: <https://eprint.iacr.org/2025/717>.
- [LCH14] Sian-Jheng Lin, Wei-Ho Chung, and Yung-Hsiang S. Han. “Novel Polynomial Basis and Its Application to Reed–Solomon Erasure Codes”. In: *IEEE 55th Annual Symposium on Foundations of Computer Science*. 2014, pp. 316–325. DOI: [10.1109/FOCS.2014.41](https://doi.org/10.1109/FOCS.2014.41).
- [LFKN92] Carsten Lund, Lance Fortnow, Howard Karloff, and Noam Nisan. “Algebraic Methods for Interactive Proof Systems”. In: *Journal of the ACM* 39.4 (Oct. 1992), pp. 859–868. DOI: [10.1145/146585.146605](https://doi.org/10.1145/146585.146605). URL: <https://doi.org/10.1145/146585.146605>.
- [Set20] Srinath Setty. “Spartan: Efficient and General-Purpose zkSNARKs Without Trusted Setup”. In: *Advances in Cryptology – CRYPTO 2020*. Ed. by Daniele Micciancio and Thomas Ristenpart. Cham: Springer International Publishing, 2020, pp. 704–737. ISBN: 978-3-030-56877-1. DOI: [10.1007/978-3-030-56877-1_25](https://doi.org/10.1007/978-3-030-56877-1_25).
- [Tha13] Justin Thaler. “Time-Optimal Interactive Proofs for Circuit Evaluation”. In: *Advances in Cryptology – CRYPTO 2013*. Ed. by Ran Canetti and Juan A. Garay. Berlin, Heidelberg: Springer Berlin Heidelberg, 2013, pp. 71–89. ISBN: 978-3-642-40084-1.
- [Tha22] Justin Thaler. *Proofs, Arguments and Zero-Knowledge*. Vol. 4. Foundations and Trends in Privacy and Security 2–4. now publishers, 2022.
- [Xie+19] Tiacheng Xie, Jiaheng Zhang, Yupeng Zhang, Charalampos Papamanthou, and Dawn Song. “Libra: Succinct Zero-Knowledge Proofs with Optimal Prover Computation”. In: *Advances in Cryptology – CRYPTO 2019*. Berlin, Heidelberg: Springer-Verlag, 2019, pp. 733–764. ISBN: 978-3-030-26953-1. DOI: [10.1007/978-3-030-26954-8_24](https://doi.org/10.1007/978-3-030-26954-8_24).
- [ZCF24] Hadas Zeilberger, Binyi Chen, and Ben Fisch. “BaseFold: Efficient Field-Agnostic Polynomial Commitment Schemes from Foldable Codes”. In: *Advances in Cryptology – CRYPTO 2024*. Berlin, Heidelberg: Springer-Verlag, 2024, pp. 138–169. ISBN: 978-3-031-68402-9. DOI: [10.1007/978-3-031-68403-6_5](https://doi.org/10.1007/978-3-031-68403-6_5).